

Restoran POS V3 Audit Rapor Paketi

Oluşturulma: 14 Nisan 2026 19:49 Kaynak: docs/audit Format: Birlesik PDF paketi

00 - Genel Repo Denetimi

Tarih: 2026-04-13 Kapsam: Genel repo taraması, mimari çıkarımı, kritik akışların ilk değerlendirmesi. Sınır: Bu raporda kod davranışı değiştirilmedi; yalnızca denetim dokümanı oluşturuldu.

1. Proje özeti

Bu repo, restoran POS kullanım senaryosu için masa, sipariş, mutfak, ödeme, müşteri, stok, rezervasyon, caller ID ve yazıcı akışlarını kapsayan masaüstü/web hibrit bir üründür.

Ana mimari:

- Frontend: React 18 + Vite. Giriş dosyaları `client/src/main.jsx` ve `client/src/App.jsx`. Route/top-level ekran orkestrasyonu `client/src/App.jsx:46` sonrası tek dosyada yoğunlaşmış.
- Backend: Node.js + Express + SQLite. API entrypoint `server/index.js`; route kayıtları `server/index.js:111-127`.
- Veritabanı: `better-sqlite3`, WAL modu ve foreign key açık. Kanıt: `server/config/database.js:11-16`.
- Desktop shell: Electron, Express backend'i child process olarak başlatıyor. Kanıt: `electron/main.cjs:1-5`, child env kurulumu `electron/main.cjs:170-204`.
- Yerel donanım köprüsü: `store-bridge`, API'den print job poll ediyor ve USB/network yazıcıya ESC/POS buffer gönderiyor. Kanıt: `store-bridge/jobs/poller.js:5-13`, `store-bridge/jobs/poller.js:72-99`.
- Yazdırma modeli: POS backend `print_jobs` üretir; StoreBridge claim edip fiziksel yazıcıya yollar. Kanıt: `server/services/printJobs.js:50-82`, `store-bridge/jobs/poller.js:19-31`.

Kritik iş akışları:

- Auth: JWT bearer token, kullanıcı/rol DB'den okunuyor. Kanıt: `server/middleware/auth.js:5-33`.
- Business scoping: route middleware `req.businessId` atıyor. Kanıt: `server/middleware/auth.js:45-49`.
- Sipariş: `server/routes/orders.js` hem listeleme, paket sipariş, mutfak job üretimi, item lifecycle gibi çok sayıda davranışı taşıyor. Kanıt: `server/routes/orders.js:1-18`, `server/routes/orders.js:52-67`.
- Ödeme: tekil ve parçalı ödeme, masa kapatma, müşteri istatistiği ve fiş job üretimi aynı route dosyasında. Kanıt: `server/routes/payments.js:164-183`, `server/routes/payments.js:197-240`.
- Yazıcı kod sayfası: StoreBridge renderer içinde PC857 ve Windows-1254 desteği var. Kanıt: `store-bridge/printers/renderers.js:16-28`, `store-bridge/printers/renderers.js:38-44`, `store-bridge/printers/renderers.js:115-154`, `store-bridge/printers/renderers.js:278-287`.

2. Güçlü yönler

- P1 olumlu: Yazdırma, doğrudan HTTP mock'tan job queue + bridge modeline taşınmış görünüyor. `client/src/services/api.js:195-199` eski `/api/print/*` uçlarının deprecated olduğunu açıkça söylüyor; aktif akışın `print_jobs` + StoreBridge olduğu not edilmiş. Bu doğru mimari yöndür.
- P1 olumlu: Print job idempotency düşünülmüş. `server/services/printJobs.js:63-80` `INSERT OR IGNORE` ve `idempotency_key` kullanıyor; mutfak job'larında hash tabanlı anahtar var (`server/services/printJobs.js:168-173`).
- P1 olumlu: Backend startup'ta production JWT secret zorunluluğu var. `server/config/index.js:22-27` production'da fallback secret ile açılışı engelliyor.

- P1 olumlu: API 404'leri JSON dönüyor; SPA fallback'ın API hatalarını HTML'e çevirmemesi için özel blok var.
Kanıt: `server/index.js:129-139`.
- P2 olumlu: Server test klasörü mevcut ve iş kritik alanlardan bazılarını kapsıyor: auth middleware, validation, migration idempotency, order transaction, printer encoding, print job idempotency ve integration testleri.
Kanıt: `server/tests/*.test.js`.
- P2 olumlu: Client build üretilebiliyor. `npm run build --prefix client` başarılı bitti; ancak bundle boyutu uyarısı var.

3. Kritik problemler

P0 - Net P0 bulunmadı

Bu ilk genel taramada uygulamayı doğrudan veri kaybına, yetkisiz toplu erişime veya açılışın tamamen imkansız hale gelmesine götüren kanıtlanmış P0 saptanmadı. Ancak aşağıdaki P1'ler üretim kalitesi için sert biçimde ele alınmalı.

P1 - Server test komutu çalışmıyor

Kanıt:

- Root test script'i server testine delegasyon yapıyor: `package.json:13`.
- Server test script'i `vitest run`: `server/package.json:9`.
- `vitest` devDependency olarak tanımlı: `server/package.json:27-30`.
- Çalıştırılan komut sonucu: `npm test --prefix server -> 'vitest' is not recognized as an internal or external command`.

Etki: Repo testleri varmış gibi görünüyor ama mevcut çalışma ortamında test kapısı kırık. Bu, yapılan her düzeltmenin regresyon riskini yükseltir. Özellikle POS, ödeme ve yazdırma gibi para/donanım akışlarında kabul edilemez.

Öneri: `server/node_modules` bağımlılıklarını temiz ve eksiksiz kur; CI/yerel doğrulama için root `npm test` komutunu güvenilir hale getir. Ek olarak lockfile ve workspace stratejisini sadeleştir; şu anda root, client, server, store-bridge ayrı package-lock ile yönetiliyor.

P1 - Monolitik dosyalar ürün kalitesini düşürüyor

Kanıtlanan büyük dosyalar:

- `client/src/components/orders/OrderScreen.jsx` : 1428 satır.
- `store-bridge/printers/renderers.js` : 1271 satır.
- `client/src/components/settings/PrinterDetailPage.jsx` : 972 satır.
- `electron/main.cjs` : 860 satır.
- `server/migrations/run.js` : 673 satır.
- `server/routes/orders.js` : 627 satır.
- `client/src/components/payments/PaymentScreen.jsx` : 497 satır.
- `server/routes/payments.js` : 389 satır.

Etki: Bu dosyalar tek sorumluluk ilkesini zorluyor. POS ekranı, ödeme, yazıcı ayarı ve ESC/POS rendering gibi en riskli alanlar aynı zamanda en büyük ve en kırılgan dosyalar. Yeni özellik ekleme maliyeti artar; küçük düzeltmeler

yan etki üretir.

Öneri: Öncelik sırası: `OrderScreen.jsx`, `renderers.js`, `PrinterDetailPage.jsx`, `electron/main.cjs`. Her biri için önce davranışı sabitleyen test/snapshot, sonra küçük servis/hook/component ayrıştırması yapılmalı.

P1 - Yazdırma karakter seti karmaşıklığı tek dosyada yoğun ve cihaz bağımlı

Kanıt:

- PC857 ESC t kod sayfası sabiti: `store-bridge/printers/renderers.js:16-22`.
- Windows-1254 ESC t sabiti: `store-bridge/printers/renderers.js:23-28`.
- `encodeWin1254` TL fallback ile çalışıyor: `store-bridge/printers/renderers.js:38-44`.
- PC857 manuel tablo içeriyor: `store-bridge/printers/renderers.js:67-106`.
- Runtime encoding mode çözümü `win1254` default: `store-bridge/printers/renderers.js:278-287`.
- Bridge, receipt veya win1254 için `escT = 32` ve `skipPhoenixCmd = true` zorluyor: `server/routes/bridge.js:47-53`.

Etki: Türkçe karakter bozulması tek bir yerde değil; backend printer option normalizasyonu, bridge renderer, fiziksel yazıcı firmware'i ve env/pos-config override zincirinin birleşiminde oluşabilir. Kodda farklı cihazlar için doğru düşünülmüş parçalar var, fakat tek büyük dosyada ve env tabanlı koşullarla yönetildiği için hata ayıklama maliyeti yüksek.

Öneri: Yazdırma için ayrı bir `encoding` modülü çıkar; PC857/WIN1254 encode testlerini hem byte snapshot hem ESC/POS komut snapshot olarak büyüt. Her fiziksel yazıcı profili için `encodingMode`, `escT`, `skipPhoenixCmd`, `lineWidth` değerleri tek profil objesinde tutulmalı.

P1 - Paketleme çıktıları ve node_modules repo içinde taramayı kırıyor

Kanıt:

- Repo kökünde `dist-electron`, `node_modules`, `client/node_modules`, `server/node_modules`, `store-bridge/node_modules` mevcut.
- Dosya tarama sırasında `server/node_modules/restoran-pos/...` altında recursive `file:..` bağlantıları nedeniyle PowerShell `Could not find a part of the path ... node_modules\restoran-pos\...` hataları verdi.
- `client/package.json:17`, `server/package.json:22`, `store-bridge/package.json` içinde `restoran-pos: file:..` bağımlılığı var.

Etki: Yerel paket bağımlılığı repo kökünü alt paketlerin `node_modules` içine tekrar taşıyor. Bu, taramayı, paketlemeyi, antivirüs/Windows path davranışını ve CI sürelerini gereksiz kırılgan hale getirir.

Öneri: npm workspaces veya açık paket sınırı kullan. Alt paketlerin root projeyi dependency olarak çekmesi gerçekten gerekiyorsa bunun nedenini belgeleyip publish/workspace protokolüne taşı.

P1 - Frontend bundle uyarısı: tek parça uygulama büyüyor

Kanıt:

- `npm run build --prefix client` başarılı.
- Vite uyarısı: minified chunk 500 kB üstü.
- Üretilen ana JS: `assets/index-CwuMQpUU.js` 948.07 kB, gzip 264.37 kB.

- XLSX chunk: `assets/xlsx-D_018YDs.js` 429.03 kB, gzip 143.08 kB.

Etki: POS genelde düşük donanımlı Windows kasalarda çalışır. Büyük ilk yükleme, Electron içinde de startup ve memory baskısı yaratır.

Öneri: Route bazlı lazy loading; müşteri import/export için `xlsx` modülünü yalnız ilgili ekranda dynamic import ile yükle; rapor/chart ekranlarını ayrı chunk'a ayır.

4. Amatör görünen alanlar

- P1: `OrderScreen.jsx` 1428 satır. Sipariş ekranı bir ekran bileşeni olmaktan çıkıp iş kuralı, navigasyon, state, API orchestration ve UI davranışını aynı yerde toplama riski taşıyor. Bu profesyonel POS ürününde bakım sorunudur.
- P1: `store-bridge/printers/renderers.js` 1271 satır. Encoding, layout, ESC/POS komutları, metin sarma, diagnostik ve template render aynı dosyada. Yazıcı sorunu çıktığında nokta atışı izolasyon zorlaşır.
- P1: `electron/main.cjs` 860 satır. Config okuma, JWT üretimi, DB taşıma, backend process, bridge process, caller ID helper ve pencere yönetimi aynı dosyada. Masaüstü lifecycle hataları bu yapıda pahalı olur.
- P2: `client/src/App.jsx:46-245` top-level route, auth role, modal state, quick payment state ve navigation fallback'lerini aynı komponentte yönetiyor. Şimdilik çalışabilir ama büyüme yönü yanlış.
- P2: API client tek sınıfta tüm domainleri topluyor. `client/src/services/api.js:77-210` auth, masa, ürün, sipariş, ödeme, müşteri, caller ID, rapor, stok, waiter call ve print uçlarını aynı dosyaya yığmış.
- P2: Root scriptlerde Windows `.bat`, cross-env, Electron paketleme ve server/client komutları karışık. `package.json:8-37` tek dosyada çok sayıda operasyonel sorumluluk taşıyor.

5. Teknik borç alanları

- P1: Test altyapısı yerelde kırık. Test dosyaları var ama `vitest` bulunmadığı için çalıştırılmıyor.
- P1: Alt paketlerde `file:..` bağımlılığı tarama ve dependency graph üzerinde recursive karmaşa yaratıyor.
- P1: Yazıcı profilleri env/JSON/DB seçeneklerine dağılmış. Kanıt: Electron bridge env üretimi `electron/main.cjs:211-226`, bridge normalize `server/routes/bridge.js:28-55`, renderer çözümleme `store-bridge/printers/renderers.js:253-287`.
- P2: Frontend test script'i yok. `client/package.json:6-9` yalnız `dev`, `build`, `preview` içeriyor.
- P2: Lint/typecheck script'i yok. Root scripts `package.json:8-37`, client scripts `client/package.json:6-9`, server scripts `server/package.json:6-10` içinde lint veya typecheck bulunmuyor.
- P2: Migration dosyası 673 satır ve hem schema hem veri düzeltme mantığı taşıyor. Uzun vadede migration sürümleme ve rollback disiplini zayıflar.
- P2: Runtime config kaynakları çok: `.env`, `pos-config.json`, Electron-generated env, bridge env, DB settings. Bu esneklik iyi ama tek doğruluk kaynağı belirsiz.

6. Hızlı kazanımlar

- P1: Server test kurulumunu düzelt: `npm install --prefix server`, ardından `npm test --prefix server`. CI yoksa minimum yerel gate olarak belgeye ekle.
- P1: Yazıcı encoding testlerini genişlet: `ÇĞİÖŞÜ çğİöşü`, TL sembolü, uzun ürün adı, sağ/sol hizalama ve farklı `escT` profilleri.

- P1: `client/src/components/orders/OrderScreen.jsx` için ilk parçalama: API orchestration hook, order total hesapları utility, UI alt komponentleri.
- P1: `store-bridge/printers/renderers.js` için ilk parçalama: `encoding`, `layout`, `escposCommands`, `templates`.
- P2: Vite code splitting: `xlsx`, reports/charts, settings/printer pages lazy load.
- P2: `docs/audit` altında her derin konu için ayrı rapor: `01-printing-flow.md`, `02-order-payment-integrity.md`, `03-frontend-state.md`, `04-packaging-electron.md`.

7. Uygulanmış düzeltmeler

- `docs/audit/00-overall-audit.md` oluşturuldu.
- Kod davranışı değiştirilmedi.
- Düşük riskli kod düzeltmesi uygulanmadı; çünkü bu turun açık kapsamı "ilk iş olarak sadece genel repo taraması" idi.

8. Çalıştırılan komutlar ve sonuçlar

- `Get-ChildItem -Force`: repo kök yapısı çıkarıldı. Ana klasörler: `client`, `server`, `electron`, `store-bridge`, `scripts`, `tools`, `resources`, `dist-electron`, `data`.
- `rg --files`: çalışmadı. Hata: WindowsApps içindeki `rg.exe` için erişim engellendi. Alternatif olarak PowerShell `Get-ChildItem` kullanıldı.
- `git status --short --branch`: branch `main...origin/main`. Mevcut kullanıcı/depo değişiklikleri görüldü: silinmiş `Restoran-POS-v3-Ilerleme-Raporu.html`, untracked `ANALIZ_RAPORU_2026.html`, untracked PDF. Bu dosyalara dokunulmadı.
- `Get-Content package.json`, `client/package.json`, `server/package.json`, `store-bridge/package.json`: script ve dependency yüzeyi incelendi.
- `Get-Content README.md -TotalCount 220`: ürün amacı, modlar ve kurulum bilgileri incelendi.
- `Get-Content server/index.js`, `client/src/App.jsx`, `client/src/services/api.js`, `server/config/database.js`: ana entrypoint ve request akışı incelendi.
- `Get-Content server/routes/orders.js`, `server/routes/payments.js`, `server/services/printJobs.js`, `server/routes/bridge.js`, `store-bridge/jobs/poller.js`, `store-bridge/printers/renderers.js`: sipariş, ödeme ve yazdırma akışı örnekleri.
- `npm test --prefix server`: başarısız. `vitest` komutu bulunamadı.
- `npm run build --prefix client`: başarılı. Vite build tamamlandı; büyük chunk uyarısı verdi.

9. Sonraki önerilen adımlar

1. `01-printing-flow.md`: Frontend'de yazdırma tetikleyen bütün UI noktaları, backend job üretimi, bridge claim/send akışı ve Türkçe karakter bozulma noktaları uçtan uca çıkarılmalı. Bu, ilk kullanıcı fotoğrafındaki gerçek problemi hedefler.
2. `02-order-payment-integrity.md`: Sipariş oluşturma, item ekleme/iptal/azaltma, ödeme kapatma, masa temizleme ve müşteri istatistiği transactional bütünlük açısından denetlenmeli.

3. `03-frontend-state.md`: `OrderScreen.jsx`, `PaymentScreen.jsx`, `App.jsx` state ve navigation davranışları çıkarılmalı; amatör/kırılgan UI koşulları listelenmeli.
4. `04-test-and-build-health.md`: Server test kurulumu onarılmalı, frontend test eksikliği ve CI gate önerisi raporlanmalı.
5. `05-electron-packaging.md`: `file:..` dependency döngüsü, dist-electron kaynak kopyaları, native sqlite rebuild ve bridge/caller ID process lifecycle denetlenmeli.

00a - Önceliklendirilmiş Eylem Planı

Tarih: 2026-04-13 Temel rapor: `docs/audit/00-overall-audit.md` Kapsam: P0/P1/P2 audit maddelerini eyleme dönüştürme. Bu dosya plan dokümanıdır; kod değişikliği içermez.

Öncelik özeti

- P0: Doğrulanmış P0 yok. İlk genel taramada doğrudan veri kaybı, yetkisiz toplu erişim veya uygulamanın tamamen açılmaması seviyesinde kanıt bulunmadı.
- P1: Test kapısının kırık olması, yazdırma/encoding karmaşıklığı, monolitik kritik dosyalar, paketleme/node_modules bağımlılık döngüsü ve frontend bundle büyümesi.
- P2: Frontend test/lint/typecheck eksikleri, API client ve App seviyesinde sorumluluk yığılması, migration/runtime config disiplin borcu, script karmaşası.

Ürün / İş Kuralı

P1 - Sipariş ve ödeme iş kuralları büyük route dosyalarında yoğun

- Sorun özeti: Sipariş oluşturma, item lifecycle, paket sipariş, mutfak job üretimi, ödeme kapatma ve masa temizleme gibi iş kritik kurallar büyük route dosyalarında toplanmış.
- Kullanıcıya/işletmeye etkisi: Hatalı masa kapatma, eksik mutfak fişi, yanlış ödeme durumu veya müşteri istatistiği gibi doğrudan operasyonel problemler çıkabilir.
- Teknik kök neden: `server/routes/orders.js` 627 satır, `server/routes/payments.js` 389 satır; route, transaction, domain kuralı ve yan etki aynı katmanda.
- Önerilen çözüm: Önce davranışı kapsayan integration/unit testleri güçlendir; sonra sipariş/ödeme domain servisleri çıkar. Route dosyaları yalnız HTTP doğrulama ve servis çağırısı yapmalı.
- Risk seviyesi: P1.
- Uygulama zorluğu: Orta-yüksek.

P2 - Runtime config kaynakları iş davranışını da etkiliyor

- Sorun özeti: `.env`, `pos-config.json`, Electron env üretimi, bridge env ve DB settings aynı davranış alanlarını etkiliyor.
- Kullanıcıya/işletmeye etkisi: Aynı işletmede farklı makinelerde farklı yazıcı/port/token/encoding davranışı oluşabilir; destek maliyeti artar.
- Teknik kök neden: Config tek doğruluk kaynağına bağlanmamış; bridge ayarları `electron/main.cjs`, `server/routes/bridge.js`, `store-bridge/printers/renderers.js` boyunca yayılmış.
- Önerilen çözüm: Config precedence dokümanı ve validasyon katmanı ekle. Yazıcı profilini tek nesne olarak normalize et.
- Risk seviyesi: P2.
- Uygulama zorluğu: Orta.

P1 - Büyük frontend bundle düşük donanımlı POS cihazlarını zorlar

- Sorun özeti: Client build başarılı ama ana chunk 948.07 kB minified; XLSX chunk 429.03 kB.
- Kullanıcıya/işletmeye etkisi: Electron veya eski kasalarda açılış süresi ve bellek tüketimi artar; yoğun servis saatlerinde yavaşlık algısı oluşur.
- Teknik kök neden: Route bazlı code splitting yok; ağır ekranlar ve kütüphaneler ilk yüklemeye yakın konumda.
- Önerilen çözüm: React lazy/Suspense ile route bazlı bölme; `xlsx`, reports/charts ve printer settings gibi ağır alanları dynamic import ile ayır.
- Risk seviyesi: P1.
- Uygulama zorluğu: Orta.

P2 - App seviyesinde modal, role, route ve navigation state yığılmış

- Sorun özeti: `client/src/App.jsx` route tanımı, auth role kararları, sidebar, ödeme modal state'i ve navigation fallback'lerini aynı bileşende taşıyor.
- Kullanıcıya/işletmeye etkisi: Ödeme sonrası yanlış ekrana dönme, sidebar görünürlük hataları veya role bazlı erişim davranışında kırılganlık oluşabilir.
- Teknik kök neden: Top-level component çok fazla UI orchestration sorumluluğu üstleniyor.
- Önerilen çözüm: Protected route, payment modal orchestration ve route config parçalara ayrılmalı; önce küçük, davranış değiştirmeyen extraction yapılmalı.
- Risk seviyesi: P2.
- Uygulama zorluğu: Orta.

Frontend Mimarisi

P1 - OrderScreen monolitik ve profesyonel bakım standardının altında

- Sorun özeti: `client/src/components/orders/OrderScreen.jsx` 1428 satır; ekran bileşeni iş kuralı, API orchestration, state ve UI detaylarını aynı yerde taşıyor.
- Kullanıcıya/işletmeye etkisi: Sipariş ekranında yapılacak küçük değişiklikler ürün ekleme, toplam hesaplama, ödeme tetikleme veya mutfak akışını bozabilir.
- Teknik kök neden: Hook, utility ve alt component sınırları yeterince ayrılmamış.
- Önerilen çözüm: Önce saf hesaplama ve format fonksiyonlarını utility'ye çıkar; sonra API orchestration hook; en son UI alt bileşenleri.
- Risk seviyesi: P1.
- Uygulama zorluğu: Yüksek.

P1 - PrinterDetailPage çok büyük ve entegrasyon UI'ı kırılgan

- Sorun özeti: `client/src/components/settings/PrinterDetailPage.jsx` 972 satır; yazıcı konfigürasyon UI'ı tek dosyada yoğun.
- Kullanıcıya/işletmeye etkisi: Yanlış encoding, yanlış cihaz seçimi veya kaydedilen ayarların bridge ile uyumsuz olması yazdırmayı bozabilir.

- Teknik kök neden: Form state, discovery seçimi, printer option normalizasyonu ve UI aynı dosyada.
- Önerilen çözüm: Printer form modelini utility/hook olarak ayır; encoding/device option alanlarını küçük komponentlere böl.
- Risk seviyesi: P1.
- Uygulama zorluğu: Orta-yüksek.

P2 - API client tek sınıfta tüm domainleri topluyor

- Sorun özeti: `client/src/services/api.js` auth, masa, ürün, sipariş, ödeme, müşteri, caller ID, rapor, stok, waiter call ve print uçlarını tek sınıfta topluyor.
- Kullanıcıya/işletmeye etkisi: Yeni endpoint eklemeleri kolay görünür ama uzun vadede yanlış domain çağırısı, copy-paste hata ve test zorluğu üretir.
- Teknik kök neden: Domain bazlı client modülleri yok.
- Önerilen çözüm: Davranışı koruyarak domain modüllerine böl; geriye uyum için mevcut `api` facade bir süre korunabilir.
- Risk seviyesi: P2.
- Uygulama zorluğu: Orta.

Backend / API

P1 - Test kapısı kırık olduğu için API değişiklikleri güvenle doğrulanamıyor

- Sorun özeti: `npm test --prefix server` `vitest` bulunamadığı için çalışmıyor.
- Kullanıcıya/işletmeye etkisi: Ödeme, sipariş, yazdırma gibi para ve operasyon akışlarında regresyon riski artar.
- Teknik kök neden: Server devDependency tanımlı ama yerel `server/node_modules/.bin/vitest` mevcut değil veya kurulum bozuk.
- Önerilen çözüm: Server bağımlılıklarını kur; test komutunu root ve CI için tek güvenilir kapı yap; testlerin çalıştığını audit dosyasında kaydet.
- Risk seviyesi: P1.
- Uygulama zorluğu: Düşük.

P1 - Yazdırma job üretimi ve mock/bridge davranışı çok katmana yayılmış

- Sorun özeti: Backend print job üretimi, mock processing, bridge claim ve renderer farklı dosyalarda doğru ayrılmış olsa da profil/encoding kararları dağınık.
- Kullanıcıya/işletmeye etkisi: Fiş çıkmaması, yanlış yazıcıya gitmesi veya Türkçe karakter bozulması işletmenin canlı kullanımını doğrudan etkiler.
- Teknik kök neden: `server/services/printJobs.js`, `server/routes/bridge.js`, `store-bridge/jobs/poller.js`, `store-bridge/printers/renderers.js` arasında net profil kontratı zayıf.
- Önerilen çözüm: Print payload schema ve printer profile contract dokümente edilmeli; renderer snapshot testleri artırılmalı.
- Risk seviyesi: P1.
- Uygulama zorluğu: Orta.

Veritabanı

P2 - Migration dosyası büyümüş ve sürdürülebilir sürümleme zayıf

- Sorun özeti: `server/migrations/run.js` 673 satır; schema ve veri düzeltme mantığı aynı dosyada büyüyor.
- Kullanıcıya/işletmeye etkisi: Sahadaki eski veritabanlarından yükseltmede beklenmeyen kolon/indeks/veri uyumsuzluğu riski artar.
- Teknik kök neden: Migrationlar küçük, sıralı ve geri izlenebilir dosyalara ayrılmamış.
- Önerilen çözüm: Mevcut idempotent yaklaşımı koruyarak yeni migrationları numaralı dosyalara ayır; migration audit tablosu veya schema version ekle.
- Risk seviyesi: P2.
- Uygulama zorluğu: Orta-yüksek.

Entegrasyonlar

P1 - Türkçe karakter/yazıcı kod sayfası cihaz bağımlı ve kırılgan

- Sorun özeti: PC857, Windows-1254, ESC t ve Phoenix clone komutları aynı akışta env/DB/printer option ile çözülüyor.
- Kullanıcıya/işletmeye etkisi: Müşteri fişinde veya mutfak fişinde `ÇĞİÖŞÜ çğıöşü` bozulur; ürün adları anlaşılmaz hale gelir, marka güveni düşer.
- Teknik kök neden: Encoding kararı tek bir profil sözleşmesine bağlanmamış; `win1254` default, `escT=32`, `skipPhoenixCmd` gibi varsayımlar cihaz modeline bağlı.
- Önerilen çözüm: Yazıcı profilleri oluştur: `generic-pc857`, `jp80h-win1254`, `ascii-safe` gibi. Profil test fişi ve byte snapshot ekle.
- Risk seviyesi: P1.
- Uygulama zorluğu: Orta.

P2 - Caller ID ve StoreBridge operasyonel süreçleri aynı desktop lifecycle içinde yoğun

- Sorun özeti: Electron main process server, bridge ve caller ID helper lifecycle'larını birlikte yönetiyor.
- Kullanıcıya/işletmeye etkisi: Bridge restart veya caller ID hatası masaüstü uygulamanın genel stabilitesini etkileyebilir.
- Teknik kök neden: `electron/main.cjs` 860 satır ve process yönetimi tek dosyada.
- Önerilen çözüm: Process manager yardımcı modülleri çıkar; log ve health state ayrıştır.
- Risk seviyesi: P2.
- Uygulama zorluğu: Orta.

Desktop / Paketleme

P1 - `file:..` dependency ve paketleme çıktıları tarama/paketleme döngüsü yaratıyor

- Sorun özeti: Alt paketler root projeyi `file:..` dependency olarak çekiyor; `node_modules/restoran-pos` altında recursive repo kopyaları/tarama hataları oluşuyor.

- Kullanıcıya/işletmeye etkisi: Build/paketleme yavaşlar, Windows path/symlink/antivirüs sorunları artar; destek verilebilirlik düşer.
- Teknik kök neden: Workspace yerine dosya bağımlılığıyla root paket alt paketlerin içine bağlanmış.
- Önerilen çözüm: npm workspaces tasarla veya root dependency ihtiyacını kaldır; dist ve node_modules tarama kapsamından sistematik dışlanmalı.
- Risk seviyesi: P1.
- Uygulama zorluğu: Orta-yüksek.

P1 - Electron main process çok fazla sorumluluk taşıyor

- Sorun özeti: Config okuma, JWT üretimi, DB taşıma, backend spawn, bridge spawn, caller ID helper ve BrowserWindow aynı dosyada.
- Kullanıcıya/işletmeye etkisi: Desktop açılış, kapanış, port çakışması, bridge restart gibi sorunların kök nedeni zor bulunur.
- Teknik kök neden: `electron/main.cjs` 860 satır; process lifecycle modüler değil.
- Önerilen çözüm: `config`, `serverProcess`, `bridgeProcess`, `callerIdProcess`, `sqliteMigration` modüllerine ayırıştır.
- Risk seviyesi: P1.
- Uygulama zorluğu: Yüksek.

P2 - Root script yüzeyi operasyonel olarak karışık

- Sorun özeti: Root `package.json` içinde dev, prod, Electron, native rebuild, dist, caller ID ve Windows batch komutları yoğun.
- Kullanıcıya/işletmeye etkisi: Yanlış komut çalıştırma ve eksik build alma riski artar.
- Teknik kök neden: Script ayrımı görev bazlı gruplanmamış.
- Önerilen çözüm: README'deki kullanıcı komutları ile developer/release komutlarını ayrı dokümente et; script isimlerini netleştir.
- Risk seviyesi: P2.
- Uygulama zorluğu: Düşük-orta.

Test / QA

P1 - Server test altyapısı çalışmıyor

- Sorun özeti: Test dosyaları var ama komut çalışmıyor.
- Kullanıcıya/işletmeye etkisi: Kritik akışlarda sessiz regresyon riski.
- Teknik kök neden: Bağımlılık kurulumu veya workspace yapısı bozuk.
- Önerilen çözüm: `npm install --prefix server`; `npm test --prefix server`; sonucu CI veya lokal gate olarak sabitle.
- Risk seviyesi: P1.
- Uygulama zorluğu: Düşük.

P1 - Yazıcı encoding test kapsamı yetersiz

- Sorun özeti: Encoding test dosyası var ama audit bulgusu, fiziksel cihaz profili ve ESC/POS komut snapshotlarının büyütülmesi gerektiğini gösteriyor.
- Kullanıcıya/işletmeye etkisi: Türkçe karakter problemi sahada tekrar eder.
- Teknik kök neden: Byte encode testi tek başına yeterli değil; ESC t, Phoenix skip, template line wrapping birlikte doğrulanmalı.
- Önerilen çözüm: Printer preview/encoding testlerine profil bazlı snapshot ekle.
- Risk seviyesi: P1.
- Uygulama zorluğu: Düşük-orta.

P2 - Frontend test, lint ve typecheck kapıları yok

- Sorun özeti: Client scriptleri yalnız `dev`, `build`, `preview`; root/server/client scriptlerinde lint/typecheck yok.
- Kullanıcıya/işletmeye etkisi: UI regresyonları ve basit kalite hataları build'e kadar yakalanmaz.
- Teknik kök neden: QA araçları script yüzeyine eklenmemiş.
- Önerilen çözüm: Önce davranış değiştirmeyen `lint` veya en azından `build` gate dokümanı; sonra Vitest/RTL veya Playwright smoke testleri.
- Risk seviyesi: P2.
- Uygulama zorluğu: Orta.

Güvenlik / Operasyon

P1 - Production JWT secret kontrolü olumlu, ama operasyonel test kapısı eksik

- Sorun özeti: Production fallback secret engelleniyor; ancak test komutu kırık olduğu için güvenlik middleware değişiklikleri düzenli doğrulanamıyor.
- Kullanıcıya/işletmeye etkisi: Auth veya business scope regresyonu fark edilmeden sahaya gidebilir.
- Teknik kök neden: Güvenlik kontrolleri var, QA kapısı güvenilir değil.
- Önerilen çözüm: Auth middleware ve integration testleri CI/lokal gate içinde çalışır hale getir.
- Risk seviyesi: P1.
- Uygulama zorluğu: Düşük.

P2 - Build artifact ve node_modules kaynak taramasını kirletiyor

- Sorun özeti: `dist-electron` ve çoklu `node_modules` repo içinde mevcut; audit/tarama araçları bunları dolaşırken hata ve zaman aşımı üretebiliyor.
- Kullanıcıya/işletmeye etkisi: Geliştirici verimliliği düşer; yanlış dosyada analiz/değişiklik riski artar.
- Teknik kök neden: Kaynak ve çıktı ayrımı yerel workspace'te net değil; tarama komutları ignore disiplini olmadan çalışıyor.
- Önerilen çözüm: Geliştirici dokümanına kaynak tarama scope'u ekle; workspace/package stratejisini düzelt.
- Risk seviyesi: P2.
- Uygulama zorluğu: Düşük-orta.

Önceliklendirilmiş uygulama sırası

Hemen düzeltilmesi gerekenler

1. Server test altyapısını çalışır hale getir.
2. Yazıcı encoding/ESC-POS test kapsamını genişlet.
3. `file:..` dependency ve `node_modules` recursive tarama sorununu en azından dokümente edip tarama/build kapsamından izole et.
4. Büyük frontend bundle için düşük riskli route lazy loading planını başlat.

Bu hafta ele alınacaklar

1. `store-bridge/printers/renderers.js` içinden encoding/profile yardımcılarını çıkarmaya başla.
2. `OrderScreen.jsx` için saf utility ve hook extraction planını uygula.
3. `PrinterDetailPage.jsx` için form modelini UI'dan ayır.
4. Electron process lifecycle'ı için modül sınırlarını çıkar ve küçük refactorlara başla.
5. Migration stratejisini numaralı/idempotent migration dosyalarına taşıma planı hazırla.

Sonraya bırakılacaklar

1. Tam npm workspaces dönüşümü.
2. Büyük route/service ayrıştırmaları.
3. Playwright tabanlı uçtan uca POS smoke testleri.
4. Electron process manager kapsamlı ayrıştırması.
5. Full printer profile registry ve cihaz model kataloğu.

En kritik 5 sorun

1. Server test komutunun çalışmaması.
2. Türkçe karakter/yazıcı encoding zincirinin cihaz bağımlı ve dağınık olması.
3. `OrderScreen.jsx` ve yazdırma renderer'ının monolitik yapısı.
4. `file:..` dependency ile recursive `node_modules`/tarama/paketleme karmaşası.
5. Frontend bundle'ın düşük donanımlı POS cihazları için büyümesi.

En güvenli 5 hızlı kazanım

1. Server bağımlılıklarını kurup `npm test --prefix server` kapısını çalışır hale getirmek.
2. Yazıcı encoding testlerine Türkçe karakter ve TL sembolü snapshotları eklemek.
3. Audit altında derin konu raporlarını oluşturmak: printing, payment integrity, frontend state, packaging.
4. Client build uyarısını görünür yapmak ve lazy-load planını dokümente etmek.
5. Kaynak tarama komutlarında `node_modules`, `dist-electron`, `release` kapsam dışı disiplinini belgelemek.

En riskli ama gerekli 5 refactor alanı

1. `client/src/components/orders/OrderScreen.jsx` parçalama.

2. `store-bridge/printers/renderers.js` encoding/layout/template ayrıştırması.
3. `electron/main.cjs` process lifecycle modüllerine bölme.
4. `server/routes/orders.js` ve `server/routes/payments.js` domain servislerine ayrıştırma.
5. npm workspace/package dependency yapısını yeniden tasarlama.

00b - İlk Sertleştirme Geçişi

Tarih: 2026-04-13 Temel raporlar:

- docs/audit/00-overall-audit.md
- docs/audit/00a-prioritized-action-plan.md

Kapsam: En güvenli ve yüksek değerli 3 kalite iyileştirmesi. Büyük mimari değişiklik, yeni özellik veya davranışsal kapsam genişletmesi yapılmadı.

Seçilen 3 iyileştirme

1. Yazıcı encoding mantığını renderer monolitinden ayırma

Neden seçildi:

- Audit'te P1 olarak işaretlenen Türkçe karakter/yazıcı kod sayfası karmaşıklığına doğrudan temas ediyor.
- Davranışı değiştirmeden, en riskli dosyalardan biri olan store-bridge/printers/renderers.js boyutunu ve sorumluluk yoğunluğunu azaltıyor.
- Sonraki adımda yazıcı profili testlerini büyötmek için daha temiz bir modöl sınırı veriyor.

Etkilenen dosyalar:

- store-bridge/printers/encoding.js
- store-bridge/printers/renderers.js

Yapılan değişiklik:

- PC857 ve Windows-1254 encode fonksiyonları, ESC t çözümleyicileri, fallback/transliteration davranışı ve printable text normalize fonksiyonu yeni encoding.js modölüne taşındı.
- renderers.js mevcut encodePC857 ve encodeWin1254 export'larını koruyacak şekilde re-export yapıyor; mevcut test/import yüzeyi kırılmadı.

2. Yazıcı encoding test kapsamını güçlendirme

Neden seçildi:

- Türkçe karakter bozulması sahadaki ana risklerden biri.
- Küçük test ekleriyle default Win1254 davranışı ve PC857 davranışı daha net kilitlendi.

Etkilenen dosyalar:

- server/tests/encodePC857.test.js

Yapılan değişiklik:

- Varsayılan renderer modunun Windows-1254 ESC t 32 kullandığı ve Phoenix FS komutunu göndermediği doğrulandı.
- PC857 modunda ESC t 12 kullanıldığı, Phoenix FS komutunun korunduğu ve Ç karakterinin PC857 byte değeriyle çıktığı doğrulandı.

3. Frontend route bazlı lazy loading

Neden seçildi:

- Audit'te P1 olarak işaretlenen büyük frontend bundle uyarısına doğrudan ve düşük riskli yanıt veriyor.
- Ekranları ilk yüklemeyi ayırarak düşük donanımlı POS cihazlarında açılış yükünü azaltır.

Etkilenen dosyalar:

- `client/src/App.jsx`

Yapılan değişiklik:

- Ana ekran route componentleri `React.lazy` ile dinamik import'a taşındı.
- Route ağacı `Suspense` ile sarıldı.
- Ödeme modal componentleri de lazy yüklenecek şekilde sarıldı.
- Basit bir route fallback eklendi.

Değişen dosyalar

- `docs/audit/00a-prioritized-action-plan.md`: P0/P1/P2 maddeleri kategori bazlı eylem planına dönüştürüldü.
- `docs/audit/00b-first-hardening-pass.md`: Bu sertleştirme geçişinin raporu oluşturuldu.
- `client/src/App.jsx`: Route bazlı lazy loading eklendi.
- `server/tests/encodePC857.test.js`: Encoding/ESC-POS komut testleri genişletildi.
- `store-bridge/printers/encoding.js`: Yeni encoding modülü eklendi.
- `store-bridge/printers/renderers.js`: Encoding sorumlulukları yeni modüle taşındı, mevcut public export korundu.

Doğrulama sonuçları

- `npm install --prefix server`
- Başarılı.
- Önceki `vitest` bulunamadı blokajını çözdü.
- `npm run build --prefix client`
- Başarılı.
- Önceki büyük ana chunk yaklaşık 948 kB idi.
- Bu geçişten sonra ana `index` chunk yaklaşık 264.88 kB oldu.
- Ekranlar ayrı chunk'lara bölündü: örnek `OrderScreen` 38.42 kB, `PaymentScreen` 30.46 kB, `PrinterDetailPage` 26.72 kB.
- Hala büyük vendor chunklar var: `xlsx` 429.03 kB, `PieChart` 369.36 kB.
- `npm test --prefix server`
- İlk deneme başarısız oldu; sebep `better-sqlite3` native modülünün farklı Node ABI ile derlenmiş olmasıydı.
- `npm rebuild better-sqlite3 --prefix server` çalıştırıldı.
- İkinci deneme başarılı: 12 test dosyası, 108 test geçti.

- Lint/typecheck
- Root/client/server scriptlerinde `lint` veya `typecheck` komutu bulunmadı.
- Bu nedenle lint/typecheck çalıştırılmadı.

Kalan açık riskler

- Yazıcı encoding ayrıştırması davranış korumalı yapıldı, ancak fiziksel yazıcı üzerinde gerçek çıktı testi yapılmadı. Özellikle JP80H/Phoenix benzeri cihazlarda fiili `ESC t` davranışı yerinde doğrulanmalı.
- Route lazy loading build seviyesinde doğrulandı; gerçek Electron penceresinde kullanıcı akışlarıyla smoke test yapılmadı.
- `xlsx` ve chart/vendor chunkları hala büyük. Bu geçiş ana app chunkını küçülttü ama bütün bundle optimizasyonunu tamamlamadı.
- Server testleri native rebuild sonrası geçti; bu durum CI veya temiz makinede bağımlılık kurulum/rebuild adımının netleştirilmesi gerektiğini gösteriyor.
- Lint/typecheck gate'i hala yok.

01 - Product & Business Rules Audit

Tarih: 2026-04-14 Kapsam: masa, siparis, odeme, hizli odeme, masa kapatma/tasima, paket siparis, yazici/mutfak ve Caller ID is akislari. Kaynak baglam: `docs/audit/00-overall-audit.md`, `docs/audit/00a-prioritized-action-plan.md`, `docs/audit/00b-first-hardening-pass.md`.

Bu rapor kod degisikligi yapmadan, repo icindeki mevcut davranisi urun/operasyon kurallari acisindan cikarmak icin hazirlanmistir.

1. Urun akisi ozeti

Uygulama restoran POS icin iki ana siparis tipini destekliyor:

- `dine_in`: masa uzerinden acilan salon siparisi.
- `takeaway`: paket siparis / al-gotur akisi.

Ana urun modeli su sekilde calisiyor:

- Kullanici masa ekranindan bos/dolu/rezerve masaya tiklar.
- Siparis ekraninda urunler once lokal sepete eklenir.
- `Kaydet` ile siparis olusturulur veya kayitli siparise yeni kalemler eklenir.
- Backend, kaydedilen yeni kalemleri otomatik mutfaga gonderir.
- Kaydedilmemis urun varken odeme aksiyonlari frontend tarafinda gizlenir.
- Odeme ayrintili odeme ekrani veya hizli odeme modalindan alinir.
- Masa kapatma sadece odenmis hesap icin masalar ekraninda veya odeme ekraninda mumkundur.
- Masa tasima hem masalar ekranindan hem siparis ekranindan yapilabilir; her iki durumda da hedef siparise yonlendirme yerine masalar ekranina donulur.
- Paket sipariste musterisi secimi zorunludur; teslimat durumu masalar ekranindaki paket siparis yan panelinden yonetilir.
- Caller ID gelen aramayi paket siparis acisina baglar; siparis kaydedildiginde call log ile order iliskilendirilir.

Kritik urun gercegi: Frontend "Kaydet" islemini siparisin olusturulmasi/urun eklenmesi gibi sunuyor; backend ise kaydedilen her yeni kalemi hemen mutfaga gonderip siparis durumunu `in_kitchen` yapabiliyor. Bu, POS dilinde "kaydet" ile "mutfaga gonder" davranisinin ayni islemdede birlesmesi anlamina geliyor.

2. Kritik kullanici akislari

2.1 Masa acma

Mevcut davranisi:

- Kullanici `TablesScreen` uzerinde herhangi bir masaya tiklar.
- Transfer modu aktif degilse `onOpenOrder(table)` calisir.
- `App.jsx` bu masayi `/order/table/:id` rotasina `table`, `existingOrderId`, `orderType: dine_in` state'i ile tasir.
- Masa bos ise `existingOrderId` yoktur; siparis ekrani yeni sepet ile acilir.
- Masa dolu ise `existingOrderId` vardir; mevcut siparis yuklenir.

Kanit:

- `client/src/components/tables/TablesScreen.jsx`: masa tiklama ve transfer modu ayrimi.
- `client/src/App.jsx`: `handleOpenOrder` ile masa siparis ekranina yonlendirme.
- `client/src/components/orders/OrderScreen.jsx`: `existingOrderId || table?.current_order_id` ile mevcut siparis yukleme.

Urun yorumu:

- Bos masa acma davranisi dogru.
- Rezerve masa icin ayri bir onay/kural yok; rezerve masa da siparis ekranina normal giriyor. Bu urun kurali belirsiz.

2.2 Siparis olusturma

Mevcut davranis:

1. Kullanici urun secerek lokal sepete ekler.
2. Ayni urun/portion/modifier/not kombinasyonu tekrar eklenirse sepette miktar artar.
3. Kaydet butonu sadece sepette urun varken anlamlidir.
4. Yeni masa siparisinde `api.createOrder` cagrilir.
5. Backend siparisi `saved` status ile olusturur, masayi `occupied` yapar.
6. Backend hemen yeni kalemleri mutfak icin finalize eder, kalemleri `sent`, siparisi `in_kitchen` yapar.
7. Frontend basarili kayıt sonrası varsayılan olarak masalar ekranina doner.

Kanit:

- `client/src/components/orders/OrderScreen.jsx`: `cartItems`, `quickAddFromGrid`, `handleSaveOrder`.
- `server/routes/orders.js`: `POST /orders`, masa durumunu guncelleme ve `finalizeKitchenForNewItems`.
- `server/services/printJobs.js`: mutfak yazdirma islerinin olusturulmasi.

Urun yorumu:

- "Kaydet" aksiyonu kullaniciya sadece siparisi kaydediyor gibi gorunur, fakat fiilen mutfaga gonderir.
- Profesyonel POS'ta bu ya bilincli olarak "Mutfaga Gonder" diye adlandirilmali ya da "Kaydet" ve "Gonder" ayrilmalidir.

2.3 Urun ekleme

Mevcut davranis:

1. Urun gridinden urun secilir.
2. Urun aktif degilse veya fiyat/portion bulunamazsa hata verilir.
3. Urun lokal sepete eklenir.
4. Mevcut siparise eklenen yeni urunler de once lokal sepette tutulur.
5. Kaydedilene kadar mevcut siparisin backend toplamlarina dahil degildir.

Kanit:

- `client/src/components/orders/OrderScreen.jsx`: `quickAddFromGrid`, `addItemToCart`, `cartSubtotal`, `savedTotal`, `displayTotal`.

Urun yorumu:

- Kayitli siparise yeni urun ekleme davranisi guvenli: odeme aksiyonlari kaydedilmemis urun varken gizleniyor.
- Ancak "ekrandaki toplam" ile "backendde odenebilir toplam" ayrimi kullaniciya yeterince net anlatilmiyor.

2.4 Siparisi kaydetme

Mevcut davranis:

1. Sepet bos ise kaydetme reddedilir.
2. Paket siparis ise musteri secimi zorunludur.
3. Yeni siparis icin `api.createOrder`, kayitli siparis icin `api.addOrderItems` cagrilir.
4. Basarili kayit sonrasi lokal sepet temizlenir.
5. Dine-in ve takeaway icin varsayilan yonlendirme masalar ekraninadir.

Kanit:

- `client/src/components/orders/OrderScreen.jsx`: `handleSaveOrder`.
- `server/routes/orders.js`: `POST /orders`, `POST /orders/:id/items`.

Urun yorumu:

- Masa siparisi kaydedildikten sonra masalar ekranina donmek salon operasyonu icin kabul edilebilir.
- Paket sipariste kayit sonrasi masalar ekranina donmek urun acisindan tartismali; paket akisinin kendi listesine/aktif paket siparise odaklanmasi daha profesyoneldir.

2.5 Kaydedilmis siparise yeni urun ekleme

Mevcut davranis:

1. Kayitli siparis acilir.
2. Yeni urunler mevcut siparis kalemlerinden ayri olarak lokal sepete eklenir.
3. Kaydedilmemis degisiklik varken odeme ve hizli odeme aksiyonlari gizlenir.
4. Kaydet ile backend `addOrderItems` calisir.
5. Backend yeni kalemleri hemen mutfaga gonderir.

Kanit:

- `client/src/components/orders/OrderScreen.jsx`: `hasUnsavedChanges`, `canOpenPayment`, bottom actions.
- `server/routes/orders.js`: `POST /orders/:id/items`, `finalizeKitchenForNewItems`.

Urun yorumu:

- Bu akista "once kaydet, sonra odeme" kurali dogru ve risk azaltici.
- Fakat kullanici yeni urunleri ekledikten sonra "Kaydet" yaptiginda bunun mutfaga fis olarak gittigi daha acik olmalidir.

2.6 Odeme alma

Mevcut davranis:

1. Odeme ekrani yalnızca kayitli, kapanmamis ve aktif kalemi olan siparis icin acilir.
2. Kismi odeme desteklenir.

3. "Odeme al", "Ode ve kapat", "Ode ve yazdir", "Ode, yazdir ve kapat" aksiyonlari vardır.
4. Kapatma seciliyse kalan tutarin tamami odenmeden islem engellenir.
5. Tamamen odenmis sipariste odeme inputu yerine "Yazdir" ve "Masayi Kapat" aksiyonlari gosterilir.
6. Backend odemeyi kaydeder, `close_order` true ise tam odendi mi kontrol eder; tam degilse transaction rollback olur.

Kanit:

- `client/src/components/payments/PaymentScreen.jsx` : aksiyon tipleri, tam odeme kontrolu, kapatma/yazdirma davranisi.
- `server/routes/payments.js` : `POST /payments`, `closeOrderAndTableIfPaid`.

Urun yorumu:

- Kismi odeme + kapatma engeli dogru.
- "Ode ve Yazdir" aksiyonu kapatmadan fis basabilir. Bu bilincli olabilir, ancak POS terminolojisinde "ara hesap/fis" ile "final fis" ayrimi yok.
- Tamamen odenmis sipariste tekrar hizli odeme modalindan kapatma mumkun degil; kapatma icin masalar ekranindaki "Masayi Kapat" veya ayrintili odeme ekranindaki kapatma aksiyonu kullaniliyor.

2.7 Hizli odeme

Mevcut davranis:

1. Hizli odeme kalan tutarin tamamini odetir.
2. Sadece nakit/kart secimi vardır.
3. Kismi tutar girilemez.
4. "Odeme al", "Ode ve kapat", "Ode ve yazdir", "Ode, yazdir ve kapat" aksiyonlari vardır.
5. Kalan tutar sifirsa modal aksiyonlari disabled olur; mevcut odenmis masayi bu modal ile kapatmak mumkun degildir.

Kanit:

- `client/src/components/payments/QuickPaymentModal.jsx` : `totalDue` uzerinden tam odeme, action type'lar, disabled butonlar.

Urun yorumu:

- Hizli odemenin tam bakiye odemesi almasi dogru bir POS kisayolu.
- Ancak "hesap zaten odenmis ama masa kapanmamis" durumunda hizli odeme modalinin kapatma sunmaması operasyonel belirsizlik yaratir.

2.8 Masayi kapatma

Mevcut davranis:

1. Masa karti odenmis siparisi `HESAP ODENDI` olarak gosterir.
2. Masalar ekraninda "Masayi Kapat" aksiyonu yalnızca paid hesapta gorunur.
3. Kapatma `api.updateOrderStatus(orderId, 'closed')` ile yapilir.
4. Backend siparisi kapatir, masayi bosaltir, musterı istatistiklerini gunceller, dine-in icin receipt job ekleyebilir.
5. Odeme ekraninda da tam odenmis siparis icin "Masayi Kapat" vardır.

Kanıt:

- `client/src/components/tables/TablesScreen.jsx`: `isTableBillPaid`, masa aksiyon modali, `handleClosePaidTable`.
- `client/src/components/payments/PaymentScreen.jsx`: `closePaidOrder`.
- `server/routes/orders.js`: `PATCH /orders/:id/status` `closed` branch.

Urun yorumu:

- Odenmis masa ile kapali masa arasinda ara durum vardır. Bu dogru bir isletme kuralidir: hesap odendi, masa fiziksel olarak henüz bosalmamis olabilir.
- Ancak bu ara durumun adi ve aksiyon sahipliği net degil. Kullanici bazen ödeme ekranından, bazen masa ekranından kapatir.

2.9 Masa tasima

Mevcut davranis:

1. Masalar ekranında dolu kaynak masa için "Masayi Tasi" baslatilir.
2. Hedef masa secilir.
3. Backend hedef masa `occupied` ise reddeder; rezerve masa için acik engel yoktur.
4. Source order hedef `table_id`'ye tasirir.
5. Source masa bosaltilir, hedef masa kaynak masa `status/current_order_id/guest_count` degerlerini alır.
6. Frontend reload yapar ve masalar ekranında kalir.
7. Siparis ekranından tasima da mümkündür; basarili tasima sonrası masalar ekranına doner.

Kanıt:

- `client/src/components/tables/TablesScreen.jsx`: transfer mode, `handleTableTransfer`.
- `client/src/components/orders/OrderScreen.jsx`: `openMoveDialog`, `handleMoveTable`.
- `server/routes/tables.js`: `POST /tables/:id/transfer`.

Urun yorumu:

- Hedef rezerve masaya tasima belirsiz ve risklidir.
- Tasima sonrası hedef masanın siparisine otomatik gecilmemesi operasyon açısından ekstra adım yaratir.

2.10 Paket siparis akisi

Mevcut davranis:

1. Paket siparis `/order/takeaway` rotasından acilir.
2. Caller ID veya paket yan panelinden mevcut paket siparise gecilebilir.
3. Yeni paket sipariste musterî secimi zorunludur.
4. Siparis kaydedilince takeaway label print job eklenir.
5. Acik paket siparisler masalar ekranında yan panelde listelenir.
6. Paket siparis durumları yan panelden `out_for_delivery` ve `delivered` olarak ilerletilir.
7. `delivered` aksiyonu backendde siparisi `closed` yapar.

Kanıt:

- `client/src/App.jsx` : `handleNewTakeawayOrder` .
- `client/src/components/orders/OrderScreen.jsx` : takeaway customer zorunlulugu.
- `client/src/components/tables/TablesScreen.jsx` : open takeaway sidebar ve delivery action buttonlari.
- `server/routes/orders.js` : takeaway open list, label print, delivery status update.

Urun yorumu:

- Paket sipariste musteri zorunlulugu dogru.
- Teslim edildi aksiyonunun direkt `closed` yapmasi urun olarak kabul edilebilir; fakat odeme alınmadan teslim edildi yapilabiliyorsa bu buyuk is kurali riskidir. Kodda delivery status update icin odeme kontrolu gorunmuyor.

2.11 Yazici / mutfak akisi

Mevcut davranis:

1. Yeni siparis veya yeni eklenen kalemler kaydedildiginde mutfak print job olusur.
2. Mutfak print job item bazli printer routing yapar.
3. Iptal veya miktar azaltma gibi degisiklikler icin mutfak adjustment job olusabilir.
4. Masa kapatma veya print receipt aksiyonu receipt job olusturur.
5. Paket siparis olusunca label job olusur.
6. Print job idempotency key ile duplicate basim engellenmeye calisilir.

Kanit:

- `server/routes/orders.js` : `finalizeKitchenForNewItems` , item update/cancel.
- `server/routes/payments.js` : odeme sonrasi receipt job.
- `server/services/printJobs.js` : kitchen, adjustment, receipt, takeaway label enqueue akislari.

Urun yorumu:

- Mutfak ve siparis yasam dongusu birbirine sikica bagli.
- "Kaydet" = "mutfaga gonder" oldugu icin UI metinleri ve operasyon egitimi bu gercege uygun olmalı.

2.12 Caller ID akisi

Mevcut davranis:

1. Backend gelen aramayi `call_logs` icine `ringing` olarak yazar.
2. Frontend admin/cashier icin her 4 saniyede yeni ringing call log poll eder.
3. Popup'ta musteri ve gecmis siparis bilgileri gosterilir.
4. "Siparisi Ac" kullaniciyi paket siparis ekranina goturur.
5. Siparis kaydedilirse `call_log_id` order ile iliskilendirilir ve call log status `opened_order` olur.
6. Kullanici siparis ekranini kaydetmeden terk ederse server tarafinda call log `ringing` kalabilir; frontend sadece session bazli suppress yapar.

Kanit:

- `server/services/callerIdService.js` : `processIncomingCall` , `linkCallLogToOrder` .
- `server/routes/callerid.js` : recent/history/status endpointleri.
- `client/src/context/IncomingCallContext.jsx` : popup polling, dismiss, open order navigation.

- `server/routes/orders.js` : order create sonrası call log link.

Urun yorumu:

- Caller ID'nin paket siparise bağlanması doğru.
- "Siparisi Ac" tiklanınca call log durumunun hemen `opened_order` veya `in_progress` yapılmaması amatör bir boşluk; kaydedilmeden terk edilen aramalar operasyon ekranında tekrar belirebilir veya yanlış durumla kalabilir.

3. Mevcut is kuralı matrisi

Durum	Kullanici ne goruyor	Ana aktif aksiyon	Gizli/engelli olmasi gerekenler	Mevcut risk
Bos masa	Bos masa karti	Masaya tiklayip siparis acma	Odeme, hizli odeme, yazdir, kapat	Uygun
Rezerve masa	Rezerve masa karti	Masaya tiklayip siparis acma	Normal siparis acma onaysiz olmamali	Rezerve kuralı belirsiz
Dolu masa, kaydedilmis siparis	Masa dolu, toplam/tutar bilgisi	Siparisi ac, ode, hizli ode, yazdir, tasi, iptal	Masa kapat sadece tam odenmisse görünmeli	Genel olarak uygun
Dolu masa, hesap odenmemis	Dolu masa	Odeme / hizli odeme	Masayi kapat	Uygun
Dolu masa, hesap tam odenmis ama kapanmamis	HESAP ODENDI rozeti	Masayi kapat, yazdir	Yeni odeme alma	Ara durum dogru ama sahiplik belirsiz
Yeni siparis, sepette urun var	Sepet ve Kaydet	Kaydet	Odeme, hizli odeme	Uygun
Yeni siparis, sepet bos	Bos sepet	Urun ekleme	Kaydet, odeme, hizli odeme	Uygun
Kayitli siparis, yeni urun eklenmis	Kayitli kalemler + lokal sepet	Kaydet	Odeme, hizli odeme	Uygun, ama "mutfaga gider" metni eksik
Kayitli siparis, kaydedilmemis degisiklik yok	Kalemler ve toplam	Odeme, hizli odeme, urun ekleme	Kaydet	Uygun
Siparis kapali	Kapali siparis	Geri don / görüntuleme	Urun ekleme, odeme, iptal	Frontend/route bazinda daha net kilitlenmeli
Siparis iptal	Iptal siparis	Geri don / görüntuleme	Urun ekleme, odeme, kapatma	Backend koruyor, UI netligi denetlenmeli
Paket siparis yeni	Musteri secimi + sepet	Musteri sec, urun ekle, kaydet	Musterisiz kaydet	Uygun
Paket siparis acik	Yan panelde kart	Siparisi ac, label yazdir, iptal, teslimat durum ilerlet	Odeme alınmadan teslim edildi tartismali	Odeme/teslim kuralı eksik
Paket siparis teslimata cikti	Yan panelde durum	Teslim edildi	Tekrar teslimata cikar	Uygun
Paket siparis teslim edildi	Listeden dusmeli / kapanir	Kapali kayit	Aktif panel aksiyonlari	Backend kapatiyor
Hizli odeme, borc var	Modal, kalan tutar	Tam tutari nakit/kart ode	Kismi odeme	Uygun
Hizli odeme, borc yok	Modal aksiyonlari disabled	Kapatma beklenebilir	Odeme alma	Kapatma yok, operasyonel bosluk
Masa tasima, kaynak dolu hedef bos	Transfer modu	Hedef sec ve tasi	Dolu hedef	Uygun

Durum	Kullanici ne goruyor	Ana aktif aksiyon	Gizli/engelli olmasi gerekenler	Mevcut risk
Masa tasima, hedef rezerve	Transfer mumkun gorunebilir	Tasi	Rezerve hedefe onaysiz tasima	Riskli
Caller ID ringing	Popup	Siparisi ac / kapat	Tekrar popup spam	Kaydetmeden terk edilirse server durumu kalir

4. Celiskiler ve belirsizlikler

4.1 Kullanici beklentisi ile mevcut davranis farki

P1 - "Kaydet" aksiyonu fiilen mutfaga gonderiyor.

- Etki: Garson sadece siparisi kaydettigini sanabilir; mutfakta fis basilmis olur.
- Kok neden: Backend `createOrder` ve `addOrderItems` sonras `finalizeKitchenForNewItems` calistiriyor.
- Oneri: V1'de buton metni "Kaydet ve Mutfaga Gonder" yapilmali veya UI'da net yardim metni verilmeli. Daha ileri surumde "Kaydet" ve "Mutfaga Gonder" ayrilabilir.

P1 - Paket siparis teslim edildi aksiyonu odeme kontrolu olmadan siparisi kapatiyor olabilir.

- Etki: Tahsilatsiz paket siparis kapatilabilir, kasa/acik hesap uyumsuzlugu olur.
- Kok neden: Delivery status endpointinde payment total kontrolu gorunmuyor.
- Oneri: "Teslim edildi" icin odeme zorunlulugu urun karari olarak netlestirilmeli. Kapida odeme desteklenecekse teslimde odeme modalina zorunlu yonlendirme gerekir.

P2 - "Ode ve Yazdir" ile "Ode, Yazdir ve Kapat" ayrimi UI'da yeterince keskin degil.

- Etki: Kullanici fis bastiginda masanin kapandigini sanabilir.
- Kok neden: Payment action type'lari close/print kombinasyonlari ile calisiyor, fakat receipt tipi/semantigi ayrilmamis.
- Oneri: Ara hesap, tahsilat fisi, final fis ayrimi urun dilinde netlestirilmeli.

4.2 Yeni masa / kayitli masa farki

P2 - Bos/rezerve/dolu masa acilis kurali ayni tik davranisina bagli.

- Etki: Rezerve masa yanlislikla normal siparise donusebilir.
- Kok neden: Table click handler sadece transfer mode kontrolu yapiyor, reservation icin ayri confirm/kural yok.
- Oneri: Rezerve masada "Rezervasyonu kullanarak siparis ac" onayi veya rezervasyon iptali/oturma akisi olmalidir.

P2 - Yeni siparis ve kayitli siparis ayni ekranda dogru ayriliyor; ancak kullaniciya durum etiketi zayif.

- Etki: Garson "bu siparis kayitli mi, sepette bekleyen var mi" ayrimini kacirabilir.
- Kok neden: Lokal sepet ve kayitli siparis birlikte gosteriliyor, metinsel uyar sinirli.
- Oneri: "Kaydedilmemis urunler" ve "Mutfaga gonderilmis urunler" ayrimi daha belirgin yapilmali.

4.3 Kaydetmeden once / kaydettikten sonra aksiyonlar

P1 - Kaydedilmemiş ürün varken ödeme gizleniyor; bu doğru. Fakat mevcut kayıtlı kalemler için kısmi ödeme almak istenirse engellenir.

- Etki: Gerçek operasyonda misafir bir kısmi ödeyip masada kalırken garson yeni ürün eklemiş olabilir; ödeme almak için önce yeni ürünü mutfaka göndermek zorunda kalır.
- Kok neden: `canOpenPayment` tüm unsaved cart varlığını global kilit gibi kullanıyor.
- Oneri: V1 için mevcut kural korunabilir. V2'de "kaydedilmemiş ürünleri iptal et/kaydetmedenodemeye git" kararı sunulabilir.

P2 - Kayıt sonrası otomatik masalar ekranına dönüş her akışta aynı.

- Etki: Paket sipariste veya ek ürün girişinde kullanıcı beklediği sipariş detayında kalmayabilir.
- Kok neden: `handleSaveOrder` default navigate tables.
- Oneri: Dine-in için masalar ekranına dönmek korunabilir; takeaway için paket paneline veya sipariş detayına odaklı davranış tasarlanmalıdır.

4.4 Hızlı ödeme mantığı

P2 - Hızlı ödeme tam bakiye ödeme için iyi; fakat ödenmiş masayı kapatamaz.

- Etki: "Hızlı Öde" acıldığında borç yoksa kullanıcı masayı kapatmak için geri dönmek zorunda kalır.
- Kok neden: Modal butonları `totalDue <= 0.02` iken disabled.
- Oneri: Borç yoksa modal "Bu hesap ödenmiş, masayı kapat" aksiyonuna dönüşmelidir veya hızlı ödeme butonu yerine direkt kapatma gösterilmelidir.

P2 - Hızlı ödeme sadece nakit/kart destekliyor.

- Etki: Yemek kartı, havale, online ödeme gibi operasyonlar kayıt altına alınamaz.
- Kok neden: Payment schema ve UI payment type enum'u dar.
- Oneri: V1'de kabul edilebilir; V2 için ödeme tipi konfigürasyonu gereklidir.

4.5 Masa kapatma mantığı

P1 - Masayı kapatma için backend `PATCH /orders/:id/status closed` branchinde ödeme kontrolü net görünmüyor.

- Etki: Frontend sadece ödenmiş masada kapatma gösteriyor olsa da API doğrudan kullanılırsa ödemesiz kapatma riski olabilir.
- Kok neden: Kapatma kuralı frontend tarafında daha güçlü, backend status endpointinde kapatma guard'ı payment endpoint kadar belirgin değil.
- Oneri: Backend status close için de total paid >= grand total kontrolü zorunlu olmalıdır. Paket teslim gibi özel durumlar ayrıca modellenmelidir.

P2 - Ödenmiş ama kapanmamış masa ara durumu iyi, fakat status olarak ayrı modellenmiyor.

- Etki: Raporlama ve UI kararlarında hesaplanmış durumlara bağımlılık artar.
- Kok neden: Paid state order.status değil, payments toplamından türetiliyor.
- Oneri: V1'de hesaplanmış durum korunabilir; UI'da "Hesap ödendi, masa açık" dili netleştirilmeli.

4.6 Masa tasima sonrasi beklenen yonlendirme

P2 - Tasima sonrasi hedef masaya/siparise yonlendirme yok.

- Etki: Garson tasimanin sonucunu gormek icin hedef masayi tekrar bulur.
- Kok neden: Hem table transfer hem order screen move akisi reload/navigate tables ile bitiyor.
- Oneri: Basarili tasima sonrasi hedef masa karti highlight edilmeli veya hedef siparis acilmalidir.

P1 - Hedef masa rezerve ise backend reddetmiyor.

- Etki: Rezervasyon uzerine aktif siparis tasinabilir.
- Kok neden: Transfer endpoint sadece `target.status === 'occupied'` kontrol ediyor.
- Oneri: Hedef sadece `empty` olmalidir; `reserved` icin yetkili onayli ayri akis tasarlanmalidir.

5. Amator gorunen davranislar

P1 - Islem isimleri teknik davranisi sakliyor.

- "Kaydet" kelimesi mutfak fisi basabilecek bir aksiyonu ifade ediyor.
- "Yazdir" receipt mi, ara hesap mi, final fis mi belirsiz.
- "Teslim Edildi" odeme/kapama semantigini gizliyor.

P1 - Kritik kurallar frontendde guclu, backendde her endpointte ayni sertlikte degil.

- Odeme ile kapatma payment endpointinde daha guvenli.
- Status close endpointi ayni payment guard'i acik sekilde gostermiyor.
- Paket teslim kapatma akisi odeme kuralindan bagimsiz gorunuyor.

P2 - Rezerve masa ve dolu masa operasyonlari ayni transfer/acma davranisina cok yakin.

- Rezervasyon profesyonel POS'ta ayri operasyonel durumdur.
- Onaysiz siparis acma veya tasima rezervasyon disiplinini bozar.

P2 - Caller ID "siparisi acildi" durumu sadece siparis kaydedilince servera yaziliyor.

- Kullanici popup'tan siparis ekranina gecer ama siparisi kaydetmeden cikarsa arama durumu belirsiz kalir.
- Bu, canli operasyonda tekrar eden popup veya eksik takip yaratir.

P2 - Paket siparis yan paneli masalar ekranina gomulu.

- Isletme icin paket siparis ayri bir operasyon kuyrugudur.
- Masa gridinin yanina eklenmis panel V1 icin pratik, fakat urun olgunlugu dusuk gorunuyor.

P2 - Siparis statuleri urun diliyle tam ortusmuyor.

- `saved`, `in_kitchen`, `closed`, `cancelled` gibi teknik statusler var.
- "Odenmis ama masa acik", "teslimata cikti", "hazirlaniyor", "servis edildi" gibi urun durumlari farkli kaynaklardan turetiliyor.

6. Profesyonel urun icin onerilen kural seti

6.1 V1 icin korunmasi gereken kurallar

1. Kaydedilmemis urun varken odeme/hizli odeme aksiyonlarini gizle.

- Bu, kullanıcının ekranda gordugu toplam ile backenddeki tahsil edilebilir toplam arasinda hata yapmasini engeller.
1. Hizli odeme kalan tutarin tamamini kapatsin.
- Hizli odeme profesyonel POS'ta "tek hamlede tahsilat" akisidir; kısmi odeme ayrıntılı odeme ekranında kalmalıdır.
1. Masa kapatma sadece hesap tam odenince gorunsun.
- Odenmis ama fiziksel olarak acik masa ara durumu korunmalıdır.
1. Paket sipariste musteri secimi zorunlu kalsin.
- Caller ID, adres, gecmis siparis ve teslimat icin temel veri kalitesi gerekir.
1. Kayitli siparise yeni urunler once lokal sepete gelsin.
- Yanlis dokunmalarin mutfaga aninda gitmesini engeller.
1. Gonderilmis siparis kalemlerinde miktar/portion degisikligi kisitlansin.
- Mutfak operasyonu icin gonderilmis kalemde rastgele degisiklik profesyonel degildir.
1. Mutfak adjustment job mantigi korunsun.
- Iptal/azaltma mutfaga ayrica bildirilmelidir.

6.2 V1 icin duzeltilmesi gereken kurallar

1. "Kaydet" butonunun is kuralı netlestirilmeli.
- En dusuk riskli V1 cozum: buton/metin "Kaydet ve Mutfaga Gonder" olarak degismeli veya uyari metni belirginlesmeli.
1. Backend masa kapatma guard'i payment guard'i ile ayni seviyeye getirilmeli.
- Order status close, odeme tam degilse reddetmelidir.
1. Paket "Teslim Edildi" aksiyonu odeme durumunu kontrol etmelidir.
- Odenmemis paket icin "once odeme al" yonlendirmesi gerekir.
1. Masa transfer hedefi sadece bos masa olmalı.
- Rezerve masaya tasima icin ayrica yetkili onay gerekir.
1. Caller ID open-order status server tarafina hemen yazilmalıdır.
- "Siparisi Ac" tiklanınca call log `opened_order` veya yeni bir `in_progress` status almalıdır.
1. Yazdirma aksiyonlari urun dilinde ayrilmalıdır.
- Ara hesap, tahsilat fisi ve final fis ayrimi yapilmalıdır.
1. Paket siparis kayit sonrasi yonlendirme ayrilmalıdır.
- Dine-in masalar ekranina donebilir; takeaway aktif paket siparis kuyruguna veya detayina odaklanmalıdır.

6.3 Onerilen durum modeli

Salon siparisi için:

- `draft` : frontend lokal, backend kaydi yok.
- `open_unsent` : backendde kayitli ama mutfaga gonderilmemis. V2 için.
- `in_kitchen` : mutfaga gonderildi.
- `partially_paid` : en az bir odeme var, hala borc var.
- `paid_open` : hesap odendi, masa fiziksel olarak acik.
- `closed` : masa kapandi.
- `cancelled` : iptal edildi.

Paket siparis için:

- `draft` : musteri/urun hazirlaniyor.
- `in_kitchen` : mutfaga gitti.
- `ready` : hazır. V2 mutfak durumuyla.
- `out_for_delivery` : teslimata cikti.
- `delivered_unpaid` : kapida odeme senaryosu varsa.
- `delivered_paid_closed` : teslim edildi ve kapandi.
- `cancelled` : iptal edildi.

V1'de bu modelin tamamini uygulamak sart degil; fakat UI kararlarinda bu ayrimlari isimlendirilmesi gerekir.

7. Hemen duzeltilmesi gerekenler

P1 - Backend status close için odeme kontrolu eklenmeli.

- Risk: API veya farkli UI yolu ile odemesiz masa kapatma.
- Is etkisi: Gun sonu kasa ve satis raporu hatasi.
- Kapsam: `server/routes/orders.js` closed branch.

P1 - Paket `delivered` aksiyonunda odeme kurali netlestirilmeli.

- Risk: Tahsilat alınmadan paket siparis kapanabilir.
- Is etkisi: Kasa acigi, operasyonel takip kaybi.
- Kapsam: `server/routes/orders.js` takeaway delivery status endpoint, frontend delivery button akisi.

P1 - "Kaydet" aksiyonunun urun dili duzeltilmeli.

- Risk: Mutfaga istenmeden fis gitmesi.
- Is etkisi: Yanlis hazirlanan urun, fire, mutfak karmasasi.
- Kapsam: `client/src/components/orders/OrderScreen.jsx` buton metni/yardim metni; daha buyuk karar için backend akisi.

P1 - Transfer hedefi rezerve masa için engellenmeli veya onayli hale getirilmeli.

- Risk: Rezervasyonun ezilmesi.
- Is etkisi: Musteri memnuniyetsizligi ve masa yonetimi hatasi.
- Kapsam: `server/routes/tables.js`, `client/src/components/tables/TablesScreen.jsx`.

P2 - Caller ID open-order durum boslugu kapatilmali.

- Risk: Arama popup'inin yanlis tekrar etmesi veya takipte belirsizlik.
- Is etkisi: Telefon siparislerinde kayip/tekrar is.
- Kapsam: `client/src/context/IncomingCallContext.jsx`, `server/services/callerIdService.js`, `server/routes/callerid.js`.

8. Sonraki asamada koda dokulebilecek dusuk riskli iyilestirmeler

1. UI metinlerini netlestir:

- "Kaydet" yerine "Kaydet ve Mutfaga Gonder".
- "Yazdir" yerine "Hesap Yazdir" veya "Fis Yazdir".
- "Ode ve Yazdir" yerine "Ode ve Hesap Yazdir"; kapatma iceren aksiyonda "Masayi Kapat" kelimesi mutlaka gorunsun.

1. Kaydedilmemis urun alanini daha belirgin yap:

- "Kaydedilmemis urunler" ve "Mutfaga gönderilmiş urunler" ayrimi.
- Odeme gizliyen sebep metni daha net: "Yeni urunler once mutfaga gönderilmeli."

1. Hizli odemede borc yoksa modal davranisini degistir:

- "Bu hesap zaten odenmis" bilgisi.
- Yetki uygunsu "Masayi Kapat" aksiyonu.

1. Transfer sonrasi hedef masayi vurgula:

- Basarili transferden sonra masalar ekraninda hedef masa highlight.
- Alternatif: hedef masanin siparisine otomatik gec.

1. Rezerve masa acilisinda onay ekle:

- "Bu masa rezerve. Rezervasyonu kullanarak siparis acilsin mi?"
- Yetki/rol gereksinimi sonradan eklenebilir.

1. Paket siparis yan panelinde odeme durumunu goster:

- `odenmedi`, `kismi odendi`, `odendi` etiketi.
- Teslim edildi butonunun neden disabled oldugu acik olsun.

1. Caller ID popup'tan siparis acildiginda server status guncelle:

- V1 icin `opened_order` status hemen yazilabilir.
- Daha dogru model icin `in_progress` status eklenebilir.

1. Aksiyon görünürlükleri icin tek bir frontend policy fonksiyonu olustur:

- Masa aksiyonlari ve siparis aksiyonlari ayni kurallari kullanmali.
- Bu dusuk/orta riskli bir kalite iyilestirmesidir; davranis degisikligi icermeden test edilebilir.

Ek: Onceliklendirilmis urun backlog'u

Hemen

- P1: Odeme tamamlanmadan `closed` status engeli.
- P1: Paket teslim edildi aksiyonunda odeme kuralı.
- P1: Kaydet/mutfaga gonder urun dili.
- P1: Rezerve masaya transfer/acma kuralı.

Bu hafta

- P2: Hizli odemede odenmis masa kapatma davranisi.
- P2: Caller ID "siparisi acildi ama kaydedilmedi" durumu.
- P2: Transfer sonrasi hedef masa yonlendirme/highlight.
- P2: Paket siparis yan panelinde odeme durum etiketi.

Sonra

- V2: Kaydet ve mutfaga gonder aksiyonlarini ayirma.
- V2: Siparis status modelini urun durumlariyla yeniden tasarlama.
- V2: Paket siparis icin ayri operasyon ekranı.
- V2: Odeme tipi konfigurasyonu ve ara/final fis ayrimi.

02 - UX/UI Audit

Tarih: 2026-04-14 Kapsam: masa ekranı, sipariş ekranı, adisyon paneli, ödeme/hızlı ödeme, modallar, ayarlar ekranları, görsel hiyerarşi, dokunmatik kullanım ve profesyonel POS hissi. Sınır: Bu raporda uygulama kodu değiştirilmedi; yalnızca analiz ve ekran akışı çıkarımı yapıldı.

Not: Bu denetim mevcut çalışma ağacındaki en güncel duruma göre yapıldı. Önceki ürün iş kuralı raporunda önerilen bazı düzeltmeler artık kodda görünür durumda: **Kaydet ve Mutfağa Gönder**, paket sipariş ödeme etiketi, rezerve masa transfer engeli, hızlı ödemede ödenmiş masayı kapatma gibi.

1. UX/UI genel özeti

Uygulama, restoran POS için doğru ana yüzeyleri barındırıyor: masalar, sipariş alma, adisyon, ödeme, hızlı ödeme, paket sipariş, Caller ID, ayarlar ve yazıcı ayarları. Operasyon ekranlarında hızlı erişim ve büyük yüzeyler düşünülmüş; özellikle sipariş ekranındaki ürün grid'i ve sağ adisyon paneli temel POS modeline uygun.

Ancak ürün profesyonel POS hissine henüz tam ulaşmıyor. Ana sorun tek tek ekranların varlığı değil, ekranlar arası aksiyon dilinin ve hiyerarşinin yeterince sistemleşmemesi:

- Kritik aksiyonlar bazen aynı görsel ağırlıkta yan yana duruyor: **Öde**, **Hızlı Öde**, **Yazdır**, **Masayı Taşı**, **İptal**.
- Masa ekranı ile sipariş ekranı aynı işi farklı UI kalıplarıyla yaptırıyor: masa taşıma, ödeme açma, kapatma.
- Modal sistemi çok kullanılıyor; bazı modallar POS hız ekranı gibi, bazıları ayar paneli gibi davranıyor.
- Görsel dil modernleşmiş olsa da koyu tema, yoğun kartlar, çok sayıda inline stil, güçlü gölgeler ve mor/purple accent bazı ekranlarda "operasyon aracı" yerine "dashboard template" hissi verebiliyor.
- Dokunmatik kullanımda ana butonlar çoğunlukla yeterli, ama ikon menüler, satır içi silme butonları, ürün miktar +/- butonları ve paket sipariş menüsü için hedef alanlar sınırdadır.

Net cevaplar:

- Kullanıcı en kritik aksiyonu ilk bakışta anlayabiliyor mu?
- Masa ekranında kısmen. Dolu masada ana aksiyon masaya tıklamak, ama **...** menüsündeki işlemler kritik olmasına rağmen gizli. Sipariş ekranında daha iyi: sepet varsa tek ana aksiyon **Kaydet ve Mutfağa Gönder**; sepet yoksa **Ödeme / Hızlı Öde** görünüyor.
- Yeni masa ile kayıtlı masa davranışları arayüzde net ayrılmış mı?
- Orta seviyede. Sağ panelde **Mevcut Ürünler** ve **Yeni Eklenen** ayrımı iyi. Fakat yeni masa için "taslak sipariş" durumu, kayıtlı masa için "mutfağa gitmiş sipariş" durumu daha görünür bir status çipiyle ayrılmalı.
- Kaydet / Ödeme / Hızlı Öde / Masayı Kapat** aksiyonları durum bazlı tutarlı mı?
- Artık daha tutarlı. **OrderScreen.jsx:590** ödeme açma koşulunu kaydedilmemiş ürün yokken aktif ediyor; **OrderScreen.jsx:1159** ana aksiyon dilini netleştiriyor. Masa ekranında **Masayı Kapat** yalnız tam ödenmiş hesapta gösteriliyor (**TablesScreen.jsx:262**, **TablesScreen.jsx:1153**). Hala aynı aksiyonlar farklı ekranlarda farklı layout ve isim ağırlıklarıyla sunuluyor.
- Gereksiz bilgi yoğunluğu var mı?
- Evet, özellikle masa kartlarında süre, toplam, ödenen tutar, garson, hazır/ödendi rozeti, renk kodu ve menü aynı küçük kartta yarışıyor. Ödeme ekranı da güçlü ama iki kolonlu yapı, split ödeme, aksiyon tipi, ödeme tipi, tutar alanı ve ödeme kalemleri ile yoğun.

- Eski, amatör, kaba veya güvensiz görünen UI parçaları hangileri?
- `window.confirm` kullanımı, inline style yoğunluğu, küçük ikon aksiyonları, ayarlar ekranındaki ham form hissi, yazıcı ayarlarında çok teknik seçeneklerin aynı yüzeye yığılması, modalların görev tipine göre yeterince ayrışmaması.
- Dokunmatik kullanım için buton boyutu, aralık, hiyerarşi yeterli mi?
- Ana butonlarda çoğunlukla evet: `.btn` min-height 40, büyük aksiyonlarda 46-56 px var (`global.css:580` , `global.css:615-617`). Fakat satır içi ikonlar, ürün gridindeki +/- alanları, paket kartı üç nokta menüsü ve bazı ayar formları dokunmatik yoğun kullanım için dar.

2. Ekran bazlı değerlendirme

2.1 Masa ekranı

Kanıt:

- Masa veri/state yükleme ve socket/polling: `client/src/components/tables/TablesScreen.jsx:60-116`
- Transfer modu ve masa tıklama davranışı: `client/src/components/tables/TablesScreen.jsx:192-244`
- Masa kartı grid ve kart hiyerarşisi: `client/src/components/tables/TablesScreen.jsx:523-720`
- Masa aksiyon modalı: `client/src/components/tables/TablesScreen.jsx:1089-1167`
- Paket sipariş sidebar: `client/src/components/tables/TablesScreen.jsx:784-1056`

Güçlü yönler:

- Masa ekranı doğru şekilde tek bakışta operasyon merkezi olmayı hedefliyor.
- Boş, dolu, uzun süre, hazır ve ödenmiş durumları renk/rozet ile ayrılıyor.
- Dolu masa aksiyonları ana kartın içine gömülmemiş, ayrı action sheet modalında toplanmış.
- Paket siparişler yan panelde ayrı bir kuyruk olarak gösteriliyor.
- Transfer sonrası hedef masa vurgusu mevcut çalışma ağacında eklenmiş durumda; bu, yön bulmayı iyileştiriyor.

Kullanılabilirlik kusurları:

- Dolu masada kritik işlemler `...` arkasında. Yoğun servis anında `Öde` veya `Hızlı Öde` için önce küçük üç nokta hedefini bulmak gerekiyor.
- Masa kartı birincil tıklama ile sipariş açıyor, üç nokta ile işlem açıyor. Bu iyi, ancak kullanıcıya kart üzerinde "siparişi aç" affordance'ı yok.
- Transfer modu sadece "Hedef masayı seçin" metniyle ifade ediliyor (`TablesScreen.jsx:541`). Hangi masadan taşındığı aynı banner içinde görünmüyor.
- Rezerve masa açılırken browser confirm kullanılması akışı kaba ve native olmayan hale getiriyor.
- Paket sipariş kartındaki `Teslim Edildi` butonu disabled olduğunda sebep sadece title ile veriliyor (`TablesScreen.jsx:1046`). Dokunmatik cihazda title pratikte görünmez.

Görsel kalite kusurları:

- Masa kartları yoğun bilgi ve renk koduyla çalışıyor; ödendi, hazır ve süre uyarıları aynı görsel katmanı paylaşıyor.
- `HESAP ÖDENDİ` rozeti doğru ama çok küçük; yoğun kullanımda hızlı fark edilmesi için daha güçlü layout durumu gerekebilir (`TablesScreen.jsx:677`).

- Paket kartları gradient kullanıyor; operasyon ekranında gradient dekoratif kalıyor ve okunabilirliği her zaman artırmıyor.
- `btnBase` ile paket aksiyonları lokal stil üzerinden tanımlanmış (`TablesScreen.jsx:464`); global buton sisteminden kopuk.

Profesyonel ürün hissini bozan noktalar:

- Browser `window.confirm` kullanımı. POS'ta native confirm amatör ve güven vermeyen bir his yaratır.
- Aynı ekranda hem masa grid'i hem paket sipariş kuyruğu var; V1 için kabul edilebilir ama paket operasyonu büyüdükçe yan panel sıkışır.
- İşlem modalı tüm aksiyonları eşit kareler olarak sunuyor. `İptal` gibi yıkıcı aksiyon ile `Öde` aynı grid mantığında yan yana duruyor.

2.2 Sipariş ekranı

Kanıt:

- Topbar, masa/paket/müşteri bağlamı ve arama: `client/src/components/orders/OrderScreen.jsx:642-720`
- Ödeme açma koşulu: `client/src/components/orders/OrderScreen.jsx:590-604`
- Ürün grid'i: `client/src/components/orders/OrderScreen.jsx:730-895`
- Adisyon paneli: `client/src/components/orders/OrderScreen.jsx:899-1180`
- Müşteri seçimi modalı: `client/src/components/orders/OrderScreen.jsx:1297-1415`

Güçlü yönler:

- Sol ürün grid'i + sağ adisyon paneli doğru POS modelidir.
- Topbar'da masa, salon alanı, müşteri ve paket telefon bilgisi bağlamsal olarak gösteriliyor.
- `Mevcut Ürünler` ve `Yeni Eklenen` ayrımı iyi bir hata önleme kuralıdır (`OrderScreen.jsx:936` , `OrderScreen.jsx:1037`).
- Kaydedilmemiş ürün varken ödeme aksiyonlarının gizlenmesi hata önüyor (`OrderScreen.jsx:590` , `OrderScreen.jsx:1161`).
- `Kaydet ve Mutfağa Gönder` metni, iş kuralı ile UI dilini daha doğru hizalıyor (`OrderScreen.jsx:1159`).

Kullanılabilirlik kusurları:

- Ürün kartları 200px minimum grid ile düzenleniyor; hızlı servis için ürün yoğunluğu iyi ama ürün fotoğrafı veya kategori rengi yoksa ürünler metin listesine dönüşebilir.
- Ürün kartında miktar kontrolü sağda kırmızı bir dikey alana dönüşüyor. Kırmızı renk genelde tehlike/silme anlamı taşır; burada miktar kontrolü için agresif.
- Sağ adisyon paneli 460px sabit genişlikte. Düşük çözünürlüklü POS cihazlarında ürün grid alanını fazla daraltabilir.
- `Kaydet ve Mutfağa Gönder` butonu doğru ama uzun. Küçük ekran veya Türkçe uzun metinlerde satır kırılması riski var.
- Kayıtlı satır, gönderilmiş satır, ikram, iptal gibi durumlar var ama adisyon panelinde bu durumların kullanıcı mental modeli için açıklaması yok.

Görsel kalite kusurları:

- Sipariş ekranı çok fazla inline style içeriyor. Bu, tek tek bakıldığında çalışır ama sistematik tasarım kalitesini zayıflatır.

- Bazı mikro ikonlar emoji ile verilmiş: not göstergesinde 📝. Bu, kurumsal POS hissini hafifletiyor.
- Topbar iyi düşünülmüş ama geri alanı büyük ve arama alanı ile yarışıyor; birincil görev "ürün ara/ekle" mi, "geri dön" mü ilk bakışta netleşmeli.

Profesyonel ürün hissini bozan noktalar:

- Müşteri seçimi modalı hem arama, hem yeni müşteri formu, hem seçili müşteri, hem adres alanını aynı yüzeye koyuyor. Paket sipariş için gerekli ama modüler akış hissi zayıf.
- Kayıtlı siparişe yeni ürün ekleme ile yeni sipariş oluşturma aynı butonla çözülüyor; metin doğru, fakat "bu ürünler mutfağa yeni fiş olarak gidecek" görsel olarak daha güçlü olmalı.

2.3 Sağ panel / adisyon paneli

Güçlü yönler:

- Sabit sağ panel POS kullanımında doğru. Kullanıcı toplamı ve adisyonu sürekli görür.
- Alt toplam alanı sticky gibi davranıyor ve ana aksiyonu en altta tutuyor.
- Yeni eklenen satırların accent sol çizgi ile ayrılması iyi.

Kullanılabilirlik kusurları:

- Mevcut kalemlerde miktar değişimi bazı durumlarda küçük `+/-` butonlarıyla, bazı durumlarda salt miktar metniyle gösteriliyor. Durum farkı doğru ama neden düzenlenemediği satırda açıklanmıyor.
- Silme/void ikonları küçük ve yıkıcı. Yoğun dokunmatik kullanımda yanlış void riski var.
- Toplam alanında `Ara toplam` ve `Toplam` var; KDV/indirim yoksa bu iki değer kullanıcı için gereksiz tekrar gibi algılanabilir.

Görsel kalite kusurları:

- Panelde satır yoğunluğu yüksek. Sipariş büyüdükçe her satır fiyat, modifier, not, status, miktar ve silme ikonuyla kalabalıklaşıyor.
- Alt toplam alanının güçlü gölgesi modern ama biraz "web dashboard" hissi veriyor; POS'ta daha düz, yüksek kontrastlı kasa paneli daha net olabilir.

2.4 Ödeme ve hızlı ödeme akışı

Kanıt:

- Ayrıntılı ödeme action tipleri: `client/src/components/payments/PaymentScreen.jsx:13-18`
- Ayrıntılı ödeme ekranı/modalı: `client/src/components/payments/PaymentScreen.jsx:190`
- Split ödeme girişi: `client/src/components/payments/PaymentScreen.jsx:255`
- Tam ödeme sonrası kapatma: `client/src/components/payments/PaymentScreen.jsx:361-386`
- İşlem aksiyonu, ödeme tipi, alınacak tutar: `client/src/components/payments/PaymentScreen.jsx:397-452`
- Hızlı ödeme operation tipleri ve tam ödenmiş kapatma: `client/src/components/payments/QuickPaymentModal.jsx:7-12`, `QuickPaymentModal.jsx:40-57`

Güçlü yönler:

- Ayrıntılı ödeme ekranı kısmi ödeme ve split ödeme gibi gerçek restoran ihtiyaçlarını destekliyor.
- Tam ödenmiş hesapta yeni ödeme almak yerine `Yazdır` ve `Masayı Kapat` gösterilmesi doğru.
- Hızlı ödeme modalı tam bakiye ödemesi için sade; iki ödeme tipi büyük hedeflerle sunuluyor.

- Hızlı ödeme artık hesap ödenmişse kapatma aksiyonuna dönüşüyor; bu iş akışı iyi toparlıyor.

Kullanılabilirlik kusurları:

- `Kaydet`, `Öde ve Kapat`, `Öde ve Yazdır`, `Öde, Yazdır ve Kapat` aynı seçim grubunda. "Kaydet" ödeme bağlamında tahsilatı kaydetmek demek, sipariş ekranındaki "Kaydet" ile anlam çakışması yaratabilir.
- `Öde ve Yazdır` ile `Öde, Yazdır ve Kapat` ayrımı hala bilişsel yük yaratıyor. Kullanıcı fis basınca masanın kapanıp kapanmadığını hızlı anlayamayabilir.
- Ayrıntılı ödeme modalı çok büyük ve yoğun. Sol tarafta kalemler, sağda toplam/aksiyon/tip/tutar var; kasiyer için güçlü ama garson için ağır.
- "Tümünü Al" ile "Kalanı Al" kısmi ödeme senaryosunda birbirine yakın ve her zaman anlamlı değil.

Görsel kalite kusurları:

- Ödeme ekranında kart içinde kart hissi var: özet kutuları, kalan kutusu, aksiyon kutusu, ödeme tipi kutusu, tutar kutusu.
- Aksiyon butonlarının tone sistemi teknik olarak var ama ürün dili daha güçlü değil: final kapatma aksiyonu en baskın işlem olmalı.

Profesyonel ürün hissini bozan noktalar:

- Ödeme aksiyonları çok teknik kombinasyonlar gibi görünüyor. Profesyonel POS'ta "Tahsil Et", "Tahsil Et ve Masayı Kapat", "Hesap Yazdır" gibi iş dili daha net olmalı.
- Hızlı ödeme ve ayrıntılı ödeme aynı kavramları farklı layout içinde sunuyor. Kullanıcı öğrenme maliyeti artıyor.

2.5 Modal yapıları

Kanıt:

- Global modal sistemi: `client/src/styles/global.css:686-720`
- Masa action sheet modalı: `client/src/components/tables/TablesScreen.jsx:1089-1167`
- Sipariş müşteri modalı: `client/src/components/orders/OrderScreen.jsx:1297-1415`
- Hızlı ödeme modalı: `client/src/components/payments/QuickPaymentModal.jsx:90-240`

Güçlü yönler:

- Modal altyapısı tutarlı sınıflar kullanıyor: `.modal`, `.modal-header`, `.modal-body`, `.modal-footer`.
- ESC ve dış tıklama bazı menü/modal durumlarında düşünülmüş.
- Action sheet yaklaşımı dokunmatik POS için doğru yönde.

Kullanılabilirlik kusurları:

- Her modal aynı davranış tipinde değil. Bazıları hızlı aksiyon sheet, bazıları form, bazıları detay ekranı gibi.
- Yıkıcı işlemler için bazı yerlerde özel confirm modalı var, bazı yerlerde `window.confirm` var. Bu tutarsız.
- Modal kapatma ikonları genellikle 36px civarı; yoğun dokunmatik kullanım için 44px hedef daha güvenli.

Görsel kalite kusurları:

- `.modal` radius 16px (`global.css:686`) operasyonel POS için biraz fazla yumuşak. Modern olabilir, ama cihaz üstü kasa uygulamasında daha keskin ve stabil görünüm tercih edilebilir.
- Büyük modallar web uygulaması hissi veriyor; kiosk/POS ekranında tam ekran panel veya bottom sheet ayrımı daha profesyonel olur.

2.6 Ayarlar ekranı

Kanıt:

- Ayarlar ana ekranı kartları: `client/src/components/settings/SettingsHome.jsx`
- Ayarlar layout sadece `Outlet`: `client/src/components/settings/SettingsLayout.jsx`
- Yazıcı detay ekranı teknik form ve preview: `client/src/components/settings/PrinterDetailPage.jsx:1-180`
- Ayar ekranı dosya boyutları: `PrinterDetailPage.jsx` 44 KB, `MenuSettingsPage.jsx` 35 KB, `MenuProductEditorPage.jsx` 21 KB.

Güçlü yönler:

- Ayarlar ana ekranı kategorileri kartlarla ayırıyor, kullanıcı yön bulabiliyor.
- Yazıcı önizleme paneli var; bu çok değerli, çünkü yazıcı ayarı görsel geri bildirim ister.
- Menü tanımları, salon bölgeleri, yazıcı yönlendirme gibi domainler ayrı route'lara bölünmüş.

Kullanılabilirlik kusurları:

- `SettingsLayout` yalnız `Outlet` döndürüyor. Ayarlar alt ekranlarında ortak bağlam, breadcrumb, ayarlar navigasyonu veya güvenli çıkış modeli yok.
- Yazıcı ayarları çok teknik seçenekleri aynı formda topluyor: `escT`, `skipInit`, `skipPhoenixCmd`, `encodingMode`, `lineWidth`. Bu alanlar sahadaki işletmeci için anlaşılır değil.
- Ayarlar ana ekranındaki açıklama metinleri bazı yerde geliştirici dili içeriyor: "simülasyonu (test)", "Ayarlar > Yazıcılar".

Görsel kalite kusurları:

- Ayarlar ana ekranı temel kart grid'i iyi ama operasyon ekranlarından kopuk, daha standart admin panel hissinde.
- Yazıcı preview kağıdı iyi fikir ama çevresindeki teknik form yoğunluğu profesyonel kurulum sihirbazı hissi vermiyor.

Profesyonel ürün hissini bozan noktalar:

- Kritik teknik ayarlar "gelişmiş" katmanına ayrılmamış.
- Yazıcı/encoding gibi hata riski yüksek alanlarda adım adım kurulum, test fişi ve son doğrulama akışı yok.

3. Kritik kullanılabilirlik sorunları

P1 - Kritik masa aksiyonları gizli ve aynı ağırlıkta

- Kanıt: Masa işlemleri `...` menüsü ve action sheet ile açılıyor (`TablesScreen.jsx:706`, `TablesScreen.jsx:1089-1167`).
- Etki: Garson veya kasiyer yoğun anda `Öde`, `Hızlı Öde`, `Masayı Kapat` gibi işlemleri hızlı bulamayabilir.
- Kök neden: Kart birincil tıklaması sipariş açma, işlem menüsü ikincil küçük ikon. Action sheet içinde tüm aksiyonlar benzer grid tile.
- Öneri: Dolu masada kart üzerinde 1 adet bağlamsal ana aksiyon göster: borç varsa `Öde`, tam ödendiye `Masayı Kapat`, hazır varsa `Siparişi Aç`.

P1 - Ödeme dili ve yazdırma/kapatma kombinasyonları fazla bilişsel yük yaratıyor

- Kanıt: `paymentActions` dört kombinasyon içeriyor (`PaymentScreen.jsx:13-18`).
- Etki: Kullanıcı fis bastığında masanın kapanıp kapanmadığını karıştıracaktır.
- Kök neden: Teknik boolean kombinasyonları UI aksiyonu olarak sunulmuş: `closeOrder` , `printReceipt` .
- Öneri: İş diliyle yeniden düzenle: `Tahsil Et` , `Tahsil Et ve Masayı Kapat` , `Hesap Yazdır` , `Final Fiş ve Kapat` .

P1 - Paket sipariş yan paneli ödeme ve teslimat kararlarını sıkışık gösteriyor

- Kanıt: Paket kartı toplam, süre, durum, ödeme etiketi, menü ve iki teslimat butonunu tek kartta topluyor (`TablesScreen.jsx:784-1056`).
- Etki: Paket operasyonu arttığında kartlar hem bilgi hem aksiyon yoğunluğu nedeniyle yavaşlar.
- Kök neden: Paket sipariş ana ekran yerine masa ekranı yan paneline gömülü.
- Öneri: V1'de yan panel korunabilir ama kart hiyerarşisi sadeleşmeli: müşteri, tutar, ödeme durumu, tek sonraki aksiyon. V2'de ayrı paket operasyon ekranı.

P2 - Dokunmatik hedefler bazı kritik mikro aksiyonlarda sınırda

- Kanıt: Global `.btn-icon` min-width 36 ve min-height auto (`global.css:617`); ürün grid +/- butonları inline 4px padding ile çalışıyor; satır silme ikonları 4px paddingli.
- Etki: Yanlış azaltma/silme/menü açma riski.
- Kök neden: Desktop mouse ergonomisi ile dokunmatik POS ergonomisi aynı tasarım sisteminde karışmış.
- Öneri: Operasyon ekranları için minimum 44x44 px hit target policy.

P2 - Ayarlar ekranı teknik kullanıcıya göre tasarlanmış, işletmeciye göre değil

- Kanıt: Yazıcı detay ekranı çok sayıda teknik state ve seçenek taşıyor (`PrinterDetailPage.jsx:1-180`).
- Etki: Yanlış yazıcı/encoding ayarı canlı işletmede fiş bozulması veya yazdırmama riski yaratır.
- Kök neden: Kurulum sihirbazı yerine teknik form yaklaşımı.
- Öneri: Basit/gelişmiş ayrımı, profil seçimi, test fişi, son doğrulama.

4. Görsel kalite ve profesyonellik sorunları

P1 - Tasarım sistemi var ama uygulama yüzeyi inline stillerle parçalanıyor

- Kanıt: Operasyon ekranlarında büyük miktarda inline style kullanımı var: masa kartı, paket kartı, sipariş grid'i, adisyon paneli, ödeme kartları.
- Etki: Aynı tip UI parçaları küçük farklarla çoğalır; profesyonel sistem hissi azalır.
- Kök neden: Component tokenları ve varyant sınıfları yeterli kullanılmıyor.
- Öneri: `pos-action-tile` , `pos-status-chip` , `pos-order-line` , `pos-summary-panel` , `pos-touch-icon-button` gibi sınıflar/komponentler çıkar.

P1 - Renk dili operasyonel anlamdan çok tema hissi taşıyor

- Kanıt: Accent koyu temada `#6366f1` , light temada `#6C63FF` (`global.css:25` , `global.css:126`); gradient ve mor/purple vurgular mevcut.

- Etki: Restoran POS gibi hızlı ve hata önleyici üründe mor/purple dashboard estetiği, durum renklerinin operasyonel netliğini azaltabilir.
- Kök neden: Genel SaaS/dashboard paleti POS domainine uyarlanmış.
- Öneri: Accent'i daha nötr/kurumsal bir renge çek, durum renklerini sadece operasyon anlamı için kullan: yeşil odendi/hazır, sarı aktif/uyarı, kırmızı risk/iptal, mavi transfer/bilgi.

P2 - Border radius ve gölge dili fazla yumuşak

- Kanıt: `--radius-md: 12px`, `--radius-lg: 16px`, `--radius-xl: 20px` (`global.css:90-94`); `.modal` radius `var(--radius-lg)` (`global.css:686`).
- Etki: Bazı ekranlarda modern ama oyuncak/web dashboard hissi oluşuyor.
- Kök neden: Tüm ürün aynı yumuşak kart diline yaslanıyor.
- Öneri: Operasyon ekranlarında radius 8px standardına yaklaş; ayarlar/rapor gibi düşük frekanslı ekranlarda daha yumuşak dil kalabilir.

P2 - Kart içinde kart ve kutu yoğunluğu

- Kanıt: Ödeme ekranında özet kutuları, kalan kutusu, aksiyon kutusu, ödeme tipi kutusu, tutar kutusu aynı modal içinde.
- Etki: İlk bakışta "şimdi ne yapmalıyım?" sorusu yavaş cevaplanır.
- Kök neden: Her bilgi grubu ayrı kartlaştırılmış.
- Öneri: Ödeme ekranında tek güçlü toplam paneli ve tek ana aksiyon alanı bırak; yardımcı seçenekleri ikincil yap.

5. UI davranış matrisi

Durum	Görünmesi gereken ana aksiyon	Gizli/disabled olması gerekenler	Mevcut durum	UX kararı
Boş masa	Masayı aç / sipariş başlat	Öde, hızlı öde, yazdır, kapat, taşı	Kart tıklaması sipariş açıyor	Uygun, ama kartta ana aksiyon etiketi eklenmeli
Rezerve masa	Rezervasyonu kullanarak sipariş aç	Onaysız açma, hedef transfer	Onay eklendi, transfer engeli var	Uygun, native modal ile iyileştirilmeli
Dolu masa, borç var	Öde veya siparişi aç	Masayı kapat	Ödeme menü içinde, kapat gizli	Ana aksiyon daha görünür olmalı
Dolu masa, hesap ödendi	Masayı kapat	Yeni ödeme alma	HESAP ÖDENDİ , kapatma menüde	Kart üzerinde direkt kapat önerilir
Dolu masa, hazır ürün var	Siparişi aç / servis aksiyonu	Kapat, ilgisiz aksiyonlar	HAZIR rozeti var	İyi, ama hazır ve ödendi önceliği netleşmeli
Transfer modu	Hedef boş masayı seç	Rezerve/dolu hedef, kaynak tekrar seçimi dışında tüm işlemler	Banner var, hedef olmayanlar tıklanabilir ama reddediliyor	Dolu/rezerve hedefler görsel disabled olmalı
Yeni sipariş, sepet boş	Ürün ekle	Kaydet, ödeme, hızlı ödeme	Boş sepet mesajı	Uygun
Yeni sipariş, sepet dolu	Kaydet ve Mutfağa Gönder	Ödeme, hızlı ödeme	Tek ana buton var	Uygun
Kayıtlı sipariş, yeni ürün eklendi	Kaydet ve Mutfağa Gönder	Ödeme, hızlı ödeme	Tek ana buton var	Uygun
Kayıtlı sipariş, sepet boş	Ödeme / Hızlı Öde	Kaydet	İki aksiyon var	Uygun, ana öneri durum bazlı seçilmeli
Kayıtlı sipariş, tamamen ödendi	Masayı kapat / yazdır	Yeni ödeme alma	Ödeme ekranında doğru, masada menü içinde	Masada daha görünür olmalı
Paket sipariş, müşteri yok	Müşteri seç	Kaydet/gönder	Uyarı ve modal var	Uygun
Paket sipariş, hazırlanıyor, ödenmedi	Ödeme al veya teslimata çıkarma kuralına göre sıradaki aksiyon	Teslim edildi	Ödeme etiketi var, teslim disabled	Disabled sebebi touch cihazda görünür olmalı
Paket sipariş, teslimatta, ödendi	Teslim edildi	Tekrar teslimata çıkar	Doğru	Uygun
Paket sipariş, teslimatta, ödenmedi	Ödeme al	Teslim edildi	Teslim disabled	Kartta "Ödeme alınmadan teslim edilemez" görünür olmalı
Hızlı ödeme, borç var	Nakit/Kart ile tahsil et	Kısmi tutar	Büyük butonlar var	Uygun
Hızlı ödeme, borç yok	Masayı kapat	Ödeme tipi seçimi	Artık kapatma gösteriyor	Uygun
Ayrıntılı ödeme, borç var	Seçilen ödeme aksiyonunu tamamla	Kapatma için kısmi ödeme	Kontrol var	Uygun, dil sadeleşmeli
Ayrıntılı ödeme, borç yok	Yazdır / Masayı Kapat	Yeni ödeme alma	Doğru	Uygun

6. Amatör görünen UI alanları

1. Browser confirm kullanımı.
 - Rezerve masa açma ve transfer gibi kritik işlemlerde native confirm, ürünün kendi güvenilirlik hissini kırar.
1. Teknik aksiyon kombinasyonlarının doğrudan butona dönüşmesi.
 - `Öde ve Yazdır`, `Öde, Yazdır ve Kapat` gibi metinler backend boolean kombinasyonları gibi duruyor.
1. Inline style yoğunluğu.
 - Profesyonel UI sistemi olan ürünlerde operasyon ekranları tasarım tokenları ve component varyantları üzerinden yönetilir.
1. Mikro ikon butonlarının kritik işlemler taşıması.
 - Üç nokta menüsü, çöp ikonları, küçük +/- kontrolleri dokunmatik hata riskini artırır.
1. Ayarlar ekranında teknik ham seçeneklerin işletmeciyeye açılması.
 - `escT`, `skipPhoenixCmd`, encoding gibi alanlar gelişmiş profil arkasına alınmalı.
1. Aynı aksiyonun farklı ekranlarda farklı sunulması.
 - Masa ekranında ödeme action sheet içinde; sipariş ekranında sağ panel altında; ödeme ekranında modal içinde; hızlı ödeme ayrı modalda.
1. Paket siparişin masa ekranına yan panel olarak sıkışması.
 - V1 için pratik, ancak profesyonel paket operasyonu için ayrı queue daha doğru.

7. Hemen düzeltilmesi gerekenler

P1 - Masa kartında durum bazlı ana aksiyon görünür olmalı

- Dolu ve borçlu masa: `Öde` veya `Siparişi Aç`.
- Dolu ve ödenmiş masa: `Masayı Kapat`.
- Hazır ürünlü masa: `Siparişi Aç` / `Servis` aksiyonu.
- Amaç: Kritik işi üç nokta menüsünden çıkarmak.

P1 - Ödeme aksiyon dili sadeleştirilmeli

- `Kaydet` ödeme bağlamında `Tahsilatı Kaydet` olmalı.
- `Öde ve Yazdır` yerine fis türü netleşmeli.
- Kapatma içeren aksiyonlarda "Masayı Kapat" ifadesi açık olmalı.

P1 - Disabled aksiyonların nedeni dokunmatik cihazda görünür olmalı

- Paket siparişte `Teslim Edildi` disabled ise sebep kart üzerinde görünmeli.
- Tooltip/title yeterli değil.

P2 - Touch target standardı sertleştirilmeli

- Operasyon ekranlarında tüm kritik hedefler minimum 44x44 px olmalı.
- Satır silme, +/- ve üç nokta butonları bu standarda çekilmeli.

P2 - Ayarlar yazıcı ekranı basit/gelişmiş olarak ayrılmalı

- İşletmeci için: yazıcı adı, tür, bağlantı, test fişi, varsayılan yap.
- Gelişmiş için: encoding, ESC t, Phoenix komutu, line width.

8. Sonraki turda uygulanabilecek düşük riskli iyileştirmeler

1. Masa action sheet'te aksiyon önceliği düzenle.

- `Öde` ve `Masayı Kapat` en üstte ve daha geniş; `İptal` ayrı danger bölgesinde.

1. Masa kartına bağlamsal küçük CTA ekle.

- Borçlu: `Öde`.
- Ödenmiş: `Kapat`.
- Boş: `Sipariş aç`.

1. Paket kartında disabled sebep satırı göster.

- "Teslim için ödeme tamamlanmalı" gibi kalıcı metin.

1. Global `btn-icon` ve operasyon ikon butonlarını 44px hedefe çek.

- Özellikle masa menüsü, satır silme, ürün +/-.

1. Ödeme aksiyon metinlerini iş diline çevir.

- Davranış değiştirmeden sadece copy ve hiyerarşi düzeni.

1. `window.confirm` yerine mevcut modal sistemiyle confirm bileşeni kullan.

- Rezerve masa ve masa transfer onayları için.

1. Sipariş panelinde `Mevcut Ürünler` ve `Yeni Eklenen` ayrımını güçlendir.

- Yeni ürün bölümünde "Mutfağa gönderilmedi" çipi.

1. Ayarlar ana ekranında operasyonel olmayan teknik ifadeleri temizle.

- "simülasyonu (test)" ve "Ayarlar > Yazıcılar" gibi iç/dokümantasyon dili kaldırılmalı.

1. Yazıcı ayarında "Gelişmiş ayarlar" katmanı aç/kapa yapılmalı.

- Varsayılan görünüm kurulum sihirbazı gibi sade olmalı.

1. UI sistem borcu için küçük component listesi çıkar.

- `StatusChip`, `ActionTile`, `TouchIconButton`, `OrderLine`, `SummaryPanel`, `ConfirmDialog`.

Bu iyileştirmeler düşük risklidir; çoğu davranış değiştirmeden copy, hiyerarşi, görünürlük ve dokunmatik hedef standardı iyileştirmesi olarak uygulanabilir.

03 - Frontend Architecture Audit

Tarih: 2026-04-14 Kapsam: frontend state kaynakları, route yapısı, ekranlar arası veri akışı, sipariş/masa/ödeme/ayarlar ekranları, component sorumlulukları, tekrar eden mantıklar ve kırılğan koşullar. Sınır: Bu raporda uygulama kodu değiştirilmedi; yalnızca analiz, state akışı çıkarımı ve raporlama yapıldı.

Bağlam olarak dikkate alınan raporlar:

- docs/audit/00-overall-audit.md
- docs/audit/00a-prioritized-action-plan.md
- docs/audit/00b-first-hardening-pass.md
- docs/audit/01-product-business-rules.md
- docs/audit/02-ux-ui-audit.md

Not: Bu denetim mevcut çalışma ağacının güncel haline göre yapıldı. Önceki UX önerilerinden bazıları uygulanmış, bazıları kullanıcı tercihiyle geri alınmış durumda. Örneğin masa kartındaki bağlamsal `Sipariş Aç / Öde` CTA kaldırılmış; `Kaydet` ve `Mutfağa Gönder` dili de kullanıcı tercihiyle `Kaydet` olarak bırakılmıştır.

1. Mevcut frontend mimarisi özeti

Frontend React 18 + Vite tabanlı, ekran bazlı klasörlere ayrılmış bir POS uygulaması. Ana klasör yapısı:

- client/src/components : ekran ve UI bileşenleri.
- client/src/context : auth, toast, socket, incoming call gibi global contextler.
- client/src/services : tekil API client.
- client/src/constants : ortak enum/format yardımcıları.
- client/src/utills : küçük yardımcılar.
- client/src/styles : global CSS ve tasarım tokenları.

Ana route ve uygulama orkestrasyonu `client/src/App.jsx` içinde. Route componentleri `React.lazy` ile bölünmüş; bu önceki bundle riskine doğru bir cevap. Kanıt: `client/src/App.jsx:10-32`.

Mevcut yüksek seviyeli mimari:

- `AuthContext` kullanıcı/token state'ini ve role kontrolünü sağlar.
- `App.jsx` route ağacını, sidebar görünürlüğü, ödeme/hızlı ödeme modal state'ini ve sipariş ekranı wrapper'ını yönetir.
- Ekran bileşenleri kendi verilerini doğrudan `api` servisinden çeker.
- Socket context yalnız event subscribe mekanizması sağlar; domain eventlerinin nasıl yorumlanacağı ekranlara bırakılmıştır.
- Backend kaynaklı veriler çoğunlukla ekran state'lerinde tutulur; merkezi query/cache katmanı yoktur.

Dosya boyutu ve sorumluluk yoğunluğu kanıtı:

Dosya	Satır	Değerlendirme
client/src/components/orders/OrderScreen.jsx	1509	P1 - Sipariş domaini, sepet, müşteri, transfer, ödeme gating, modal ve UI tek dosyada.
client/src/components/tables/TablesScreen.jsx	1444	P1 - Masa grid, transfer, paket yan panel, caller modal, action sheet ve socket/polling tek dosyada.
client/src/components/settings/PrinterDetailPage.jsx	993	P1 - Yazıcı formu, discovery, preview, encoding, print options ve kayıt akışı tek dosyada.
client/src/components/settings/MenuSettingsPage.jsx	869	P2 - Kategori, ürün, sıralama, bulk işlem ve modal state aynı dosyada.
client/src/components/reports/ReportsScreen.jsx	804	P2 - Günlük rapor, kapalı siparişler, export, grafik ve filtre state'i aynı ekranda.
client/src/components/payments/SplitPaymentModal.jsx	680	P2 - Ayrı ödeme state makinesi modal içinde yoğun.
client/src/components/payments/PaymentScreen.jsx	497	P2 - Ödeme state'i ve UI birlikte; bazı mantıklar hızlı ödeme ile tekrar ediyor.
client/src/services/api.js	227	P2 - Tüm domain endpointleri tek service/facade içinde.

Genel karar: Klasör ayrımı yüzeyde iyi, fakat gerçek mimari ekran dosyalarında toplanmış. Component sınırları "domain davranışı" üzerinden değil, çoğunlukla "ekran" üzerinden kurulmuş. Bu V1 için çalışır ama ürün büyüdükçe hızlı bozulur.

2. State akışı özeti

2.1 Global state kaynakları

State kaynağı	Dosya	Sorumluluk	Risk
Auth/user/token	client/src/context/AuthContext.jsx:8-36	Kullanıcı, loading, login/logout, token set/reset.	Orta. Token <code>api</code> singleton ve <code>localStorage</code> ile çift bağlı.
Toast	client/src/context/ToastContext.jsx:7-17	Geçici bildirimler.	Düşük. Basit ve yeterli.
Socket	client/src/context/SocketContext.jsx:13-76	Socket bağlantısı ve event subscribe.	Orta. Domain event yorumları ekranlara dağılmış.
Incoming Call	client/src/context/IncomingCallContext.jsx:50-151	Caller ID polling, banner, dismiss/open order.	Orta-yüksek. UI, polling ve order navigation callback'i aynı contextte.
API token	client/src/services/api.js:5-12	Token <code>ApiService</code> instance içinde ve <code>localStorage</code> da.	Orta. AuthContext ile sıkı coupling.

Global state yeterince hafif tutulmuş; bu olumlu. Fakat veri fetching ve cache için bir katman yok. Her ekran kendi API çağrısını, loading/error state'ini ve refresh stratejisini kendi içinde tekrar kuruyor.

2.2 Route ve ekranlar arası veri akışı

Route yapısı `App.jsx` içinde hem erişim kontrolünü hem de ekran wiring'ini yönetiyor:

- Role bazlı erişim `ProtectedRoute` ile yapılıyor: `client/src/App.jsx:34-46`.
- Masa ekranından siparişe geçiş route state ile taşıyor: `client/src/App.jsx:73-75`.
- Paket sipariş state'i yine route state ile taşıyor: `client/src/App.jsx:77-88`.
- Ödeme ve hızlı ödeme modal state'i `App.jsx` içinde tutuluyor: `client/src/App.jsx:50-52`.
- Ödeme sonrası dönülecek yer string ref ile belirleniyor: `client/src/App.jsx:94-124`.
- `OrderScreenWrapper`, route `location.state` içinden table/order/customer/callLog bilgilerini alıp props olarak geçiriyor: `client/src/App.jsx:260-286`.

Bu akış pratik ama kırılgan:

- `location.state` sayfa yenilemede kaybolabilir.
- Ödeme sonrası davranış `paymentAfterCompleteRef.current = 'back' | 'tables'` gibi string state ile yönetiliyor.
- `App.jsx` route config, payment modal orchestration, sidebar state ve display settings side effect'ini aynı yerde taşıyor.

2.3 Sipariş ekranı state akışı

`OrderScreen.jsx` aynı anda şu state kaynaklarını taşıyor:

- Menü verisi: `categories`, `products`, `activeCat`, `searchQuery`, `categoriesLoading` (`OrderScreen.jsx:26-36`).
- Lokal sepet: `cartItems` (`OrderScreen.jsx:30`).
- Backend siparişi: `existingOrder` (`OrderScreen.jsx:31`).
- Ürün detay/modifier modal state'i: `modifierModal`, `lineDetailModal` (`OrderScreen.jsx:32-34`).
- Transfer state'i: `moveModalOpen`, `emptyTables` (`OrderScreen.jsx:37-38`).
- Paket/müşteri state'i: `takeawayPhone`, `selectedCustomer`, `customerModalOpen`, `customerSearchQuery`, `customerList`, `newCustomer`, `selectedTakeawayAddress` (`OrderScreen.jsx:39-47`).
- Confirm modal state'i: `confirmDialog` (`OrderScreen.jsx:48`).

Ana state flow:

1. Kategori/ürünler ekran açılınca yüklenir: `OrderScreen.jsx:54`, `OrderScreen.jsx:107-123`.
2. Var olan sipariş `existingOrderId || table?.current_order_id` ile yüklenir: `OrderScreen.jsx:58-61`, `OrderScreen.jsx:126-129`.
3. Ürünler önce lokal `cartItems` içine eklenir: `OrderScreen.jsx:187-264`.
4. `handleSaveOrder`, mevcut sipariş varsa `api.addOrderItems`, yoksa `api.createOrder` çağırır: `OrderScreen.jsx:325-373`.
5. Topamlar frontendde `savedTotal + cartSubtotal` olarak türetilir: `OrderScreen.jsx:603-607`.
6. Ödeme açma koşulu `existingOrder && !hasUnsavedChanges && existingOrder.status !== 'closed' && activeExistingItems.length > 0`: `OrderScreen.jsx:611-612`.

Bu model iş kuralını açıkça UI içine gömüyor. Özellikle `hasUnsavedChanges` ve `canOpenPayment` ödeme görünürlüğü için kritik policy, ama ekrana özel local expression olarak kalmış durumda.

2.4 Masa ekranı state akışı

`TablesScreen.jsx` yoğun bir state merkezi gibi davranıyor:

- Masa alanları ve aktif alan: `areas`, `activeArea`, `loading` (`TablesScreen.jsx:39-41`).
- Transfer modu: `transferMode` (`TablesScreen.jsx:42`).
- Paket sipariş queue: `takeawayOrders` (`TablesScreen.jsx:43`).
- Caller ID modal/history: `callModalOpen`, `callHistory`, `callHistoryLoading` (`TablesScreen.jsx:44-46`).
- Saat/timer: `now` (`TablesScreen.jsx:47`).
- Masa menü/confirm/cancel state'leri: `openMenuTableId`, `tableCancelTarget`, `tableCancelLoading`, `tableConfirm` (`TablesScreen.jsx:48-55`).
- Paket menü/confirm/loading state'leri: `openTakeawayMenuId`, `takeawayConfirmTarget`, `takeawayActionLoading` (`TablesScreen.jsx:51-53`).
- Transfer highlight state'i: `recentlyMovedTableId` (`TablesScreen.jsx:54`).

Ana side effect akışı:

- `loadTables` hem data yükler hem aktif area kararını verir: `TablesScreen.jsx:64-80`.
- `loadTakeaway` paket siparişleri ayrı endpointten çeker: `TablesScreen.jsx:83-91`.
- Route state değişimi, socket eventleri ve fallback polling aynı ekranı refresh eder: `TablesScreen.jsx:93-127`.
- `now` her saniye güncellenir: `TablesScreen.jsx:129-132`.
- ESC ve outside click davranışı local effectlerde yönetilir: `TablesScreen.jsx:140-161`.

Bu ekran masa operasyonu ile paket operasyonunu tek componentte birleştiriyor. Masa grid'i, paket yan paneli, caller ID modalı, transfer confirm'i ve action sheet aynı dosyada. Bu, frontend mimarisinin en kırılgan alanlarından biri.

2.5 Ödeme/hızlı ödeme state akışı

Ödeme domaini üç ayrı componentte dağılmış:

- Ayrıntılı ödeme: `PaymentScreen.jsx`.
- Hızlı ödeme: `QuickPaymentModal.jsx`.
- Ayrı ödeme/split: `SplitPaymentModal.jsx`.

Tekrar eden hesaplama:

- `PaymentScreen` kendi `round2`, `paidTotal`, `totalDue`, `isFullyPaid` hesabını yapıyor: `PaymentScreen.jsx:20`, `PaymentScreen.jsx:61-67`.
- `QuickPaymentModal` aynı hesabı tekrar ediyor: `QuickPaymentModal.jsx:19`, `QuickPaymentModal.jsx:36-39`.
- `TablesScreen` masa kartı için benzer "tam ödendi mi" hesabı yapıyor: `TablesScreen.jsx:284-289`.
- `TablesScreen` paket ödeme durumu için ayrı helper yazmış: `TablesScreen.jsx:291-300`.
- `SplitPaymentModal` kendi payer/item allocation hesaplarını modal içinde tutuyor: `SplitPaymentModal.jsx:46-57`, `SplitPaymentModal.jsx:80-112`, `SplitPaymentModal.jsx:185-225`.

Bu tek başına bug değildir, ancak ödeme gibi kritik bir domain için risklidir. Tam ödeme toleransı `0.02`, kalan tutar, paid/open/closed ayrımı ve "masa kapatılabilir mi" gibi kurallar UI içinde farklı yerlerde tekrar edilirse, bir sonraki değişiklikte ekranlar ayrışır.

2.6 Ayarlar state akışı

Ayarlar route'ları bölünmüş; fakat her büyük ayar ekranı kendi küçük uygulaması gibi:

- `SettingsLayout` yalnız `Outlet` döndürüyor: `client/src/components/settings/SettingsLayout.jsx:1-4`. Ortak settings shell, breadcrumb, kaydedilmemiş değişiklik guard'ı, ortak form davranışı yok.
- `PrinterDetailPage` hem yazıcı formunu hem StoreBridge discovery'yi hem preview fetch'ini hem encoding seçeneklerini hem dirty tracking'i yönetiyor: `PrinterDetailPage.jsx:156-202`, `PrinterDetailPage.jsx:245-332`, `PrinterDetailPage.jsx:390-459`.
- Printer preview ayrı küçük component içinde debounced API call yapıyor: `PrinterDetailPage.jsx:65-106`. Bu iyi bir başlangıç, ama aynı dosyada kalmış.
- Kaydedilmemiş değişiklik guard'ı hala browser confirm ile çalışıyor: `PrinterDetailPage.jsx:386`.

Ayarlar tarafında ana sorun domain form modellerinin UI state'i olarak tutulması. `printOptions` gibi nested yapı çok sayıda `setPrintOptions` callback'ile güncelleniyor: `PrinterDetailPage.jsx:339-379`, `PrinterDetailPage.jsx:511-512`, `PrinterDetailPage.jsx:883-979`.

3. Kritik state sorunları

P1 - Sipariş state'i tek ekranda aşırı parçalı ve çok kaynaklı

Kanıt:

- Lokal sepet `cartItems`, backend sipariş `existingOrder`, müşteri state'i `selectedCustomer`, paket adresi `selectedTakeawayAddress`, modal state'leri ve transfer state'i aynı component içinde: `OrderScreen.jsx:26-48`.
- Toplamlar frontendde kaydedilmiş sipariş + lokal sepet toplamı olarak türetiliyor: `OrderScreen.jsx:603-607`.
- Ödeme açma policy'si local expression: `OrderScreen.jsx:611-612`.

Etki:

- Kullanıcı ekranda bir toplam görürken backendde ödenebilir toplam farklı olabilir.
- Kaydet, ödeme, müşteri seçimi, transfer ve item edit gibi akışlar aynı `saving` bayrağına bağlanıyor; bu ilerde yanlış disabled/loading davranışı üretir.
- Paket ve masa siparişleri aynı component state modelinde taşındığı için paket özel kurallar ekran boyunca koşullara dağılır.

Teknik kök neden:

- `OrderScreen` bir "container + view" ayrımına sahip değil.
- Sipariş draft state'i, order query state'i ve UI modal state'i tek componentte tutuluyor.
- Domain policy fonksiyonları ekran dışına çıkarılmamış.

Önerilen çözüm:

- Önce davranış değiştirmeden `useOrderDraft`, `useOrderCustomer`, `useOrderMutations`, `getOrderActionState` gibi küçük hook/utility sınırları çıkar.

- `cartItems` ve `existingOrder` için selector fonksiyonları yaz: `displayTotal`, `canOpenPayment`, `canEditItem`, `isClosed`, `hasUnsavedChanges`.
- Daha sonra `OrderTicketPanel`, `ProductGrid`, `OrderTopBar`, `CustomerPickerModal` alt componentleri oluştur.

P1 - Masa ekranı data refresh kaynakları merkezi değil

Kanıt:

- İlk yükleme/route refresh: `TablesScreen.jsx:93-96`.
- Sidebar toggle/paket yükleme: `TablesScreen.jsx:98-104`.
- Socket event refresh: `TablesScreen.jsx:107-117`.
- Socket kopunca 30 saniye polling: `TablesScreen.jsx:120-127`.
- İşlem sonrası manuel refreshler: `TablesScreen.jsx:364-367`, `TablesScreen.jsx:406-408`, `TablesScreen.jsx:421-426`, `TablesScreen.jsx:460-464`.

Etki:

- Aynı verinin ne zaman tazeleneyeceği ekran boyunca dağılmış.
- Race condition riski var: socket event, polling ve işlem sonrası refresh aynı anda çalışabilir.
- Paket ve masa refreshleri birlikte çağrıldığı için gereksiz API trafiği oluşur.

Teknik kök neden:

- Query/cache katmanı yok.
- Masa ve paket sipariş state'i aynı ekran state'inde yönetiliyor.
- Event handling "hangi event hangi query'yi invalidate eder" yerine doğrudan `loadTables/loadTakeaway` çağrısı olarak yazılmış.

Önerilen çözüm:

- Düşük riskli ilk adım: `useTablesData({ includeTakeaway })` hook'u çıkar.
- Socket/polling invalidation mantığını hook içine taşı.
- Masa actions sonrası `refreshAll` tek fonksiyonla yürüsün; hangi endpointin yenileneceği hook içinde belirlensin.

P1 - Ödeme kuralları üç componentte tekrar ediyor

Kanıt:

- `PaymentScreen`: `round2`, `paidTotal`, `totalDue`, `isFullyPaid`: `PaymentScreen.jsx:20`, `PaymentScreen.jsx:61-67`.
- `QuickPaymentModal`: aynı hesaplar: `QuickPaymentModal.jsx:19`, `QuickPaymentModal.jsx:36-39`.
- `TablesScreen`: masa paid hesabı: `TablesScreen.jsx:284-289`.
- `TablesScreen`: paket paid/state hesabı: `TablesScreen.jsx:291-300`.

Etki:

- Bir ödeme toleransı veya paid-state değişikliği bir ekranda güncellenip diğerinde unutulabilir.
- `paid_open`, `closed`, `unpaid`, `partial` gibi ürün durumları frontendde ekran bazlı hesaplandığı için UI tutarsızlığı riski artar.

- Ödeme/masa kapatma gibi para ve operasyon kuralı taşıyan alanlarda bu tekrar kabul edilebilir değil.

Teknik kök neden:

- `paymentPolicy` veya `orderStatusSelectors` gibi merkezi saf yardımcıları yok.
- UI componentleri domain state derive ediyor.

Önerilen çözüm:

- `client/src/utls/orderPaymentState.js` gibi saf utility oluştur.
- `getPaymentSummary(order)`, `isOrderFullyPaid(order)`, `canCloseOrder(order)`, `getTakeawayPaymentState(order)` fonksiyonları tek kaynak olsun.
- Önce mevcut hesapları birebir taşı; davranış değişikliği yapma.

P2 - Route state uygulama akışında kalıcı kaynak gibi kullanılıyor

Kanıt:

- Masa siparişine geçişte table ve existingOrderId route state ile veriliyor: `App.jsx:73-75`.
- Paket siparişte customer/existingOrderId/prefillPhone/callLogId route state ile veriliyor: `App.jsx:77-88`.
- `OrderScreenWrapper` bu state'i doğrudan props'a çeviriyor: `App.jsx:260-286`.

Etki:

- Sayfa yenileme veya deep link ile açmada UI bağlamı eksilebilir.
- Masa adı/salon alanı gibi gösterim bilgileri bazen state ile gelir, bazen backend sipariştten türetilir.
- Caller ID'den açılan sipariş ile normal paket sipariş aynı route üzerinde state farkıyla ayırır; bu debugging'i zorlaştırır.

Teknik kök neden:

- URL parametreleri, query parametreleri ve server kaynaklı hydrate modeli net ayrılmamış.

Önerilen çözüm:

- `orderType`, `existingOrderId`, `callLogId` gibi kalıcı akış parametrelerini URL/query ile taşımayı değerlendir.
- `OrderScreen` route state yoksa ilgili table/order bilgisini endpointten hydrate edebilmeli.
- İlk düşük riskli adım: wrapper içinde route-state normalize eden `buildOrderScreenProps(location.state, params)` utility yaz.

P2 - Async hata yönetimi ekran bazlı ve standart değil

Kanıt:

- `OrderScreen` birçok yerde `toast.error(err.message)` ve bazı yerde fallback mesaj kullanıyor: `OrderScreen.jsx:107-123`, `OrderScreen.jsx:325-380`, `OrderScreen.jsx:547-584`.
- `TablesScreen` işlem sonrası refresh ve hata yönetimini her handler içinde tekrar ediyor: `TablesScreen.jsx:401-445`.
- `ApiService` non-JSON yanıt ve 401 davranışını global yönetiyor, fakat request cancellation veya retry yok: `api.js:15-59`.
- `PrinterPreviewPanel` kendi debounce/fetch error state'ini local tutuyor: `PrinterDetailPage.jsx:84-106`.

Etki:

- Aynı API hatası farklı ekranlarda farklı kullanıcı metniyle görünür.
- Unmount sırasında devam eden requestler için sistematik abort yok.
- Hızlı tıklama / yavaş network koşullarında stale response riski artar.

Teknik kök neden:

- Ortak mutation/query helper yok.
- Loading state'leri domain operasyonuna özel değil, genellikle tek `saving` veya `processing` bayrağı.

Önerilen çözüm:

- Düşük riskli: `getErrorMessage(err, fallback)` helper ve `withToastError` wrapper.
- Orta riskli: küçük custom hooklar (`useAsyncAction`, `useDebounceQuery`) ile loading/error standardı.
- Uzun vadeli: React Query veya benzeri query/cache invalidation katmanı.

4. Component mimarisi sorunları

P1 - `OrderScreen.jsx` tek sorumluluğu açıkça aşıyor

Kanıt:

- 1509 satır.
- API orchestration: `OrderScreen.jsx:107-123`, `OrderScreen.jsx:325-373`, `OrderScreen.jsx:547-599`.
- Business rules: `OrderScreen.jsx:211-223`, `OrderScreen.jsx:611-612`, `OrderScreen.jsx:968`, `OrderScreen.jsx:1041-1046`.
- UI rendering: `OrderScreen.jsx:642-1245` ve modal renderları `OrderScreen.jsx:1255-1415`.
- Aynı dosyada nested `ModifierModal` ve `ClipboardEmpty` componentleri var: `OrderScreen.jsx:1505-1575`.

Etki:

- Sipariş ekranında yapılacak küçük UI değişikliği bile ödeme gating veya item lifecycle davranışına temas edebilir.
- Test yazmak pahalıdır; saf fonksiyon sınırları azdır.
- Yeni geliştirici için dosyayı anlamak operasyon akışını ezberlemeye bağlıdır.

Önerilen ayrıştırma sırası:

1. Saf selector/policy fonksiyonları.
2. API mutation hookları.
3. UI alt componentleri.
4. Modal componentlerini bağımsızlaştırma.

P1 - `TablesScreen.jsx` iki ayrı operasyon ekranını tek componentte taşıyor

Kanıt:

- Masa grid state'i ve paket sipariş queue state'i aynı componentte: `TablesScreen.jsx:39-55`.
- Masa transfer/ödeme/kapatma/yazdırma action handlerları: `TablesScreen.jsx:207-399`.
- Paket teslimat/iptal/yazdırma handlerları: `TablesScreen.jsx:401-468`.

- Masa kartları ve paket sipariş yan paneli aynı render gövdesinde: `TablesScreen.jsx:599-1115`.
- Confirm/action modal renderları aynı dosyanın sonunda: `TablesScreen.jsx:1159-1425`.

Etki:

- Salon ve paket operasyonunun değişiklik ritimleri farklıdır; tek dosyada değıştikçe regresyon alanı büyür.
- Masa event refreshleri paket refreshleriyle gereksiz birleşir.
- Action sheet, confirm modal ve paket menu gibi reusable parçalar kopyalanmaya aday kalır.

Önerilen ayrıştırma:

- `useTablesData`, `useTakeawayQueue`, `useTableActions`.
- `TableCard`, `TableActionSheet`, `TakeawaySidebar`, `TakeawayOrderCard`, `ConfirmDialog`.
- Paket sipariş için V1'de yan panel kalabilir; component sınırı yine ayrılmalı.

P1 - `PrinterDetailPage.jsx` form state ve entegrasyon state'ini tek yerde biriktiriyor

Kanıt:

- Form state listesi: `PrinterDetailPage.jsx:156-181`.
- Dirty snapshot hesabı: `PrinterDetailPage.jsx:185-202`.
- Printer settings/discovery load: `PrinterDetailPage.jsx:245-332`.
- Save payload normalizasyonu: `PrinterDetailPage.jsx:390-459`.
- Encoding/advanced alanları: `PrinterDetailPage.jsx:676-741`.
- Layout/output/copy/role option alanları: `PrinterDetailPage.jsx:883-979`.

Etki:

- Yazıcı gibi sahada en kırılgan entegrasyonlardan biri UI dosyası içinde fazla karmaşık.
- Encoding, physical printer binding ve print layout optionları aynı nested `printOptions` state'inde dağılıyor.
- Yeni printer türü veya profil eklendiğinde dosya daha da büyür.

Önerilen ayrıştırma:

- `usePrinterFormModel`.
- `PrinterConnectionSection`.
- `PrinterAdvancedEncodingSection`.
- `PrinterLayoutOptionsSection`.
- `PrinterPreviewPanel` ayrı dosyaya taşınmalı.

P2 - API client domain sınırı taşımıyor

Kanıt:

- Auth, tables, products, orders, payments, customers, caller ID, reports, reservations, stock, waiter call, print ve admin endpointleri tek class içinde: `api.js:73-227`.
- Token state aynı class içinde ve `localStorage` ile yönetiliyor: `api.js:5-12`.
- 401 durumunda doğrudan `window.location.reload()` yapılıyor: `api.js:31-34`.

Etki:

- Domain bazlı test/mockng zor.

- UI ekranları tüm API yüzeyine erişebilir; yanlış domain çağrısı için sınır yok.
- Auth failure davranışı UI routing yerine sayfa reload ile çözülüyor; Electron içinde kaba bir recovery modeli.

Önerilen çözüm:

- Davranış korumalı olarak `ordersApi`, `tablesApi`, `paymentsApi`, `adminPrintersApi` gibi modüllere ayır.
- Mevcut default `api` facade bir süre korunabilir.
- Auth reset için callback/event mekanizması eklenebilir; reload son seçenek olmalı.

P2 - Settings shell yok, ayarlar ekranları ortak davranış paylaşıyor

Kanıt:

- `SettingsLayout` yalnız `Outlet` döndürüyor: `SettingsLayout.jsx:1-4`.
- Printer detail kendi dirty guard'ını local `window.confirm` ile yapıyor: `PrinterDetailPage.jsx:386`.
- Menu settings kendi delete/bulk confirm'lerini browser confirm ile yapıyor: `MenuSettingsPage.jsx:372`, `MenuSettingsPage.jsx:391`, `MenuSettingsPage.jsx:505`.

Etki:

- Ayarlar ekranları farklı kaydet/çık/confirm davranışları geliştirir.
- Profesyonel admin deneyimi yerine birbirinden bağımsız formlar hissi oluşur.

Önerilen çözüm:

- `SettingsPageShell`, `SettingsSection`, `UnsavedChangesGuard`, `ConfirmDialog` gibi ortak yapılar çıkar.
- İlk turda sadece ortak confirm bileşeni ile başlayabilir.

5. Amatör görünen frontend alanları

P1 - Business rule'ların UI içine gömülmesi

Örnekler:

- Siparişte ödeme açma koşulu: `OrderScreen.jsx:611-612`.
- Existing item edit kuralı: `OrderScreen.jsx:968`.
- Masa paid kuralı: `TablesScreen.jsx:284-289`.
- Paket ödeme state'i: `TablesScreen.jsx:291-300`.
- Ödeme kapatma kuralı: `PaymentScreen.jsx:143-146`.

Neden amatör görünüyor:

Bu kurallar domain policy olarak tek kaynakta olmalı. UI içinde kalınca her ekran kendi doğruluğunu üretir; ürün davranışı "component hangi koşulu yazdıysa" ona dönüşür.

P1 - Monolitik ekran bileşenleri

Örnekler:

- `OrderScreen.jsx`: 1509 satır.
- `TablesScreen.jsx`: 1444 satır.
- `PrinterDetailPage.jsx`: 993 satır.

Neden amatör görünüyor:

Ekran dosyaları hem state makinesi hem API orchestration hem UI template hem modal registry gibi çalışıyor. Bu yapı ürün olgunlaştıkça "her şeyi bilen component" anti-pattern'ine döner.

P1 - Tekrar eden ödeme hesapları

Örnekler:

- `round2`, `paidTotal`, `totalDue`, `isFullyPaid` tekrarları: `PaymentScreen.jsx:20`, `PaymentScreen.jsx:61-67`, `QuickPaymentModal.jsx:19`, `QuickPaymentModal.jsx:36-39`.
- Masa paid hesabı ayrı: `TablesScreen.jsx:284-289`.

Neden amatör görünüyor:

Ödeme domaininde tolerans ve tamamlanma kuralı UI componentlerinde tekrar edilmemeli. Bu para/kasa davranışdır.

P2 - Browser confirm kullanımı hala ayarlar/menü tarafında duruyor

Örnekler:

- Printer çıkış guard'ı: `PrinterDetailPage.jsx:386`.
- Menü kategori/ürün/bulk confirm: `MenuSettingsPage.jsx:372`, `MenuSettingsPage.jsx:391`, `MenuSettingsPage.jsx:505`.

Neden amatör görünüyor:

Önceki turda masa/sipariş confirmleri iyileştirilmiş olsa da ayarlar tarafında native confirm kalmış. Aynı ürün içinde bazı kritik işlemler custom modal, bazıları browser dialog ile ilerliyor.

P2 - Inline style yoğunluğu component sistemini zayıflatıyor

Kanıt:

- Masa kartı, action sheet, paket kartı stilleri büyük oranda inline: `TablesScreen.jsx:599-1115`.
- Sipariş ekranı ürün grid/adisyon/topbar stilleri inline: `OrderScreen.jsx:642-1245`.
- Payment ekranı kart ve aksiyon layoutları inline: `PaymentScreen.jsx:190-491`.
- Split payment modal kendi `<style>` bloğunu taşıyor: `SplitPaymentModal.jsx:487-707`.

Neden amatör görünüyor:

Tasarım sistemi tokenları var, ama uygulama yüzeyi ad hoc inline stillerle büyüyor. Bu tekrarı artırır ve UI kalite standardını merkezi yönetmeyi zorlaştırır.

P2 - Çok fazla ekran kendi mini data layer'ını yazıyor

Örnekler:

- Customers import/export/load/stats state'i tek componentte: `CustomersScreen.jsx:8-26`, `CustomersScreen.jsx:29-235`.
- Reports günlük rapor, closed order pagination, export ve analytics state'i tek componentte: `ReportsScreen.jsx:247-400`.
- Menu settings kategori/ürün/bulk/sıralama state'i tek componentte: `MenuSettingsPage.jsx:246-263`.

Neden amatör görünüyor:

Ekranlar domain hook'ları yerine local state ve API handler yığınıyla büyüyor. Bu pattern yeni ekranlarda kopyalanır.

6. Teknik borç ve kırılabilirlik alanları

P1 - Merkezi frontend domain policy yok

Alanlar:

- Sipariş aksiyon görünürlüğü.
- Masa aksiyon görünürlüğü.
- Ödeme tamamlanma/tam kapatma toleransı.
- Paket teslimat/ödeme ilişkisi.
- Item edit/void yetkisi.

Sonuç:

Backend guard'ları olsa bile frontend davranışı ekran bazında değişir. POS gibi hızlı kullanımda kullanıcı farklı ekranlarda farklı kural görür.

P1 - Ekran state'i ile backend state'i arasında cache/invalidasyon modeli yok

Alanlar:

- Tables socket/polling/route refresh karışımı.
- Order save sonrası bazen full order reload, bazen result set etme: `OrderScreen.jsx:363-370`.
- Payment completion sonrası navigation ile refresh tetikleme: `App.jsx:117-124`.
- Split payment sonrası state reload: `SplitPaymentModal.jsx:221-225`.

Sonuç:

Stale UI riski düşük/orta düzeyde var. Özellikle hızlı ödeme, masa kapatma, socket event ve polling aynı anda çalıştığında görülebilir.

P2 - Prop drilling ve callback orchestration büyümeye açık

Kanıt:

- `TablesScreen` props ile `onOpenOrder`, `onPayment`, `onQuickPayment`, `onOpenTakeawayOrder` alıyor: `TablesScreen.jsx:33-37`.
- `OrderScreenWrapper` route state'i props'a çevirip `OrderScreen` e geçiriyor: `App.jsx:260-286`.
- `OrderScreen` ödeme ve navigation callbacklerini props ile alıyor: `OrderScreen.jsx:13-24`.

Sonuç:

Şimdilik yönetilebilir. Ancak yeni ödeme aksiyonları, caller ID, paket queue ve masa transfer dönüşleri arttıkça callback zinciri belirsizleşir.

P2 - Frontend test/lint/typecheck kapısı yok

Kanıt:

- `client/package.json:6-9` yalnız `dev`, `build`, `preview` scriptlerini içeriyor.

Sonuç:

Frontend mimari refactorları için güvenlik ağı yok. Bu yüzden her parçalama önce saf utility extraction ve build doğrulamasıyla küçük adımlara bölünmeli.

P2 - Settings formları nested state'i doğrudan güncelliyor

Kanıt:

- `printOptions` birçok yerde local callback ile güncelleniyor: `PrinterDetailPage.jsx:339-379`, `PrinterDetailPage.jsx:511-512`, `PrinterDetailPage.jsx:883-979`.
- Dirty snapshot JSON stringify ile kontrol ediliyor: `PrinterDetailPage.jsx:185-202`.

Sonuç:

Nested state shape değişirse form dirty logic, save payload ve preview aynı anda etkilenir. Yazıcı ayarları gibi cihaz bağımlı alanda bu risk önemlidir.

7. Hemen ele alınması gerekenler

P1 - OrderScreen için domain selector/policy extraction

Kapsam:

- `hasUnsavedChanges`, `canOpenPayment`, `displayTotal`, `canEditQty`, `isLineReadOnly`.
- Davranış değiştirmeden saf fonksiyonlara çıkarılmalı.

Neden hemen:

- En büyük frontend dosyası.
- Sipariş ve ödeme geçişi en kritik POS akışı.
- Düşük riskli ilk refactor ile test edilebilir sınır oluşur.

P1 - Ödeme durum hesaplarını tek utility'ye toplama

Kapsam:

- `round2`
- `getPaidTotal(order)`
- `getOrderTotal(order)`
- `getTotalDue(order)`
- `isFullyPaid(order)`
- `canCloseOrder(order)`
- `getTakeawayPaymentState(order)`

Neden hemen:

- Aynı kural `PaymentScreen`, `QuickPaymentModal`, `TablesScreen` içinde tekrar ediyor.
- Para/kasa davranışı merkezi olmalı.

P1 - TablesScreen data/action hook ayrımı

Kapsam:

- `useTablesData`
- `useTakeawayQueue`
- `useTableActions`
- `TableActionSheet` ve `TakeawaySidebar` component extraction.

Neden hemen:

- Masa ve paket operasyonu tek dosyada aşırı büyümüş.
- Refresh kaynakları dağılmış.

P2 - Ortak ConfirmDialog bileşeni

Kapsam:

- Masa/sipariş tarafında yeni custom confirmler var.
- Ayarlar/menü tarafında hala `window.confirm` var.

Neden:

- Düşük riskli.
- UI tutarlılığını artırır.
- Sonraki refactorlar için modal standardı oluşturur.

P2 - Settings shell ve printer form model ayrımı

Kapsam:

- `SettingsLayout` gerçek shell'e dönüşmeli.
- `PrinterDetailPage` form model/hook ve section componentlerine bölünmeli.

Neden:

- Yazıcı ayarı sahada kritik.
- Şu an dosya hem cihaz integration hem UI hem form normalize işlemlerini taşıyor.

8. Sonraki turda uygulanabilecek düşük riskli refactor önerileri

Bu turda uygulanmadı; aşağıdaki liste davranış değiştirmeden küçük PR/tur olarak ele alınabilir.

8.1 `orderPaymentState` utility

Önerilen dosya:

- `client/src/utls/orderPaymentState.js`

İçerik:

- `roundMoney(value)`
- `getPaidTotal(order)`
- `getOrderTotal(order)`

- `getTotalDue(order)`
- `isOrderFullyPaid(order)`
- `canCloseOrder(order)`
- `getPaymentStateLabel(order)`

Etkilenecek ekranlar:

- `PaymentScreen.jsx`
- `QuickPaymentModal.jsx`
- `TablesScreen.jsx`
- ileride `SplitPaymentModal.jsx`

Risk:

- Düşük/orta. İlk adımda birebir mevcut hesaplar taşınmalı, davranış değişmemeli.

8.2 `orderActionPolicy` utility

Önerilen dosya:

- `client/src/utls/orderActionPolicy.js`

İçerik:

- `canOpenPayment({ existingOrder, cartItems })`
- `canEditOrderItem(item, order)`
- `canVoidOrderItem(item, order, user)`
- `canSaveDraft({ cartItems, orderType, selectedCustomer })`

Etkilenecek ekranlar:

- `OrderScreen.jsx`
- ileride masa action sheet ve ödeme yönlendirmeleri.

Risk:

- Düşük. Önce unit test yazılmalı.

8.3 `OrderScreen` için küçük hook'lar

Tek büyük hook önerilmez. Aşamalı:

- `useOrderCatalog`: kategori, ürün, arama.
- `useOrderDraftCart`: lokal sepet işlemleri.
- `useOrderMutations`: save, item update, void.
- `useOrderCustomer`: müşteri seçimi/paket adresi.

Risk:

- Orta. UI ile state sıkı bağlı olduğu için tek turda değil, küçük parçalarda yapılmalı.

8.4 `TablesScreen` parçalama

Önerilen componentler:

- `TableCard`
- `TableActionSheet`
- `TakeawaySidebar`
- `TakeawayOrderCard`
- `TableConfirmDialog`

Önerilen hooklar:

- `useTablesData`
- `useTakeawayOrders`
- `useTableTransfer`

Risk:

- Orta. Render çıktısı çok inline style içerdiği için önce component extraction, sonra CSS cleanup yapılmalı.

8.5 Ortak async action helper

Önerilen hook:

- `useAsyncAction({ onError })`

Amaç:

- `saving`, `processing`, `loading` bayraklarını operasyon bazlı standardize etmek.
- Hata mesajlarını tek helper ile normalize etmek.

Risk:

- Düşük. Önce yeni kodda kullanılmalı, sonra eski handlerlar taşınmalı.

8.6 Ortak `ConfirmDialog`

Önerilen component:

- `client/src/components/common/ConfirmDialog.jsx`

Kullanım hedefleri:

- `OrderScreen` `confirmDialog`.
- `TablesScreen` `tableConfirm/tableCancel/takeawayCancel`.
- `PrinterDetailPage` `unsaved changes`.
- `MenuSettingsPage` `delete/bulk confirm`.

Risk:

- Düşük. Önce sadece yeni modal API'si oluşturulup bir ekranda uygulanmalı.

8.7 Settings shell

Önerilen componentler:

- `SettingsPageShell`
- `SettingsToolbar`
- `SettingsUnsavedChangesGuard`

Risk:

- Düşük/orta. Görsel davranış etkileyebilir, bu yüzden önce yalnız layout wrapper olarak başlanmalı.

Önceliklendirilmiş frontend refactor sırası

Hemen

1. P1 - Ödeme state selector utility'si.
2. P1 - Sipariş aksiyon policy utility'si.
3. P1 - `OrderScreen` içinden saf hesap/koşul fonksiyonlarının çıkarılması.
4. P2 - Ortak `ConfirmDialog` standardı.

Bu hafta

1. P1 - `TablesScreen` data hook ve `TakeawaySidebar` extraction.
2. P1 - `OrderScreen` katalog/cart/customer hooklarına ayırıştırma başlangıcı.
3. P2 - `PrinterDetailPage` form modelini hook'a alma.
4. P2 - API client domain modüllerine davranış korumalı facade ile bölme.

Sonra

1. React Query veya hafif query/cache invalidation katmanı.
2. Route state bağımlılığını URL/server hydrate modeline taşıma.
3. Split payment state machine'i ayrı reducer/hook yapısına taşıma.
4. Frontend test altyapısı: utility unit testleri, sonra Playwright smoke.
5. Inline style borcunu component CSS/tasarım sistemi katmanına taşıma.

Son karar

Frontend çalışır durumda ve route lazy loading gibi doğru yönde adımlar var. Ancak profesyonel ürün kalitesi açısından frontend mimarisi halen "ekran dosyaları içinde büyüyen local app" seviyesinde. En büyük risk, business rule'ların ekran componentlerine gömülmesi ve ödeme/sipariş/masa state'inin merkezi policy olmadan tekrar hesaplanması.

Bu noktada büyük bir mimari dönüşüm yapmak doğru değil. Doğru hareket, önce davranışı değiştirmeyen saf utility ve hook extraction ile test edilebilir sınırlar oluşturmak; sonra büyük ekranları parça parça küçültmek.

04 - Backend / API Denetimi

Denetim tarihi: 2026-04-14 Kapsam: order lifecycle, table lifecycle, payment rules, kitchen/print trigger mantigi, masa tasima, API guard'lari, status transition'lar, duplicate action riskleri, transaction ve veri tutarililigi.

Bu rapor yalnızca analizdir. Uygulama kodunda davranissal degisiklik yapilmamistir.

1. Backend mimarisi ozeti

Backend Express tabanlı, SQLite/better-sqlite3 üzerinden senkron transaction modeli kullanan bir POS API katmanı olarak çalışıyor. Ana domain yüzeyi şu dosyalarda toplanmış durumda:

- `server/index.js` : API route montajları. Tables `server/index.js:113`, orders `server/index.js:116`, payments `server/index.js:117`, admin `server/index.js:123`, bridge `server/index.js:124`.
- `server/routes/orders.js` : sipariş oluşturma, ürün ekleme, paket teslim, mutfak status geçişleri, satır güncelleme, kapatma iptal.
- `server/routes/payments.js` : tam ödeme, parçalı/split ödeme, ödeme özeti, ödeme ile sipariş/ masa kapatma.
- `server/routes/tables.js` : masa listesi, masa status güncelleme, masa transferi.
- `server/services/printJobs.js` : mutfak, fiş, paket etiketi ve yazıcı job idempotency mantığı.
- `server/routes/bridge.js` : StoreBridge/yazıcı istemcisinin job listeleme, claim etme ve sonuç bildirme API'leri.
- `server/migrations/run.js` : temel tablo semaları, enum/check constraint'leri ve migration yamaları.

Guçlu taraflar:

- Kimlik doğrulama ve business scope middleware'i merkezi kurulmuş; route'lar `req.businessId` ile işletme izolasyonu uyguluyor.
- Sipariş oluşturma, ürün ekleme, masa transferi ve ödeme alma gibi kritik işlemlerin önemli bölümlerinde SQLite transaction kullanılıyor.
- Yazıcı job tarafında `idempotency_key` UNIQUE constraint'i var (`server/migrations/run.js:297-299`) ve job insert'i `INSERT OR IGNORE` ile yapıyor (`server/services/printJobs.js:63-79`).
- Bridge claim işlemi atomik: sadece `pending` ve `claimed_at IS NULL` olan job claim ediliyor (`server/routes/bridge.js:329-342`).
- Son değişikliklerden sonra paket teslim akisi backend tarafında otomatik ödeme kaydı ile kapanacak şekilde desteklenmiş (`server/routes/orders.js:214-225`).

Ana zayıflık: backend, domain kurallarını tek bir servis/domain katmanında toplamıyor. Kurallar route içinde dağınık: status geçişleri, ödeme guard'ları, masa status kuralları, mutfak/yazıcı side effect'leri aynı dosyalarda karışık yürütülüyor. Bu, bugünkü ölçeğe çalışır; fakat restoran POS gibi hızlı ve tekrara açık ortamda duplicate action, eksik transaction ve UI'a güvenme riskini büyütür.

2. Kritik domain akislar

2.1 Masa siparisi oluşturma

- `POST /api/orders` create schema ile validate edilir (`server/routes/orders.js:62-75`).

2. Request icinde en az bir urun zorunludur (`server/routes/orders.js:367-369`).
3. Transaction icinde sira numarasi alinir, urun fiyatları çözülür, order `saved` status ile eklenir (`server/routes/orders.js:374-421`).
4. Order item'lar `new` status ile eklenir (`server/routes/orders.js:423-433`).
5. `table_id` varsa masa `occupied` yapılır ve `current_order_id` atanır (`server/routes/orders.js:436-438`).
6. Transaction bittikten sonra paket etiketi, Caller ID link'i ve mutfak finalize işlemleri çalışır (`server/routes/orders.js:442-452`).
7. `finalizeKitchenForNewItems` yeni satırları `sent` , siparisi `in_kitchen` yapar ve mutfak job tetikler (`server/routes/orders.js:87-95`).

Risk: DB kaydı transaction icinde, mutfak/yazıcı/caller side effect'leri transaction disinda. Bu ayrim, siparis kaydedildikten sonra mutfak job/status finalize basarisiz olursa siparisin kullaniciya kaydedilmis gorunup mutfak tarafina eksik dusmesine yol acabilir.

2.2 Kayitli siparise yeni urun ekleme

1. `POST /api/orders/:id/items` addItems schema ile en az bir urun ister (`server/routes/orders.js:77-80` , `server/routes/orders.js:470`).
2. Kapali veya iptal edilmis siparise urun ekleme engellenir (`server/routes/orders.js:479-480`).
3. Transaction icinde urunler eklenir ve toplamlar guncellenir.
4. Transaction sonrasi yeni item'lar mutfaga finalize edilir (`server/routes/orders.js:521`).

Risk: Olusturma akisina benzer sekilde yeni satirlar DB'ye yazildiktan sonra mutfak status/job islemi ayrik calisir. Bu, "urun eklendi ama mutfaga gitmedi" veya "satir new kaldi" gibi POS icin yuksek maliyetli operasyonel tutarsizliklara acik.

2.3 Siparis kapatma

1. `PATCH /api/orders/:id/status` status body alır (`server/routes/orders.js:536-539`).
2. `cancelled` icin yetki, mevcut status ve odeme varligi kontrol edilir (`server/routes/orders.js:542-553`).
3. `closed` icin artik backend odeme tamamlanmadan kapatmayi engelliyor (`server/routes/orders.js:584-586`).
4. `closed` oldugunda masa bosaltilir ve muster i istatistigi guncellenir (`server/routes/orders.js:592-607`).
5. Dine-in sipariste fis job'u transaction sonrasi enqueue edilir (`server/routes/orders.js:617-619`).

Guc: Odeme tamamlanmadan `closed` status engeli backend tarafinda var. Bu, onceki urun/is kurali raporlarinda P1 olarak isaretlenen en kritik davranislardan birinin dogru yere alindigini gosteriyor.

Risk: `status` request body icin zod enum validation yok. Gecersiz status, DB CHECK constraint'e carpip 500'e donebilir. Profesyonel API'de bu 400 ve domain error code olmalidir.

2.4 Paket siparis teslim akisi

1. `PATCH /api/orders/:id/takeaway/delivery` yalnızca `out_for_delivery` ve `delivered` aksiyonlarını kabul eder (`server/routes/orders.js:176-180`).
2. Sadece `takeaway` order type guncellenebilir (`server/routes/orders.js:187-188`).
3. `out_for_delivery` birden fazla cagrilirsa idempotent olarak `already_out_for_delivery` doner (`server/routes/orders.js:199-205`).

4. `delivered` için önce `takeaway_out_at` zorunludur (`server/routes/orders.js:208-209`).
5. Delivery transaction'i içinde kalan tutar kadar otomatik ödeme kaydedilir ve order `closed` yapılır (`server/routes/orders.js:214-221`).

Guc: Kullanıcının son talebiyle uyumlu olarak, paket siparis teslim edilince ödeme zorunluluğu kaldırılmış ve backend kalan tutarı otomatik odendi olarak isliyor.

Risk: Endpoint önce `closed/cancelled` kontrolü yapıyor (`server/routes/orders.js:190-192`), sonra `takeaway_delivered_at` idempotency kontrolüne geliyor (`server/routes/orders.js:193-197`). Başarılı teslimden sonra aynı `delivered` aksiyonu tekrar gelirse order status `closed` olduğu için idempotent cevap yerine 400 donebilir. Veri bozulmaz; fakat POS'ta çift tıklama/ag kopması sonrası kullanıcıya hatalı hata mesajı üretir.

2.5 Odeme alma

1. `POST /api/payments` sadece `cash` ve `card` kabul eder (`server/routes/payments.js:13-23`).
2. Kapalı/iptal siparise ödeme eklenemez (`server/routes/payments.js:203-207`).
3. Transaction içinde payment kaydı eklenir (`server/routes/payments.js:214-218`).
4. `close_order` true ise ödeme yeterliyse order ve masa kapatılır (`server/routes/payments.js:220-222`).
5. Receipt job transaction sonrası tetiklenir.

Risk: Normal odemede kalan bakiye asimi kontrolü yok. Split payment tarafında bu guard var (`server/routes/payments.js:315-329`), fakat tam ödeme endpoint'i `amount > remaining` durumunu engellemiyor. Bu, grand_total üzerinde paid_total oluşmasına ve raporlamada sahte ciro görünmesine yol açabilir.

2.6 Parcalı / split ödeme

1. `POST /api/payments/:orderId/split` item allocation üzerinden miktar hesaplar.
2. Daha önce unallocated full-order payment varsa split'i engeller (`server/routes/payments.js:159` , `server/routes/payments.js:263-264` civarı).
3. Kalan tutar asimi engellenir (`server/routes/payments.js:315-329`).
4. Test davranışına göre tüm split ödemeler tamamlanınca order otomatik kapanmıyor; masa/siparis explicit kapanış bekliyor.

Yorum: Bu davranış ürün kararı olabilir. Ancak backend'de "paid but open" gibi açık bir status yok. Bu durum UI'da ve raporlarda net ayrılmazsa personel "odendi mi, kapanacak mı" belirsizliği yasar.

2.7 Masa tasima

1. `POST /api/tables/:id/transfer` target zorunlu, kaynak/hedef aynı olamaz (`server/routes/tables.js:86-94`).
2. Kaynak ve hedef masa aynı business içinde aranır (`server/routes/tables.js:95-98`).
3. Hedef masa sadece `empty` ise kabul edilir; reserved hedefe transfer engellenir (`server/routes/tables.js:99-100`).
4. Transaction içinde order `table_id` hedefe tasınır, hedef masa kaynak state'ini alır, kaynak masa boşaltılır (`server/routes/tables.js:103-110`).

Guc: Rezerve masaya transfer/acma kuralı backend'de güçlü bir guard ile korunuyor.

Risk: Generic `PATCH /api/tables/:id/status` aynı domain güvenliğini tasimiyor. Herhangi bir masa status'u, current order invariant'lari kontrol edilmeden degistirilebilir (`server/routes/tables.js:59-73`).

2.8 Yazici / mutfak akisi

1. Mutfak, fis ve paket etiketi job'lari `print_jobs` tablosuna yaziliyor.
2. Job status enumlari: `pending`, `printed`, `failed`, `cancelled` (`server/migrations/run.js:290-299`).
3. Idempotency key unique ve insert ignore ile duplicate job azaltilmis (`server/services/printJobs.js:63-79`).
4. Bridge pending joblari listeler ve atomik claim eder (`server/routes/bridge.js:300-342`).
5. Bridge job sonucunu `printed` veya `failed` olarak bildirir (`server/routes/bridge.js:357-376`).

Risk: `processPendingJobsSync` mock modu kapali degilse pending joblari server tarafinda otomatik `printed` yapabiliyor (`server/services/printJobs.js:592-610`). Production konfigrasyonunda bu kritik bir operasyonel toggle. Yanlis config ile gercek yaziciya gitmeyen job "printed" gorunebilir.

3. Status transition matrisi

3.1 Order

Status	Olusturan akis	Izin verilen temel sonraki durumlar	Backend guard durumu
<code>new</code>	DB default (<code>server/migrations/run.js:198</code>)	Pratikte nadir; order create <code>saved</code> yazar	Domain'de aktif kullanimi belirsiz
<code>saved</code>	Order create (<code>server/routes/orders.js:413-421</code>)	<code>in_kitchen</code> , <code>cancelled</code> , <code>closed</code>	Create sonrasi otomatik finalize genelde <code>in_kitchen</code> yapar
<code>in_kitchen</code>	<code>finalizeKitchenForNewItems</code> veya status patch (<code>server/routes/orders.js:87-95</code> , <code>server/routes/orders.js:575-615</code>)	<code>preparing</code> , <code>ready</code> , <code>served</code> , <code>cancelled</code> , <code>closed</code>	Status patch enum validation eksik
<code>preparing</code>	Status veya item tarafından operasyonel ilerleme	<code>ready</code> , <code>served</code> , <code>closed</code> , <code>cancelled</code>	Gecis kurallari merkezi degil
<code>ready</code>	Mutfak/satir ilerleme	<code>served</code> , <code>closed</code>	Merkezi transition graph yok
<code>served</code>	Servis edildi bilgisi	<code>closed</code>	Closed icin odeme guard'i var
<code>cancelled</code>	Status patch veya auto-cancel	Terminal	Odeme varsa iptal engeli var
<code>closed</code>	Odeme ile kapatma, status patch, paket delivered	Terminal	Odeme tamamlanmadan closed engeli var; paket delivered otomatik odeme yaratir

3.2 Order item

Status	Olusturan akis	Beklenen sonraki durum	Risk
new	Item insert (server/migrations/run.js:226)	sent , cancelled , porsiyon degisimi	Porsiyon degisimi sadece new iken engelleniyor/destekleniyor
sent	Mutfak finalize (server/routes/orders.js:90-92)	preparing , ready , served , cancelled	Satir status'u request body'den enum validation olmadan gelebiliyor
preparing	Item patch (server/routes/orders.js:657-662)	ready , cancelled	Gecis sirasi kontrolu yok
ready	Item patch/kitchen	served , cancelled	Merkezi kural yok
served	Item patch	Terminal veya cancelled ?	Terminal mantigi net degil
cancelled	Item patch/order cancel	Terminal	Cancel sonrasi tekrar status degisimi guard'i zayif
comped	DB enum (server/migrations/run.js:226)	Terminal/raporlama	Kod daha cok is_comped flag'i kullaniyor; enum ile flag cift anlam tasiyor

3.3 Table

Status	Olusturan akis	Anlam	Risk
empty	Default, order close/cancel, transfer source (server/migrations/run.js:80 , server/routes/orders.js:598-602 , server/routes/tables.js:109-110)	Aktif order yok	Generic status patch ile current_order_id temizlenmeden set edilebilir
occupied	Order create, transfer target (server/routes/orders.js:436-438 , server/routes/tables.js:107-108)	Aktif order var	Generic patch ile order olmadan occupied olabilir
reserved	Table status patch/admin akisi	Rezerve masa	Transfer target olarak engelleniyor (server/routes/tables.js:99-100)

3.4 Payment

Payment icin status kolonu yok; kayıt tipi ve scope ile durum turetiliyor.

Alan	Degerler	Kaynak
payment_type	DB: cash, card, mixed, other	server/migrations/run.js:242
payment_type	API create: sadece cash, card	server/routes/payments.js:13-18
payment_scope	full_order, split_item	server/migrations/run.js:243

Not: Paket teslim otomatik odemesi backend icinde `other` olarak kaydediliyor. Bu dogru bir teknik isaret olabilir; fakat rapor/finans UI tarafinda "otomatik teslim odemesi" olarak ayrismali.

3.5 Kitchen / print job

Status	Olusturan akis	Anlam	Risk
pending	Job enqueue	Yazdirilacak is	Bridge veya mock tarafından islenir
printed	Bridge PATCH veya mock	Basarili yazdirildi	Mock production'da acik kalirsa sahte basari riski
failed	Bridge PATCH veya mock catch	Yazdirma basarisiz	Retry/resolve workflow'u sinirli gorunuyor
cancelled	DB enum	lptal is	Aktif endpoint/akis net degil

4. API guard ve is kurali eksikleri

P1 - API schema ile DB order_type constraint'i uyumsuz

- Kanit: API `order_type` icin `delivery` kabul ediyor (`server/routes/orders.js:64`), fakat DB sadece `dine_in` ve `takeaway` kabul ediyor (`server/migrations/run.js:194`).
- Etki: Client `delivery` gonderirse validation gecer, insert DB CHECK constraint'e takilir ve profesyonel 400 yerine 500/teknik hata uretme riski olusur.
- Kok neden: API contract ile migration enum'u ayni kaynaktan uretilmiyor.
- Oneri: V1'de `delivery` henuz gercek domain tipi degilse API enumundan kaldir. Eger delivery ayri tip olacaksa migration, UI, raporlar, paket teslim endpoint'i ve testler birlikte guncellenmeli.

P1 - Normal odeme endpoint'i kalan bakiye asimini engellemiyor

- Kanit: `POST /api/payments` amount'u positive olarak validate ediyor (`server/routes/payments.js:19`) ve direkt insert ediyor (`server/routes/payments.js:214-218`). Split payment'ta kalan tutar asimi guard'i var (`server/routes/payments.js:315-329`), normal odemede yok.
- Etki: Ayni siparise fazla odeme yazilabilir. Gun sonu raporu, masa odenmislik durumu ve finansal toplamalar `grand_total`'dan yuksek gorunebilir.
- Kok neden: Full payment ile split payment ayni payment policy helper'ini kullanmiyor.
- Oneri: `remaining_total` merkezi hesaplanmeli; tip/bahsis modeli yoksa `amount <= remaining_total + tolerans` guard'i full payment icin de uygulanmeli. Bahsis desteklenecekse ayrica tipalani ve raporlama ayrimi eklenmeli.

P1 - Odeme endpoint'lerinde idempotency yok

- Kanit: `POST /api/payments` her cagrida yeni `paymentId` uretip insert ediyor (`server/routes/payments.js:209-218`). Request idempotency key veya unique client operation id yok.
- Etki: Kasiyer cift tiklar, ag timeout sonrasi tekrar dener veya POS tablet istegi yinelerse ayni odeme iki kez kaydedilebilir. Bu restoran operasyonunda dogrudan kasa/fis/rapor problemi yaratir.
- Kok neden: Payment write API'si "at least once" client davranisina karsi korunmuyor.
- Oneri: `Idempotency-Key` veya body'de `client_operation_id` kabul edilmeli; `payments` tablosunda `business/order/key` unique constraint ile tekrar cagrida mevcut payment donmeli.

P1 - Order item update backend'i UI'a fazla guveniyor

- Kanit: Item patch body'den `status`, `quantity`, `is_comped`, `comp_reason` aliniyor (`server/routes/orders.js:632-634`), `status` ve `quantity` dogrudan SQL update'e giriyor (`server/routes/orders.js:655-690`). Quantity icin positive/int guard'i, status icin enum/transition validation yok.
- Etki: Negatif veya sifir quantity, gecersiz status, terminal satirin tekrar degismesi, comp flag/status uyumsuzlugu gibi veri bozan durumlar API seviyesinde mumkun hale gelir.
- Kok neden: UI tarafindaki kontroller domain guard yerine gecmis.
- Oneri: `patchOrderItemSchema` eklenmeli; quantity integer/positive/max olmalı, status enum ve izinli gecis tablosu ile kontrol edilmeli, `is_comped` ile `status='comped'` kararından biri tek kaynak yapılmalı.

P1 - Siparis create/add-items akisinda side effect'ler transaction disinda

- Kanit: Order transaction `txn()` ile biter (`server/routes/orders.js:442`), sonra paket etiketi, Caller ID link'i ve mutfak finalize calisir (`server/routes/orders.js:444-452`). Add-items da transaction sonrasi `finalizeKitchenForNewItems` cagirir (`server/routes/orders.js:521`).
- Etki: Siparis DB'de kayitli kalip mutfak status'u/job'u eksik kalabilir. Personel "kaydetti", mutfak "gormedi" senaryosu restoran icin kritik operasyonel risktir.
- Kok neden: Write model ve outbox/side effect modeli ayrilmamis.
- Oneri: Order/item status guncellemeleri ana transaction icinde yapilmali. Yazici/caller side effect'leri icin transaction icinde outbox/job kaydi olusturulmalı, harici isleme sonradan guvenli retry ile calismali.

P2 - Generic table status endpoint invariant bozabilir

- Kanit: `PATCH /api/tables/:id/status`, status'u `current_order_id` ile tutarlilik kontrolu yapmadan update ediyor (`server/routes/tables.js:59-73`).
- Etki: Masa `occupied` ama `current_order_id` null, veya masa `empty` ama aktif order masaya bagli gibi UI ve raporlamayi bozan durumlar olusabilir.
- Kok neden: Masa status'u domain action yerine serbest alan guncellemesi gibi modellenmis.
- Oneri: Generic status patch kisitlanmalı. `reserveTable`, `clearReservation`, `seatGuests`, `closeTable` gibi domain endpoint'leri ya da en azindan status bazli invariant guard'lari getirilmeli.

P2 - Delivered retry idempotency sirasinda kapali siparis erken 400 donuyor

- Kanit: Paket delivery endpoint'i once `closed/cancelled` kontrolu yapar (`server/routes/orders.js:190-192`), sonra `takeaway_delivered_at` idempotency cevabina bakar (`server/routes/orders.js:193-197`).

- Etki: Basarili teslimden sonra ayni istek tekrar gelirse kullanıcı "Siparis kapali" hatasi alabilir. Veri bozulmaz; operasyonel guven hissi zedelenir.
- Kok neden: Terminal status guard'i idempotent sonuc kontrolunden once calisiyor.
- Oneri: `takeaway_delivered_at && action === 'delivered'` kontrolu closed guard'dan once ele alinmali.

P2 - Payment type kontrati iki farkli yerde farkli

- Kanit: DB `cash`, `card`, `mixed`, `other` kabul ediyor (`server/migrations/run.js:242`), public payment API sadece `cash`, `card` kabul ediyor (`server/routes/payments.js:16-18`), paket otomatik odeme ise backend icinde `other` uretiyor.
- Etki: Raporlama, UI etiketi ve API dokumantasyonunda "other" odemenin anlami belirsiz kalabilir. Otomatik paket kapanislarinda ciro tipi yanlis yorumlanabilir.
- Kok neden: Payment type semantigi domain seviyesinde belgelenmemis; API ve internal flow ayrimi net degil.
- Oneri: `other` sadece sistem kaynakli odeme ise `source='system_takeaway_delivery'` gibi ayri alan eklenmeli veya note/source ile raporlarda ayrismali. Public API enum'u ve DB enum'u bilincli olarak dokumante edilmeli.

5. Transaction ve veri tutariligi riskleri

P1 - Item update + totals recalculation atomik degil

- Kanit: Item update yapiliyor (`server/routes/orders.js:688-690`), ardindan `recalcOrderTotals` ve `autoCancelOrderIfNoActiveItems` transaction disinda ayri cagriliyor (`server/routes/orders.js:693-694`).
- Etki: Update basarili, toplam recalculation basarisiz olursa order total eski kalabilir. Cancel edilen son urun sonrasi order auto-cancel eksik kalabilir.
- Kok neden: "Satir guncelle + toplam hesapla + terminal durum kontrolu" tek domain command degil.
- Oneri: Bu uc adim tek transaction icine alinmali; failure halinde tum islem rollback olmalı.

P1 - Payment double submit finansal veri tutarililigini bozar

- Kanit: Payment insert tekil request kimligiyle korunmuyor (`server/routes/payments.js:209-218`).
- Etki: Iki payment kaydi, iki audit, muhtemelen iki fis/job. Geri alma veya muhasebe duzeltmesi gerekir.
- Kok neden: API idempotency tasarimi eksik.
- Oneri: Payment ve split payment endpoint'leri icin idempotency zorunlu hale getirilmeli.

P1 - Siparis kaydi ile mutfak/yazici job garanti modeli zayif

- Kanit: `txn()` bittikten sonra `enqueueTakeawayLabelJob`, `linkCallLogToOrder`, `finalizeKitchenForNewItems` calisiyor (`server/routes/orders.js:442-452`).
- Etki: Siparis kayitli ama mutfak job'u yok; Caller ID order'a baglanmamis; paket etiketi basilmamis.
- Kok neden: Transactional outbox pattern yok.
- Oneri: DB transaction icinde islenecek job/outbox kaydi olusturulmali. Harici islem retry edilebilir worker/bridge tarafina birakilmali.

P2 - Print mock production'da yanlış config ile sahte başarı üretebilir

- Kanıt: `processPendingJobsSync` config kapalı değilse pending jobları otomatik `printed` yapıyor (`server/services/printJobs.js:592-610`).
- Etki: Yazıcıya gitmeyen adisyon/mutfak fişi printed görünebilir.
- Kök neden: Test/dev kolaylığı ile production davranışı aynı servis içinde.
- Öneri: Production'da mock processor hard-disabled olmalı; config startup'ta loglanmalı ve env validation ile güvenceye alınmalı.

P2 - Status geçişleri merkezi değil

- Kanıt: Order status, item status, table status farklı route bloklarında if/else ile yönetiliyor (`server/routes/orders.js:536-620`, `server/routes/orders.js:632-696`, `server/routes/tables.js:59-73`).
- Etki: Bir ekranda engellenen geçiş başka endpoint'te mümkün olabilir. Test kapsamı büyüdükçe kurallar kopyalanır.
- Kök neden: Domain transition matrix kodda tek kaynak değil.
- Öneri: `domain/orderLifecycle`, `domain/paymentPolicy`, `domain/tablePolicy` gibi saf fonksiyonlardan oluşan merkezi guard katmanı kurulmalıdır.

6. Amator görünen backend alanları

P1 - Route dosyaları domain servis gibi davranmaya başlamış

`server/routes/orders.js` hem validation, hem fiyat çözme, hem DB write, hem mutfak/yazıcı side effect, hem audit, hem socket emit yapıyor. Bu dosya restoran POS'un en kritik domainini taşıyor ama route handler seviyesinde buyumuş. Bu profesyonel ürün kalitesinde bakım riskidir.

Öneri: Önce düşük riskli olarak saf policy/helper dosyaları çıkarılmalı; sonra route handler'lar "request -> command -> response" seviyesine indirilmeli.

P1 - Backend hata modeli tutarsız ve tanı koydurmuyor

Birçok catch bloğu `Sunucu hatası` donuyor; bazı business hataları plain Turkish string, bazı teknik hatalar DB constraint kaynaklı 500 olabilir. Örnek: tables update catch'i sadece generic 500 doner (`server/routes/tables.js:79-81`). Bridge tarafında daha iyi `claim_failed` gibi kodlu hata var (`server/routes/bridge.js:339-342`).

Öneri: `{ code, message, details?, correlation_id }` formatına geçilmeli. Kullanıcı mesajı ile teknik hata kodu ayrılmalı.

P1 - UI'a güvenen validation kalıntıları var

Order item patch, full payment overpay ve table status patch bu kategoride. UI bug'i veya manuel API çağırışı direkt veri kalitesine zarar verebilir.

Öneri: Frontend hiç yokmuş gibi backend domain guard yazılmalı.

P2 - Payment type semantigi urun diliyle net degil

`other` otomatik paket teslim odemesi icin kullaniliyor, DB `mixed` destekliyor, public API sadece `cash/card` kabul ediyor. Bu, ileride rapor ekranlarinda "diger odeme" siserek isletmecinin nakit/kart ciro takibini bozar.

Oneri: Otomatik teslim kapatma icin ayri source alanlari ve rapor etiketi eklenmeli.

P2 - Mock printer log'u ve Unicode ikonlu console ciktilari backend servisinde duruyor

`processPendingJobsSync` icinde console log var (`server/services/printJobs.js:606-608`). Bu dev/test icin faydali olabilir; fakat production log standardi, structured logging ve encoding acisindan profesyonel degil.

Oneri: Structured logger kullanilmali; mock path sadece test/dev konfigrasyonunda aktif olmalı.

7. Hemen ele alınmasi gerekenler

P1 - Payment idempotency

Odeme endpoint'leri duplicate submit'e karsi korunmalı. Bu, finansal veri tutarlıligi icin en yuksek oncelikli backend hardening maddesidir.

Minimum profesyonel cozum:

- `payments` tablosuna nullable `idempotency_key` ve `source` alanlari ekle.
- `business_id + order_id + idempotency_key` unique index ekle.
- Ayni key tekrar gelirse yeni payment yaratma; mevcut payment'i don.
- Split payment icin allocation hash veya client operation id zorunlu yap.

P1 - Full payment remaining guard

Normal odemede kalan tutar asimi engellenmeli veya tip/bahsis modeli acikca ayrilmali.

Minimum cozum:

- `getPaymentTotals(order_id)` helper'i full payment endpoint'inde kullan.
- `amount > remaining_total + 0.02` ise 400 don.
- Hata kodu: `PAYMENT_AMOUNT_EXCEEDS_REMAINING`.

P1 - Order/item validation

Order item patch icin zod schema ve transition guard eklenmeli.

Minimum cozum:

- `quantity`: int, min 1, max makul limit.
- `status`: enum ve izinli gecis.
- `is_comped` ve `status='comped'` icin tek kaynak karari.
- Satir degisimi + total recalculation tek transaction.

P1 - API/DB order_type uyumu

`delivery` kabul edilecekse DB ve domain tam desteklemeli; edilmeyecekse schema'dan kaldirilmali.

V1 icin daha guvenli secenek: API enumunu `dine_in | takeaway` ile sinirlamak.

P1 - Transactional outbox / mutfak finalize siniri

Siparis kaydi ile mutfak/yazici/caller side effect'leri arasindaki bosluk kapatilmali.

Minimum cozum:

- Order/item status finalize DB transaction icine alinmali.
- Print job insert'leri transaction icinde outbox kaydi olarak garanti edilmeli.
- Harici yazdirma islemi retry edilebilir kalmali.

8. Sonraki turda uygulanabilecek dusuk riskli duzeltmeler

1. Status endpoint validation'lari

- `PATCH /api/orders/:id/status` icin zod enum.
- `PATCH /api/orders/:orderId/items/:itemId` icin zod schema.
- `PATCH /api/tables/:id/status` icin enum + invariant guard.

Risk: Dusuk/orta. Client'in gecersiz request gonderdigi gizli yerler varsa ortaya cikar; bu iyi bir kirilma olur.

2. Paket delivered retry cevabini idempotent hale getirme

- `takeaway_delivered_at && action === 'delivered'` kontrolunu closed guard'dan once ele al.

Risk: Dusuk. Veri davranisi degismez, tekrar cagrida daha dogru API cevabi doner.

3. Full payment overpay guard

- Split payment'taki kalan bakiye guard mantigi full payment endpoint'ine tasinir.

Risk: Orta. Mevcut kullanimda bilincli fazla odeme giriliyorsa engellenir; fakat tip modeli olmadigi icin bu daha dogru davranistir.

4. Payment source etiketi

- Otomatik paket teslim odemelerine `source` / `note` standardi ver.
- Raporlarda "Paket teslim otomatik kapanis" olarak ayir.

Risk: Dusuk/orta. Migration ve rapor UI etkisi var.

5. Print mock env guard

- Production ortaminda `disablePrintJobMock=true` zorunlu hale getir.
- Startup validation ile aksi halde uygulamayi baslatma veya sert uyari ver.

Risk: Dusuk. Yazici entegrasyon testleri icin env ayrimi netlesir.

6. Route dosyalarindan saf domain helper'lari cikarma

Ilk refactor parcalari dusuk riskli olabilir:

- `paymentPolicy.js` : remaining total, close eligibility, overpay guard.
- `orderLifecycle.js` : status transition guard'lari.

- `tablePolicy.js`: table invariant guard'lari.
- `printOutboxPolicy.js`: job idempotency helper'lari.

Risk: Orta. Davranis degistirmeden test esliginde tasinmali.

Ozet onceliklendirme

P0

Bu turda kanitli P0 bulgu yok. Uygulamada finansal ve operasyonel P1 riskler var; fakat anlik veri kaybi veya tum sistemi durduran kanitli bir P0 tespit edilmedi.

P1

1. Odeme endpoint'lerinde idempotency yok.
2. Normal odeme kalan bakiye asimini engellemiyor.
3. Order item patch validation ve transition guard eksik.
4. API `delivery` order type kabul ediyor, DB etmiyor.
5. Siparis kaydi ile mutfak/yazici/caller side effect'leri atomik garanti altinda degil.
6. Item update + total recalculation transaction icinde degil.

P2

1. Generic table status endpoint masa invariant'larini bozabilir.
2. Paket delivered retry idempotency sirasi kullaniciya hatali 400 dondurebilir.
3. Payment type/source semantigi raporlama icin belirsiz.
4. Print mock production config riski tasiyor.
5. Status transition'lar merkezi degil.
6. Hata modeli kodlu, tani koydurabilir ve tutarli degil.

Son karar

Backend bugunku haliyle temel POS akislarini tasiyor ve son turdaki odeme/masa/paket guard'lari urun risklerini azaltti. Ancak profesyonel urun kalitesi icin en kritik eksik, backend'in "UI dogru davranir" varsayimini tamamen terk etmemis olmasi. Odeme idempotency, kalan bakiye guard'i, item validation ve transaction sinirlari cozulmeden sistem yogun restoran operasyonunda hatayi kullanıcı aliskanligina, ag kosullarina ve hizli dokunmatik kullanimina fazla acik birakiyor.

Uygulanan İyileştirmeler Raporu

Tarih: 2026-04-14 Kapsam: Audit raporlarında belirlenen ürün, UX/UI ve frontend mimarisi eksiklerine göre uygulanan kalıcı iyileştirmeler. Hedef okuyucu: Ürün sahibi, işletmeci, operasyon yöneticisi ve teknik ekip.

1. Yönetici Özeti

Bu çalışmada restoran POS uygulamasında daha önce raporlanan sorunlar içinden kullanıcıya doğrudan yansıyan ve ürün kalitesini düşüren alanlar ele alındı. Amaç hızlı bir geçici düzeltme yapmak değil; ödeme, sipariş, masa ve ayarlar ekranlarında daha tutarlı, güvenilir ve sürdürülebilir bir temel oluşturmaktır.

Alan	Önceki Durum	Yeni Durum	Kullanıcıya Etkisi
Ödeme kuralları	Farklı ekranlarda ayrı ayrı hesaplanıyordu	Tek merkezden yönetiliyor	Ödeme, hızlı ödeme ve masa durumları daha tutarlı
Sipariş aksiyonları	Kaydetme, ödeme açma ve satır düzenleme koşulları ekran içine dağılmıştı	Ortak karar fonksiyonlarına taşındı	Hatalı veya zamansız aksiyon riski azaldı
Onay pencereleri	Bazı yerlerde tarayıcıya ait kaba onay kutuları kullanılıyordu	Uygulama içi ortak onay penceresi kullanılıyor	Daha profesyonel ve daha anlaşılır deneyim
Ayarlar ekranları	Çıkış, silme, pasifleştirme gibi kritik işlemler tutarsızdı	Aynı onay standardına bağlandı	Yönetim tarafında güven ve tutarlılık arttı
API altyapısı	HTTP iletişimi ve domain endpointleri aynı dosyada büyüyordu	HTTP çekirdeği ayrıldı	Sonraki geliştirmeler için daha sağlam temel oluştu

2. Neler İyileştirildi?

2.1 Ödeme ve Masa Durumu Daha Güvenilir Hale Getirildi

Ödeme ekranı, hızlı ödeme ekranı ve masa ekranı daha önce "bu sipariş ödendi mi?", "ne kadar kaldı?", "masa kapatılabilir mi?" gibi soruları kendi içinde ayrı ayrı cevaplıyordu. Bu durum ileride bir ekranın doğru, diğerinin yanlış davranmasına yol açabilecek bir riskti.

Yeni yapıda ödeme durumu tek merkezden hesaplanıyor.

İyileştirme	Açıklama	Kazanç
Toplam ödeme hesabı ortaklaştırıldı	Sipariş toplamı, ödenen tutar ve kalan tutar aynı yardımcı yapıdan geliyor	Ekranlar arası tutarsızlık riski azaldı
Tam ödeme kontrolü ortaklaştırıldı	"Tam ödendi" kararı tek yerden veriliyor	Masa kapatma ve hızlı ödeme davranışı daha güvenli
Paket sipariş ödeme etiketi ortaklaştırıldı	Ödendi, kısmi ödendi, ödenmedi gibi durumlar aynı mantıkla gösteriliyor	Paket ve masa siparişi dili daha tutarlı

Etkilenen kullanıcı deneyimi:

- Kasiyer hızlı ödeme yaptıktan sonra masa durumunun yanlış görünme riski azaldı.

- Paket siparişlerde ödeme etiketi daha tutarlı hale geldi.
- Masa kapatma gibi kritik işlemler ödeme durumuyla daha net bağlandı.

3. Sipariş Ekranında Karar Mantığı Toparlandı

Sipariş ekranı POS uygulamasının en yoğun kullanılan alanı. Bu ekranda "kaydet", "ödeme al", "ürün düzenle", "ürün iptal et" gibi kararlar daha önce ekran kodunun içine dağılmış durumdaydı.

Bu turda bu kararların bir kısmı merkezi ve okunabilir kurallara taşındı.

Kural	Önceki Risk	Yeni Yaklaşım
Sipariş kaydedilebilir mi?	Sepet boşken veya paket siparişte müşteri yokken dağınık kontroller vardı	Tek karar fonksiyonuyla kontrol ediliyor
Ödeme ekranı açılabilir mi?	Kaydedilmemiş ürün varken ödeme açma riski ekran içine gömülüydü	Merkezi aksiyon kuralına taşındı
Ürün satırı düzenlenebilir mi?	Satır durumu ve sipariş durumu kontrolleri dağınıktı	Ortak policy fonksiyonuyla yönetiliyor
Ürün iptal edilebilir mi?	Yetki ve satır durumu kontrolleri ekranda karışıyordu	Daha net bir karar katmanına alındı

Bu iyileştirme kullanıcıya nasıl yansır?

- Kaydedilmemiş ürün varken yanlışlıkla ödeme akışına girme riski azalır.
- Kapalı siparişlerde yanlış düzenleme aksiyonları daha güvenli engellenir.
- Paket siparişlerde müşteri zorunluluğu daha anlaşılır şekilde korunur.

4. Uygulama İçi Onay Deneyimi Standartlaştırıldı

Audit raporlarında "amatör görünen alanlar" içinde en net sorunlardan biri native tarayıcı onay kutularıydı. Bunlar görsel olarak uygulamanın geri kalanından kopuk, açıklama açısından zayıf ve yoğun POS kullanımında güven vermeyen bir deneyim oluşturuyordu.

Bu turda ortak bir onay penceresi ve onu yöneten ortak kullanım yapısı eklendi.

Nerede Kullanılıyor?	Örnek Aksiyon	Yeni Davranış
İşletme ayarları	Kaydetmeden çıkış, değişiklikleri atma	Açıklamalı uygulama içi onay
Ekran ayarları	Kaydetmeden çıkış	Açıklamalı uygulama içi onay
Menü yönetimi	Kategori silme, ürün kaldırma, toplu pasifleştirme	Standart tehlikeli işlem onayı
Yazıcı ayarları	Kaydetmeden çıkış, varsayılanlara dönme, pasifleştirme	Daha kontrollü onay akışı
Kullanıcı yönetimi	Kullanıcı pasifleştirme	Net etki açıklamasıyla onay
Rezervasyonlar	Rezervasyon silme	Standart silme onayı
Stok	Stok kalemi silme	Standart silme onayı
Salon bölgeleri	Bölge silme	Risk açıklamalı onay

Kullanıcı açısından fark:

- Kritik işlemlerde "ne olacak?" sorusu daha net cevaplanıyor.
- Uygulama artık tarayıcı penceresi gibi değil, kendi içinde tutarlı bir ürün gibi davranıyor.
- Silme, pasifleştirme ve çıkış işlemleri daha güvenli hissediliyor.

5. Ayarlar ve Yönetim Ekranlarında Profesyonellik Artırıldı

Ayarlar ekranları işletme sahibi veya yönetici tarafından kullanılıyor. Bu alanlarda küçük tutarsızlıklar bile ürüne olan güveni azaltır. Yapılan çalışma özellikle şu hissi iyileştirmeyi hedefledi:

Önceki İzlenim	Yeni İzlenim
Bazı işlemler tarayıcı onayıyla, bazıları uygulama modaliyle ilerliyordu	Tüm kritik onaylar aynı ürün standardıyla ilerliyor
Silme/pasifleştirme aksiyonları bazen az açıklamalıydı	İşlemin sonucu açıkça yazılıyor
Ayarlar ekranları birbirinden kopuk davranıyordu	Ortak onay altyapısı ile davranış standardı başladı

Bu, özellikle kurulum sonrası işletme ayarları, yazıcı ayarları, kullanıcı yönetimi ve menü yönetimi gibi alanlarda daha profesyonel bir ürün hissi sağlar.

6. API Altyapısında Sağlamaştırma Yapıldı

Kullanıcının günlük kullanımda görmediği ama ürünün uzun vadeli kalitesini etkileyen bir altyapı iyileştirmesi de yapıldı.

Önceden API dosyası hem sunucuyla konuşma mantığını hem de tüm ürün endpointlerini aynı yerde taşıyordu. Bu yapı büyüdükçe bakım zorlaşıyor, test etmek ve domainlere ayırmak riskli hale geliyordu.

Bu turda HTTP iletişim çekirdeği ayrı bir dosyaya çıkarıldı. Mevcut ekranların kullandığı `api` arayüzü korunarak davranış kırılmadı.

Değişiklik	Neden Önemli?
HTTP/token/request çekirdeği ayrıldı	Gelecekte sipariş, masa, ödeme, yazıcı gibi API grupları daha rahat ayrılabilir
Mevcut <code>api.*</code> kullanımı korundu	Ekranlarda kırıcı değişiklik yapılmadı
Davranış korunarak mimari sınır açıldı	Büyük refactorlara daha güvenli geçiş zemini oluştu

7. Temizlenen Amatör İzler

Bu turda özellikle ürün kalitesini düşüren küçük ama görünür izler temizlendi.

Alan	Temizlenen Sorun
Native onay kutuları	<code>window.confirm</code> kullanımları kaldırıldı
Debug çıktıları	Yazıcı keşif ekranındaki gereksiz <code>console.log</code> çıktıları kaldırıldı
Tekrar eden ödeme hesabı	Ödeme hesapları ortak yapıya taşındı
Dağınık sipariş kararları	İlk karar katmanı sipariş policy dosyasına alındı
API monolit başlangıcı	HTTP çekirdeği servis dosyasından ayrıldı

8. Doğrulama Sonuçları

Uygulanan değişikliklerden sonra temel doğrulamalar çalıştırıldı.

Komut	Sonuç	Açıklama
<code>npm run build</code>	Başarılı	Frontend production build sorunsuz tamamlandı
<code>npm test</code>	Başarılı	Server testlerinde 12 test dosyası, 108 test geçti
<code>npm run lint --if-present</code>	Çalıştı, çıktı üretmedi	Repo içinde lint scripti tanımlı değil
Statik arama	Temiz	<code>window.confirm</code> ve debug <code>console.log</code> kalmadı

9. Bu Çalışmanın Ürün Değeri

Değer	Açıklama
Daha az hata riski	Ödeme ve sipariş kuralları tek merkezden yönetilmeye başlandı
Daha profesyonel görünüm	Kritik onaylar artık uygulamanın kendi tasarım diliyle gösteriliyor
Daha güvenli operasyon	Silme, pasifleştirme, çıkış ve sıfırlama işlemleri daha açıklamalı hale geldi
Daha kolay bakım	Büyük dosyaları parçalamaya başlamadan önce güvenli ortak katmanlar kuruldu
Daha iyi test zemini	Saf yardımcı fonksiyonlar ileride kolayca unit test kapsamına alınabilir

10. Kalan Açık Alanlar

Bu tur kalıcı ve sağlam bir temel attı; ancak daha büyük frontend mimari iyileştirmeleri kontrollü şekilde sonraki turlara bırakılmalı.

Kalan Alan	Neden Hemen Tamamlanmadı?	Önerilen Sonraki Adım
OrderScreen parçalama	Çok yoğun ekran; tek hamlede bölmek regresyon riski doğurur	Katalog, sepet, müşteri ve işlem hookları ayrı ayrı çıkarılmalı
TablesScreen parçalama	Masa, paket, transfer, caller ID ve refresh akışı aynı ekranda	useTablesData , TakeawaySidebar , TableCard ayrımı yapılmalı
PrinterDetailPage parçalama	Yazıcı formu, keşif, önizleme ve encoding aynı dosyada	Form modelini hook'a, önizlemeyi ayrı bileşene almak
Test altyapısı	Frontend için özel test scripti yok	Utility unit testleri ve Playwright smoke testleri eklenmeli
Lint standardı	Repo lint scripti tanımlı değil	ESLint/format kontrolü standart build pipeline'a eklenmeli

11. Sonraki Geliştirme Planı

Hemen Devam Edilecekler

Öncelik	İş	Beklenen Kazanç
P1	Sipariş ekranını küçük hooklara ayırma	Daha az kırılgan sipariş geliştirmesi
P1	Masa ekranı veri yenileme mantığını merkezi hook'a alma	Socket/polling/manuel refresh karmaşasını azaltma
P1	Frontend utility testleri ekleme	Ödeme ve sipariş kurallarını otomatik koruma

Bu Hafta Ele Alınacaklar

Öncelik	İş	Beklenen Kazanç
P2	Yazıcı detay ekranını form ve önizleme parçalarına ayırma	Yazıcı ayarlarında bakım kolaylığı
P2	API servislerini domain gruplarına bölmeye devam etme	Daha temiz entegrasyon mimarisi
P2	Settings shell / ortak ayarlar layout'u	Yönetim ekranlarında daha tutarlı ürün hissi

Sonraya Bırakılacaklar

Öncelik	İş	Gerekçe
P2	React Query veya hafif cache katmanı	Daha geniş veri mimarisi kararı gerektirir
P2	Route state yerine URL/server hydrate modeli	Davranış etkisi yüksek, planlı yapılmalı
P2	Inline style borcunun tasarım sistemine taşınması	Görsel regresyon testiyle ilerlemeli

12. Kısa Sonuç

Bu turda uygulamanın görünmeyen ama güvenilirliği belirleyen temeli güçlendirildi. Ödeme, sipariş ve masa davranışları daha merkezi hale geldi; yönetim ekranlarındaki kritik onaylar profesyonel bir standarda kavuştu; API tarafında daha temiz bir mimari sınır açıldı.

Ürün artık bir sonraki geliştirme turuna daha sağlam bir zeminden devam edebilir. En doğru devam adımı, büyük ekranları küçük ama güvenli parçalara ayırmak ve bu kuralları otomatik testlerle koruma altına almak olacaktır.

05 - Database / Transactional Systems Denetimi

Denetim tarihi: 2026-04-14 Kapsam: tablo semasi, migration disiplini, tarihsel veri guvenligi, tenancy, siparis/odeme iliskileri, urun/kategori/yazici eslemeleri, print job ve musterisi verisi.

Son uygulama turunda guvenli ve uygulanabilir iyilestirmeler koda alindi: tarihsel snapshot kolonlari, snapshot backfill, indeks dostu tarih sorgulari, seed production guard'i, yeni kurulumlar icin CHECK constraint'leri ve raporlari snapshot verisine tasinmasi uygulandi.

Veri modeli ozeti

Uygulama SQLite uzerinde tek migration dosyasi ile calisiyor: [server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js). Model cok kiracili bir yapiya hazir: neredeyse tum ana tablolarda `business_id` var ve route'lar `req.businessId` ile scope uyguluyor.

Ana tablolar:

- `businesses`, `branches`, `roles`, `users`: tenant, sube ve kullanıcı yetki temeli.
- `dining_areas`, `tables`: masa ve salon modeli.
- `categories`, `products`, `product_portions`, `product_modifiers`, `product_combos`: canlı menu tanımları.
- `orders`, `order_items`, `payments`, `payment_allocations`: tarihsel işlem kaydı.
- `printers`, `printer_routing`, `print_jobs`: yazıcı tanımı, kategori-yazıcı eşlemesi ve job geçmişi.
- `customers`, `customer_phones`, `customer_addresses`, `call_logs`: müşteri ve Caller ID bağlamı.

Sema genel olarak pratik POS akisini tasiyor; ancak profesyonel urun kalitesi acisinden en onemli ayrim, "canli tanim verisi" ile "tarihsel islem verisi" arasinda tam olarak tamamlanmamis. Siparis kalemi urun adi/fiyati snapshot'liyor; fakat kategori, vergi, yazici hedefi ve bazi raporlar hala canli menu tanimlarina bakiliyor.

Guclu yonler

1. Siparis satirinda temel urun snapshot'i var

`order_items` tablosu `product_id` ile canli urune bagli kalirken, `product_name`, `quantity`, `unit_price`, `modifiers`, `vat_rate`, `portion_id`, `portion_label` gibi islem anindaki kritik alanlari da sakliyor ([server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):217).

Etki: Urun adi veya fiyat degisse bile gecmis adisyonda temel tutar ve urun adi korunuyor.

2. Urun ve kategori silme hard delete degil

Urun silme `DELETE` yerine `is_deleted = 1`, `is_active = 0` yapiyor ([server/routes/products.js](D:/dev/restoran-pos-v3/server/routes/products.js):380). Kategori silme de kategoriye pasife aliyor ve alt urunleri pasif/silinmis isaretliyor ([server/routes/categories.js](D:/dev/restoran-pos-v3/server/routes/categories.js):102).

Etki: Gecmis siparislerdeki `product_id` FK baglari hard delete ile kopmuyor.

3. Odeme idempotency icin veri modeli guclendirildi

Son backend hardening ile `payments` tablosuna `idempotency_key` ve `source` alanlari eklendi ([server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):250). Existing DB icin kolon ekleme

ve unique index kurulumu migration icinde garantiye alindi ([server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):595).

Etki: Odeme tekrar istegi finansal kaydi iki kez olusturmayacak sekilde veri modelinde de destekleniyor.

4. Print job idempotency var

`print_jobs.idempotency_key` UNIQUE tanimli ([server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):301). Bu, mutfak/fis/paket etiketi job'larinda duplicate yazdirma riskini azaltan dogru bir karar.

5. Business scope icin temel indeksler mevcut

`users`, `tables`, `orders`, `products`, `categories`, `customers`, `payments`, `print_jobs` gibi ana tablolarda business veya iliski odakli indeksler var ([server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):343).

Tarihsel veri riskleri

P1 - Kategori bazli raporlar canli urun/kategori tanimina bagli

Kanit: Raporlarda kategori kirdirimi `order_items -> products -> categories` join'i ile hesaplanıyor ([server/routes/reports.js](D:/dev/restoran-pos-v3/server/routes/reports.js):43). `order_items` icinde `category_id` veya `category_name` snapshot'i yok ([server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):217).

Etki: Bir urun sonradan baska kategoriye tasinirse veya kategori yeniden adlandirilirse, gecmis satislar bugunku kategoriye gore raporlanir. Bu restoran sahibi icin tarihsel ciro analizini bozabilir.

Kok neden: Urun snapshot'i var ama kategori snapshot'i yok.

Uygulandi: `order_items` icine `category_id_snapshot`, `category_name_snapshot`, `printer_target_snapshot` eklendi. Yeni siparislerde bu alanlar order create/add-items sirasinda dolduruluyor; mevcut kayitlar icin best-effort backfill calisiyor.

P1 - Yazici routing tarihsel olarak snapshot'lanmiyor

Kanit: Canli urun/kategori yazici hedefleri `products.printer_target` ve `categories.printer_target` alanlarindan geliyor ([server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):98, [server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):114). Siparis satirinda o anki yazici hedefi saklanmiyor.

Etki: Kategori veya urunun yazici hedefi degisirse, gecmis order item'dan tekrar mutfak/duzeltme job'u uretilirken bugunku routing ile tarihsel routing karisabilir.

Kok neden: Yazici hedefi canli tanim verisi olarak kalmis, order item transaction snapshot'ina alinmamis.

Uygulandi: `printer_target_snapshot` eklendi ve yeni siparis satirlarinda urun/kategori routing bilgisinden donduruluyor.

P2 - Vergi ve servis ucreti modeli tutar guvenligi icin yetersiz

Kanit: `orders` tablosunda `subtotal`, `discount_amount`, `discount_percent`, `service_charge`, `vat_total`, `grand_total` alanlari REAL olarak duruyor ([server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):199). `order_items.vat_rate` var ama create akisi `vat_rate` ve `vat_total` icin fiilen 0 yaziyor.

Etki: Gelecekte KDV, servis bedeli veya yuvarlama kurallari ciddiye alindiginda eski siparislerin hangi kural setiyle hesaplandigi anlasilmayabilir.

Kok neden: Fiyat hesaplama policy versiyonu ve para birimi/tutar minor unit modeli yok.

Oneri: V2 icin `currency`, `pricing_policy_version`, `subtotal_cents`, `grand_total_cents` gibi integer minor-unit modeline gecis planlanmalı. Kisa vadede REAL kalacaksa tum hesaplar merkezi `round2` ile korunmalı ve audit testleri artırilmali.

P2 - Kullanici, musteriler ve masa isimleri tarihsel belgelerde kismen canli okunuyor

Kanit: `Orders` `user_id`, `customer_id`, `table_id` FK sakliyor ([server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):195). Bazı query'ler kullanıcı/ masa/musteri adini canli join ile aliyor.

Etki: Garson adi, masa adi veya musteriler adi degisirse eski rapor ekrani yeni adla gorunebilir. Finansal tutar bozulmaz, ama tarihsel operasyon raporu zayıflar.

Uygulandi: `orders.table_name_snapshot`, `orders.user_name_snapshot`, `orders.customer_name_snapshot` alanlari eklendi ve mevcut kayıtlar icin backfill calistirildi. Musteriler sonradan siparise baglandiginda snapshot da guncelleniyor.

Silme / update riskleri

P1 - Foreign key davranislari ON DELETE politikasiz

Kanit: Çok sayıda FK `REFERENCES` ile tanımlı, fakat cogu `ON DELETE` davranisi belirtmiyor. Ornek: `orders.table_id REFERENCES tables(id)`, `order_items.product_id REFERENCES products(id)`, `payments.order_id REFERENCES orders(id)` ([server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):195, [server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):220, [server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):241).

Etki: SQLite foreign key enforcement aktifse hard delete denemeleri patlar; aktif degilse orphan kayıt riski dogar. Uygulama tarafında soft delete kullanimi bu riski azaltiyor ama sema seviyesinde politika net degil.

Kok neden: Referential action standardi belirlenmemis.

Oneri: İşlem tabloları icin hard delete yasaklanmalı. Tanım tablolarında soft delete standart olmalı. Gerçek `ON DELETE` davranisi yalnızca lookup/child cleanup icin bilincli kullanılmalı; ornegin `product_portions` icin `ON DELETE CASCADE` zaten var ([server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):137).

P2 - Seed script production icin tehlikeli davranis tasiyor

Kanit: Seed script baslarken çok sayıda tabloyu `DELETE FROM` ile temizliyor ([server/seeds/run.js](D:/dev/restoran-pos-v3/server/seeds/run.js):12).

Etki: Yanlis ortamda calistirilirs canli veri silinebilir.

Uygulandi: Seed script production ortaminda `--force` verilmeden calismayacak sekilde guvenceye alindi.

P2 - Product/category update tarihsel raporlari dolayli etkiliyor

Kanit: Product update `name`, `price`, `category_id`, `printer_target` gibi canli alanlari degistirebiliyor ([server/routes/products.js](D:/dev/restoran-pos-v3/server/routes/products.js):249). Order item temel tutari korusa da kategori/yazici hedefi snapshot'i eksik.

Etki: Urun adi ve fiyat gecmis adisyonu bozmaz; kategori/yazici raporlari bozabilir.

Uygulandi: Raporlardaki kategori kirilimi canli kategori join'i yerine order item snapshot alanlarini kullanacak sekilde tasindi.

Parasal alanlar

P1 - REAL para alani uzun vadeli risk

Kanit: `products.price`, `orders.subtotal`, `orders.grand_total`, `payments.amount`, `payment_allocations.line_total` REAL olarak saklaniyor ([server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):110, [server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):199, [server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):246, [server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):264).

Etki: Kucuk yuvarlama farklari, split payment ve gun sonu raporlarinda birikebilir. Mevcut kod `round2` ile pratik tolerans uyguluyor; fakat database modeli profesyonel finansal sistem icin ideal degil.

Oneri: V2 migration planinda integer minor unit yani kurus bazli alanlara gecilmeli. Kisa vadede yeni para alanlari icin CHECK constraint ve merkezi rounding testleri eklenmeli.

P2 - Tutar CHECK constraint'leri eksik

Kanit: `payments.amount REAL NOT NULL` ama `CHECK(amount > 0)` yok ([server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):246). `products.price REAL NOT NULL` ama DB seviyesinde positive constraint yok ([server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):110).

Etki: API validation atlanirsa negatif/sifir tutar DB'ye girebilir.

Uygulandi: Yeni kurulum semasinda kritik numeric alanlara CHECK constraint eklendi. Mevcut SQLite tablolarinda constraint eklemek tablo recreation gerektirdigi icin canli veri uzerinde zorlayici tablo yeniden olusturma yapilmedi.

Performans ve indeks notlari

Bu turda dusuk riskli indeks iyilestirmeleri uygulandi:

- `tables.current_order_id`: masa karti ve aktif order lookup'lari icin ([server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):347).
- `orders(business_id, created_at)`: tarihsel siparis listesi ve rapor filtreleri icin ([server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):352).
- `orders(business_id, status, created_at)`: aktif/kapanmis siparis listeleri icin ([server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):353).

- `orders(business_id, closed_at)` : kapanis bazli raporlar icin ([server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):354).
- `order_items(product_id)` : urun bazli analiz ve FK lookup icin ([server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):356).
- `products(business_id, is_active, is_deleted)` : menu listesi icin ([server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):359).
- `payments(business_id, created_at)` ve `payments(business_id, payment_type, created_at)` : ciro/odeme tipi raporlari icin ([server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):366).
- `print_jobs(business_id, status, claimed_at, created_at)` : Bridge pending/unclaimed job sorgusu icin; kolon migration'dan sonra kuruluyor ([server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js):580).

Not: Rapor sorgularinda `date(created_at)` kullanimi indeks verimini sinirleyabilir. Daha profesyonel yaklasim, tarih araligini `created_at >= ? AND created_at < ?` ile sorgulamak veya ayrica `business_date` kolonu tutmaktir.

Veri modeli amatorluk belirtileri

1. Tek dosyada buyuyen migration

Tum sema ve sonradan eklenen migration yamalari tek [server/migrations/run.js](D:/dev/restoran-pos-v3/server/migrations/run.js) dosyasinda. `CREATE TABLE`, `ALTER TABLE`, backfill, tablo recreation ve seed benzeri default portion doldurma ayni dosyada.

Risk: Uretim migration sirasini izlemek, rollback planlamak ve hangi versiyonda hangi alanin geldigini anlamak zorlasiyor.

Oneri: Numarali migration dosyalari ve `schema_migrations` tablosu eklenmeli. `run.js` sadece migration runner olmalı.

2. Tarihsel snapshot standardi net degil

Order item urun adi/fiyati snapshot'liyor, ama kategori/yazici/masa/kullanici snapshot'i yok. Bu kısmi model "bazilari tarihsel, bazilari canli" gibi okunuyor.

Oneri: Her order create sirasinda hangi alanlar tarihsel belge kabul ediliyor, tek policy dokumani ve testleri olmalı.

3. Seed script destructive ve ortam guard'i zayif

Seed baslangicinda tablo temizleme var ([server/seeds/run.js](D:/dev/restoran-pos-v3/server/seeds/run.js):12). Bu demo icin pratik, production icin tehlikeli.

Oneri: `NODE_ENV=production` durumunda seed script calismamali; destructive mode icin explicit flag istemeli.

4. Para modeli DB seviyesinde yeterince sert degil

API guard'lari artirildi, fakat DB constraint'leri hala finansal sistem kadar kati degil.

Oneri: Yeni numeric alanlar icin CHECK; uzun vadede integer cents.

Onerilen iyilestirmeler

Hemen / dusuk risk

1. Uygulandi: Rapor sorgularinda `date(column)` filtreleri indeks dostu aralik filtrelerine tasindi.
2. Uygulandi: Seed script'e production guard ve `--force` parametresi eklendi.
3. Uygulandi: Yeni kurulum semasinda kritik numeric alanlara CHECK constraint eklendi; API validation zaten korunuyor.
4. Korundu: Printer routing unique index temizleme davranisi migration akisi icinde kaldı.

Bu hafta / orta risk

1. Uygulandi: `order_items` icine kategori ve yazici hedef snapshot alanlari eklendi.
2. Uygulandi: Order create/add-items sirasinda kategori/yazici snapshot dolduruluyor.
3. Uygulandi: Raporlardaki kategori breakdown snapshot alanlarına tasindi.
4. Kismen uygulandi: Seed script production guard ile guvenceye alindi; non-destructive setup modu ayri urunlestirme adimi olarak duruyor.

Sonraki faz / yuksek dikkat

1. Kismen uygulandi: Para alanlari icin yeni kurulumlarda CHECK constraint ve API validation sertlestirildi. Tam integer minor unit gecisi geriye donuk veri uyumlulugu gerektirdigi icin ayri kontrollu migration olarak kalmali.
2. Kalmali: Migration sistemini numarali migration runner'a tasima ayri refactor olarak duruyor.
3. Kalmali: Soft delete ve referential action politikasini domain dokumani ve DB constraint stratejisiyle netlestirme halen gerekli.
4. Kismen uygulandi: Tarihsel belge modeli snapshot alanlariyla guclendirildi; tam immutable fiscal snapshot sonraki fazda ele alinmali.

Uygulanmis duzeltmeler

Uygulanan duzeltmeler:

- Masa aktif siparis lookup indeksi.
- Order tarih/status/closed_at indeksleri.
- Order item product lookup indeksi.
- Order item kategori snapshot indeksi.
- Product active/deleted menu filtre indeksi.
- Payment tarih ve payment type rapor indeksleri.
- Print job bridge claim/pending indeksi.
- Order item kategori/yazici snapshot kolonlari ve backfill.
- Order header masa/musteri/kullanici snapshot kolonlari ve backfill.
- Order create/add-items akisini snapshot yazacak sekilde guncelleme.
- Gunluk kategori raporunu canli kategori yerine tarihsel snapshot verisine tasima.
- Rapor ve odeme ozeti tarih filtrelerini indeks dostu aralik sorgularina tasima.

- Seed script production guard'i.
- Yeni kurulum semasinda para/adet alanlari icin CHECK constraint'leri.

Canli veritabaninda migration basariyla calisti; 164 `order_items` satiri ve 91 `orders` satiri icin best-effort snapshot backfill uygulandi.

Son karar

Veri modeli restoran POS icin temel operasyonu tasiyor ve son uygulamalarla tarihsel raporlama guvenligi belirgin sekilde iyilesti. Kategori/yazici hedef snapshot'i artik korunuyor. Kalan ana teknik borclar: para alanlarinin REAL olarak tutulmasi, migration sisteminin tek dosyada buyumesi ve tam immutable fiscal snapshot modelinin henuz bulunmaması.